

FROM
SCRATCH

BUILD AN LLM Application

Hamza Farooq



MANNING

FROM
SCRATCH

BUILD AN LLM Application

Hamza Farooq

MEAP

 MANNING

Build an LLM Application (from Scratch)

1. [welcome](#)
2. [1 The World of Large Language Models](#)
3. [2 An in-depth look into the soul of the Transformer Architecture](#)
4. [3 Encoder models in action: Semantic-Based Retrieval Systems](#)
5. [4 Semantic Search from Scratch](#)
6. [5 Decoders in Action](#)

welcome

Dear Reader,

Thank you for purchasing the MEAP edition of *Build an LLM Application (from Scratch)*.

I'm excited to share this journey with you as we explore the intricacies of building search engines and Retrieval-Augmented Generation (RAG) systems—without relying on pre-built frameworks like Langchain or Llamaindex.

To get the most from this book, you should be familiar with general AI concepts and have some hands-on experience with Python. Prior knowledge of basic machine learning, and APIs will also help you dive deeper into the material and fully customize your applications.

Throughout this book, you'll learn how to build LLM-based applications from the ground up. We'll focus on the fundamentals of search systems, RAG solutions, and how to deeply customize each step of your AI pipelines. By the end, you'll be able to create LLM applications tailored to your specific needs, going beyond the limitations of existing frameworks. But we don't intend to stop there - we will also work with some advanced topics such as Fine-tuning, Semantic Caching, Knowledge Graphs and Agents.

I hope this book empowers you to unlock the full potential of LLMs and search technologies. Please feel free to share your thoughts and questions in the [liveBook discussion forum](#)—your feedback is invaluable in helping us improve the book.

Best regards,

Hamza Farooq

Founder Traversaal.ai

Adjunct Professor, UCLA & Stanford

In this book

[welcome](#) [1 The World of Large Language Models](#) [2 An in-depth look into the soul of the Transformer Architecture](#) [3 Encoder models in action: Semantic-Based Retrieval Systems](#) [4 Semantic Search from Scratch](#) [5 Decoders in Action](#)

1 The World of Large Language Models

This chapter covers

- An Introduction to Large Language Models: What defines a large language model?
- Applications that Large Language Models have unlocked across text generation, classification, translation, and retrieval systems.
- The Anatomy of a Large Language Model Application
- The scale and challenges of these models
- The startup world created by the Large Language Model

Language is one of the most special tools we have. It's how we connect with others, share our thoughts, and build our world. We have grown from using simple sounds to sharing ideas through complex words.

This is where Natural Language Processing (NLP) comes in. It's a mix of language study, computer science, and smart computer programs. NLP is about making computers understand and use human language. We need this because we now have more words and information than ever before. No one can read and understand all of it alone. The history of NLP dates back to the 1940s (https://cs.stanford.edu/people/eroberts/courses/soco/projects/2004-05/nlp/overview_history.html) with the development of simple translation algorithms, evolving dramatically over decades through advances in linguistic theory and computational power to today's sophisticated machine learning models that can understand and generate human-like text. As the 20th century ended, NLP had made significant strides with machines learning from vast datasets, recognizing language patterns, and understanding sentence structures. Yet, despite these advancements, capturing the nuances of the complexities of language remained out of reach.

With the explosive growth of the Internet and social media, the stage was set for a dramatic transformation. Deep learning, an advanced form of machine

learning, emerged as the key to unlocking new possibilities in language comprehension. This technique uses artificial intelligence systems inspired by the human brain, called neural networks. These networks consist of interconnected nodes, organized in multiple layers, that process information. By learning from vast amounts of data, deep learning can identify complex patterns in language—a massive leap forward in teaching machines to understand human communication. This approach, while complex, has proven remarkably effective at grasping the nuances and context of human language.

During this period, personal assistants like Apple's Siri (2011) and Amazon's Alexa (2014) entered the mainstream, revolutionizing everyday interactions with technology through voice commands. They provided a glimpse of how machines could interact with us, albeit within predefined scopes and limitations.

As deep learning continued to evolve, it paved the way for the creation of Large Language Models (LLMs), deep learning models specifically built for understanding language. These sophisticated algorithms leveraged the increased computational power and an ever-growing corpus of textual data to grasp and mimic human-like communication. Rather than merely executing specific commands, LLMs could craft full paragraphs, engage in meaningful conversations, and distill complex information across diverse topics.

LLMs represent a significant leap over the capabilities of early virtual assistants. They are not constrained by simple reactive patterns; instead, they proactively generate language and anticipate conversational turns, providing a seamless back-and-forth that often rivals human interactions.

Recognizing their transformative impact, it is essential to explore how LLMs can be applied in real-world scenarios to unlock their full potential. This book is crafted with a singular purpose – to serve as your guide through the practical applications of LLMs, unraveling the intricacies, principles, and hands-on experiences of working with these linguistic powerhouses.

In the pages that follow, our journey is not a mere plunge into the mathematical details that explain LLMs. Instead, we will focus on LLMs as the foundational building blocks of a grander machine-learning (ML)

ecosystem. As we will explore in detail, our focus extends beyond the algorithms and equations, reaching into the practical realm where the real-world applications of LLMs come to life in transformative ways.

The narrative within these chapters seeks to unearth diverse use cases that showcase the versatility and applicability of LLMs across various domains. From revolutionizing customer service to enhancing creative processes, from aiding in scientific research to improving educational tools, we'll explore how these powerful models are reshaping industries and opening new frontiers of possibility. By steering clear of an exclusive mathematical discourse, we aim to make the understanding of LLMs accessible to a broad audience, fostering an appreciation for the profound impact they can have on the field of artificial intelligence and society at large.

So, what's a great example of an LLM? I think we all know at least two of them, ChatGPT and Anthropic. The former is a cutting-edge language model developed by OpenAI, representing the pinnacle of conversational artificial intelligence. Designed on the foundations of GPT (Generative Pretrained Transformer), ChatGPT takes the capabilities of LLMs to new heights by specializing in dynamic and interactive conversations. It has the remarkable ability to generate coherent and contextually relevant responses, making it an integral part of the ever-evolving landscape of NLP. The latter is a similar company that provides LLMs-as-a-service, founded by two former OpenAI employees.

Here are a few exemplary prompts that you can use to engage in a conversation with ChatGPT:

Introduction and Casual Chat:

Prompt: "Hello ChatGPT! Concisely tell me a bit about yourself and what you can do."

Answer: I'm an AI developed by OpenAI, designed to assist with a variety of tasks, including answering questions, providing explanations, and aiding in creative endeavors.

Creative Writing Assistance:

Prompt: "Help me come up with a creative story idea!"

Answer: How about a story on a sentient robot navigating emotions after being abandoned on a space station, allying with alien visitors?

Figure 1.1 An output for a given prompt using ChatGPT



1.1 What are Large Language Models, anyway?

To better understand LLMs, consider a relatable analogy from everyday life: Personal relationships. Imagine the depth of understanding that arises in a long-term partnership, such as marriage, especially as it evolves over the years. As time unfolds, we become fluent in the silent language of our significant others—their habits, their quirks, their unspoken predilections—right down to predicting responses with remarkable precision.

Take, for instance, my wife's meticulousness about cleanliness, a trait on which I am comparatively less focused and something which I am regularly reminded of. Imagine what happens when she "accidentally" stumbles upon my home office, finding boxes and papers scattered around. It is at that point that I can almost anticipate, word for word, her subsequent reaction, tone, and facial expressions. It's a fascinating skill in pattern recognition that I often

marvel at. Regrettably, this awareness hasn't exactly spurred me to maintain the house or just my home office, as immaculately as she would prefer.

In this scenario, essentially my mind is acting as a language model. I have learned patterns and tendencies in my spouse's speech and can predict what she might say in a given situation.

LLMs operate similarly. At their core, the simplest language models function as probabilistic models, predicting the next word based on a preceding sequence of words. Much like anticipating my wife's reactions through observed patterns, these models leverage extensive training data to make informed predictions about language and context. What is large about LLMs? They are trained on massive datasets; imagine the entire world's knowledge fed into a model.

In the realm of language processing, the underlying mechanisms of LLMs involve an intricate dance of probability and pattern recognition. By analyzing vast datasets, these models develop an understanding of linguistic nuances, enabling them to generate coherent and contextually relevant text. Just as my awareness of my wife's tendencies aids in predicting her responses, LLMs draw on their training to predict the most probable next word in a given sequence.

This intricate interplay between observed patterns and predictive capabilities showcases the power and potential of LLMs in capturing the richness and complexity of human language. In essence, an LLM is an incredibly advanced AI system that can comprehend and generate human-like text based on the patterns and information it has learned from its extensive training on diverse textual data.

By utilizing these LLMs, we can unlock a host of possibilities in NLP, text generation, and even interactive applications. Throughout this book, we will explore the practical application of LLMs, leveraging the concepts and frameworks developed by startups in this field. Think of it as harnessing the work of a skilled data scientist or machine learning engineer to create our own iteration of an LLM-powered application.

It is important to point out that while our focus in this book is aimed at

textual data-based LLMs, there is another great innovation afoot, multi-modal models. These pioneering systems are expanding the horizons of machine learning by integrating and interpreting various forms of data, such as images, audio, and text, simultaneously. A great example of such model is *Gemini*, released by Google.

What do multimodal models do exactly? While a traditional LLM might excel at analyzing and generating text, a multimodal model could simultaneously interpret an image, understand spoken words, and generate relevant textual responses. This integration of multiple data types mirrors more closely how humans perceive and interact with the world, opening up exciting possibilities for more natural and versatile AI applications. From visual question-answering systems to AI-powered content creation tools that combine text and images, multimodal models represent the next frontier in making AI systems more adaptable and human-like in their ability to process information from various sources.

1.1.1 Application of LLMs

Having learned intuitively and metaphorically about LLMs, let's take a deeper dive into their applications.

Some of the key applications include:

Conversational Assistants: LLMs can power conversational agents that can engage in meaningful and contextually relevant conversations with users. These models can understand user queries, provide informative responses, and even generate human-like dialogue. A very popular application has been Retrieval Augmented Generation (more on this soon)

Text and Code Generation: LLMs can generate coherent and contextually appropriate text in various domains (including code!). This capability is particularly useful for tasks such as content generation, text summarization, language translation, and even creative writing. For example, these models can be used to generate fictional stories, poetry, or scripts for movies and plays.

Information Retrieval: They can be used to improve search engine results

by understanding user queries and retrieving relevant information from a large corpus of documents. Additionally, these models can help in organizing and categorizing information, making it easier for users to find the specific information they are looking for.

Language Understanding: They can assist in tasks such as sentiment analysis, intent recognition, named entity recognition, and even language tutoring. These models can understand the nuances of language and accurately interpret the meaning and sentiment behind text, making them valuable tools for various language-related applications.

Recommendation Systems: They can be used to build recommendation systems that provide personalized recommendations based on user preferences and historical data. These models can analyze user behavior and preferences to suggest relevant products, services, movies, books, or even travel destinations. Additionally, they can adapt and improve their recommendations over time based on user feedback and interactions.

Content Creation and Editing: LLMs can be utilized to assist in content creation and editing processes. These models can provide suggestions for improving the clarity, coherence, and overall quality of written text. They can offer alternative word choices, sentence restructuring, and grammar corrections to enhance the readability and effectiveness of written content.

Agent-based Task Fulfillment: LLMs can be integrated into software agents that autonomously perform tasks on behalf of users. These agents, powered by LLM capabilities, can interpret user instructions, navigate complex systems, and execute a series of actions to achieve a specified goal. This application extends from automating scheduling and making reservations to managing emails and interfacing with other digital services. By employing conversational interfaces and understanding user preferences, these agents can provide personalized assistance, streamline workflows, and enhance productivity in personal and professional settings.

So, what gives LLMs such impressive power, the scale at which they are trained and all the compute that is required to accomplish that task.

1.1.2 Understanding the Scale of LLMs

The scale of LLMs presents both advantages and challenges. On one hand, the vast amount of training data and parameters enable these models to generate highly accurate and contextually appropriate text. They can provide more nuanced responses, making them highly valuable in a wide range of applications.

Note

With their extensive training, LLMs have developed a deep understanding of language patterns and can generate text that closely resembles human-like communication. This is particularly evident in their ability to generate coherent and contextually appropriate responses in various NLP tasks, such as chatbots, text summarization, and content generation. The scale of these models allows them to capture the intricacies of language, including grammar, syntax, and even subtle nuances of meaning.

The sheer size requires LLMs to be compute-intensive, which means, training and fine-tuning LLMs require significant computational resources and time. Training and fine-tuning in the context of LLMs refer to the processes through which these models learn from data and optimize their parameters for specific tasks.

Training: During the training phase, an LLM is exposed to vast amounts of data, typically comprising diverse examples of human language (more on that in the next section). The model learns to predict the next word in a sequence based on the patterns it identifies in the input data. This learning process involves adjusting two key components within the model's architecture: weights and biases. Weights determine the strength of connections between different parts of the model, while biases help the model make predictions even when it receives no input. These elements are crucial to the model's ability to recognize patterns and make accurate predictions.

During training, the model continuously adjusts these weights and biases to minimize the difference between its predictions and the actual outcomes in the training data. This adjustment process is iterative, meaning it repeats many times, gradually refining the model's understanding of language patterns. The training continues until the model achieves a satisfactory level of proficiency in understanding and generating language. At this point, we

typically refer to it as a "pre-trained" model, ready for further fine-tuning or direct application to specific tasks.

Fine-Tuning: Fine-tuning is a subsequent step that involves adjusting a pre-trained model to perform a specific task or cater to a particular domain. Instead of starting from scratch, fine-tuning allows developers to leverage the knowledge the model gained during its initial training on a broader dataset.

For example, a pre-trained LLM might be fine-tuned to specialize in legal or medical language by exposing it to relevant examples in these domains. Fine-tuning helps adapt the model to specific nuances and requirements, enhancing its performance for targeted applications.

1.1.3 Training LLMs

LLMs require vast amounts of data for training. ChatGPT was largely trained on one of the largest text corpora, called the *Common Crawl*, which has over 250 billion pages spanning 16 years. Common Crawl is a nonprofit 501 organization that crawls the web and freely provides its archives and datasets to the public. Common Crawl's web archive consists of petabytes of data collected since 2008. It completes crawls generally every month. Common Crawl was founded by Gil Elbaz. This vast expanse is important for several crucial reasons:

- **Learning General Patterns:**
LLMs aim to understand and replicate the intricate patterns and structures present in human language. A diverse and extensive dataset provides the model with a broad range of linguistic examples, enabling it to learn general language patterns applicable across various contexts.
- **Semantic Understanding:**
To develop a nuanced understanding of language semantics, LLMs need exposure to a wide array of topics, styles, and expressions. Vast datasets help these models capture the richness of language, from colloquialisms to formal expressions, enhancing their ability to comprehend and generate text accurately.
- **Contextual Awareness:**
LLMs rely on context to generate coherent and contextually relevant

responses. Large datasets allow models to learn the subtle contextual cues and dependencies that influence the meaning of words and phrases within different contexts.

- **Robustness and Adaptability:**

Exposure to a diverse dataset makes LLMs more robust and adaptable. It equips them to handle a broad spectrum of input scenarios, ensuring that the model can perform well across a range of tasks and domains.

- **Handling Ambiguity:**

Language is inherently ambiguous, and different words or phrases can have multiple meanings depending on context. Large datasets help LLMs navigate this ambiguity by learning the contextual nuances that disambiguate meanings.

- **Avoiding Overfitting:**

Training on a substantial amount of data helps prevent overfitting, a situation where a model becomes too specialized in the training data and struggles to generalize to new, unseen examples. The richness and diversity of a large dataset contributes to the model's ability to generalize effectively.

In essence, the requirement for vast amounts of data is fundamental to the success of LLMs, allowing them to capture the complexity and variability inherent in natural language and enabling them to deliver more accurate and contextually aware results across a wide range of applications.

We know the training process for LLMs requires immense computational power and substantial amounts of training data. It often involves training on distributed systems with multiple Graphics Processing Units (GPUs) or Tensor Processing Units (TPUs) to accelerate the process. GPUs are specialized processors originally designed for rendering graphics but now widely used in AI computations, while TPUs are custom-built AI accelerator chips developed by Google specifically for machine learning tasks. The use of these powerful processing units is crucial because the training process can take weeks or even months to complete, depending on the size of the model and the available resources. This extended training time underscores the complexity and scale of developing advanced LLMs.

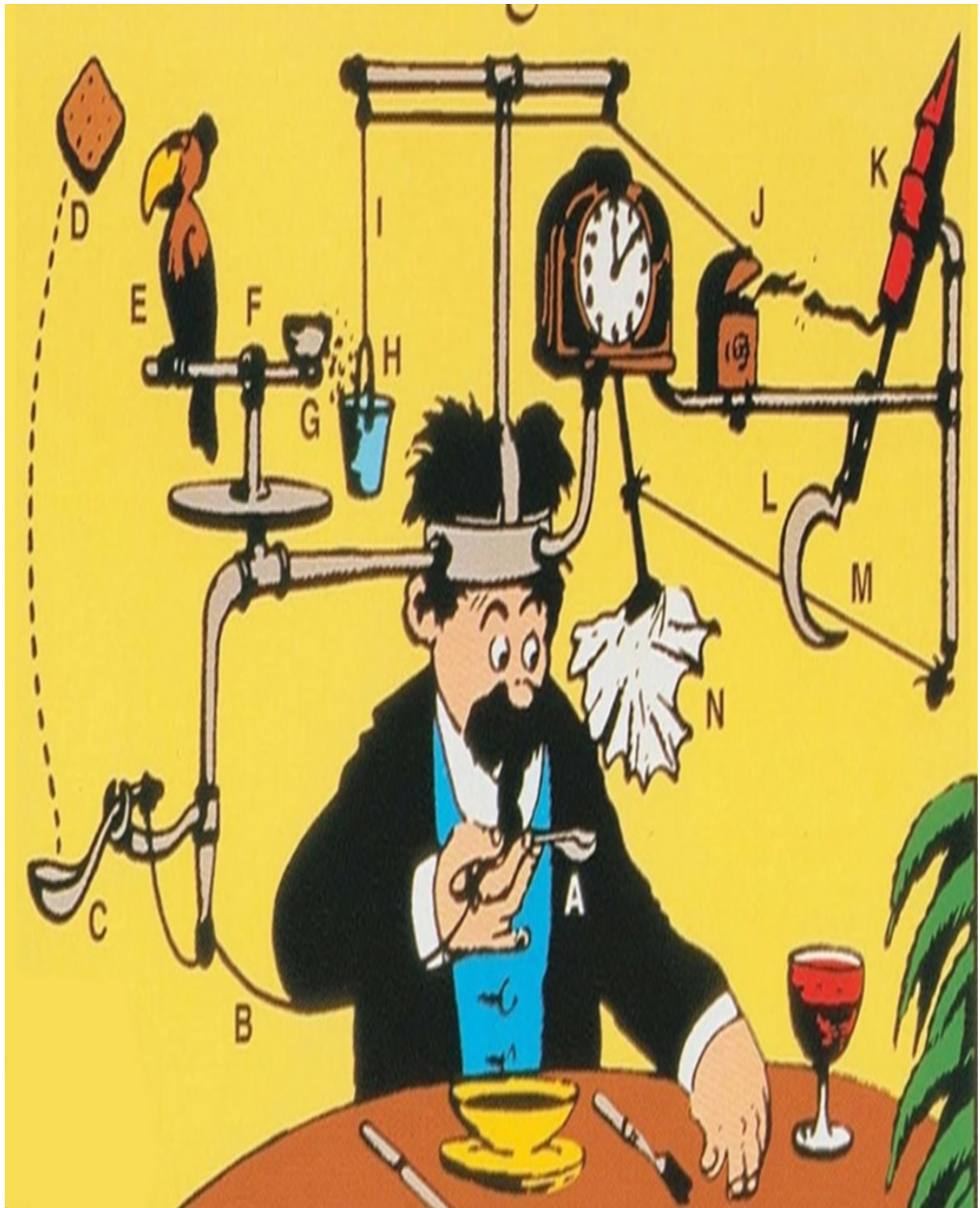
Despite the challenges, training LLMs has proven to be a worthwhile

investment, as it enables the models to acquire a deep understanding of language and generate high-quality text. The high upfront costs are often being recuperated by charging for API calls based on the token length of subscription models. The advancements in training techniques and computational resources have paved the way for the development of state-of-the-art language models.

1.2 The Anatomy of an LLM Application

When venturing into the realm of constructing an LLM application, it quickly becomes apparent that numerous intricate components are at play. The process can, at times, resemble the challenge of navigating Rube Goldberg's self-operating napkin—an intricate and convoluted journey where seemingly straightforward tasks can unexpectedly transform into elaborate processes.

Figure 1.2 Rose Goldberg's famous self-operation napkin constructing an LLM application demands a thoughtful orchestration of resources, from computational power to application definition, echoing the complexity of Rube Goldberg's contraptions.



Understanding that LLMs are compute-intensive introduces the necessity of acquiring GPUs, emphasizing the importance of selecting the appropriate hardware. The decision-making process extends beyond mere hardware

considerations to identifying the specific application for which the LLM will be tailored. Defining the application's purpose is a pivotal step, whether it involves natural language understanding, text generation, sentiment analysis, or any other task within the broad spectrum of natural language processing.

Moreover, as we delve deeper into the intricacies of LLM applications, considerations expand to encompass the training and fine-tuning phases. These stages involve exposing the model to extensive datasets and refining its parameters to ensure optimal performance for the intended application. The interplay of hardware, software, and application specificity requires a nuanced approach to navigate the complexities of deploying effective and efficient LLMs.

By carefully navigating through these intricacies, developers can unlock the full potential of LLMs and harness their capabilities to create sophisticated, context-aware applications in the realm of natural language processing.

Let's pick one example: RAG, or Retrieval-Augmented Generation.

RAG, short for Retrieval-Augmented Generation, is an important tool in AI that enhances language models. It works by finding relevant information from specific data sources and combining it with the model's great generation skills, to provide more accurate and up-to-date responses. This makes RAG incredibly good at internet search since instead of providing just links, they can a well written response, we will explore [Ares](#), in the upcoming chapters.

Figure 1.3 A Python code snippet demonstrating how to use the Ares API to retrieve information about taco spots in San Francisco using the internet. Instead of just showing URLs, the API returns actual answers with web URLs as source

Sample Code for Python

```
import requests
import json

url = "https://api-ares.traversaal.ai/live/predict"

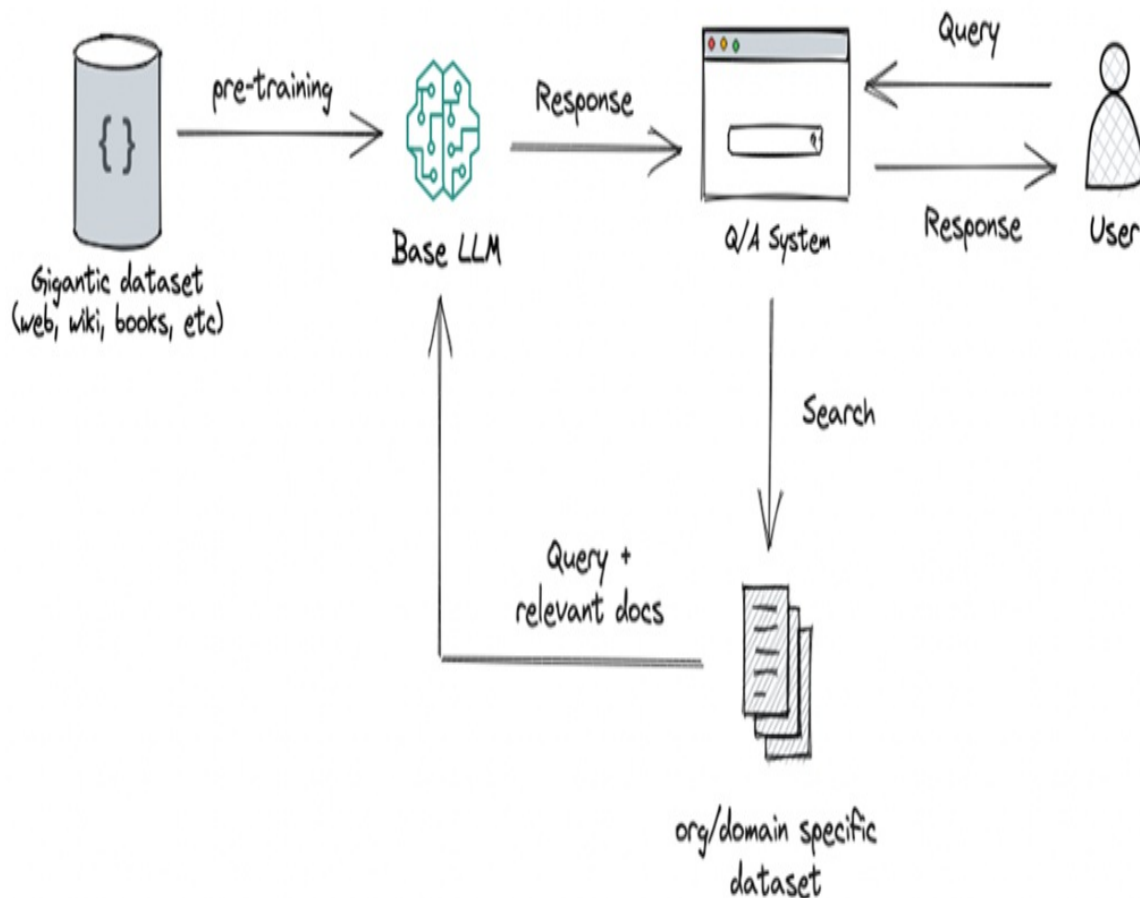
payload = { "query": ["taco spots in San Francisco"] }
headers = {
    "x-api-key": "_your api key_",
    "content-type": "application/json"
}

response = requests.post(url, json=payload, headers=headers)
```

RAG doesn't search the entire internet but instead looks through a carefully selected set of documents or data. It finds information that closely matches the given query, though it's important to note that RAG doesn't guarantee the reliability of this information. By using RAG, language models can access additional context beyond their training data, allowing them to generate more informed and contextually appropriate responses.

This approach is particularly useful when dealing with specialized topics or when the most current information is crucial. However, it's most effective when applied to a focused set of documents rather than an overwhelmingly large collection of data.

Figure 1.4 Retrieval Augmentation Generation is used to enhance the capabilities of LLMs, especially in generating relevant and contextually appropriate responses. The approach involves incorporating an initial retrieval step before generating a response to leverage information from a knowledge base.



Here's how it works:

Retrieval Step:

- In the first phase, the system performs a retrieval process. It searches a pre-existing database, knowledge base, or corpus to identify relevant information based on the input query or context.

Candidate Selection:

- The retrieved information is then used to generate a set of candidate responses. These candidates can be phrases, sentences, or paragraphs that are deemed relevant to the input or context.

Context Integration:

- The candidates obtained from the retrieval step are integrated into the context of the original input. This integration helps the language model to consider not only its pre-existing knowledge but also information relevant to the specific query or context at hand.

Response Generation:

- The final step involves the language model generating a response based on the integrated context. By incorporating information retrieved from external sources, the model aims to provide responses that are not only linguistically appropriate but also contextually relevant.

The key advantage of Retrieval Augmentation is that it allows LLMs to benefit from external knowledge or context, potentially improving the quality and relevance of the generated responses. This is particularly useful in scenarios where the model needs to be aware of recent information, updates, or specific details that may not be present in its pre-training data. This approach is often employed in conversational AI, question-answering systems, and chatbots to enhance the system's ability to provide accurate and up-to-date information by leveraging external knowledge sources.

1.3 Challenges and Limitations of LLMs

While large LLMs have shown immense potential, they also face certain challenges and limitations. Some of these include:

- **Data Bias:** LLMs are trained on large datasets, which may contain biases present in the data. These biases can be reflected in the model's output, potentially perpetuating stereotypes or discrimination.

- **Ethical Concerns:** The use of LLMs raises ethical concerns, such as the potential for generating misleading or harmful content. It is important to carefully monitor and regulate the use of these models to ensure responsible and ethical deployment.
- **Interpretability:** LLMs can be black boxes, making it difficult to understand how they arrive at their decisions or generate certain responses. This lack of interpretability can hinder trust and transparency in their applications.
- **Hallucinations:** Hallucinations refer to instances where large language models (LLMs) generate content that is factually incorrect or nonsensical, despite being presented coherently. This can occur when the model confidently produces information that it has not been explicitly trained on or when it encounters out-of-distribution prompts. These hallucinations challenge the reliability of LLMs in critical applications and underscore the need for robust validation and fact-checking mechanisms.

Addressing these challenges and limitations will be crucial in maximizing the potential benefits of LLMs while minimizing their risks. As the field continues to evolve, researchers and practitioners are actively working towards developing solutions and best practices for responsible and effective use of these models.

In the next chapter, we will take a deeper dive into the Transformer architecture and address the specific components that make LLMs so much better than their predecessors.

1.4 The Startup World of LLMs

Why would we want to include this topic in the very first chapter? Simple, the rise of LLMs has sparked a veritable gold rush of innovation and investment. Recognizing its significance helps us understand how this technological leap has galvanized entrepreneurs and venture capitalists, leading to an explosion of new ventures and unprecedented funding levels never seen before in the landscape of AI-driven businesses.

This has created the need to learn about LLMs as fast as possible and as

much as possible. Worry not! This book aims to equip readers with a comprehensive understanding of LLMs and empower them to harness the power of these models in practical scenarios. From conceptual frameworks to hands-on examples, our journey will guide you through the process of creating context-aware, efficient, and effective LLM Applications. Whether you're a seasoned developer or just stepping into the world of LLMs, this resource will serve as a valuable companion in your exploration of LLM applications. Let's embark on this enlightening journey together.

As we close this chapter, let's explore various types of Startups I have witnessed at the time of writing this book:

- Some are 'wrappers' built on top of ChatGPT, offering additional functionality. The interesting ones in that realm are *Tome* and *beautiful.ai*. These startups have created a unique blend of text generation with art, making decks beautiful again.
- Then some are involved in developing the infrastructure tools for building large-scale LLM Solutions. There is a huge spectrum amongst these startups, but some good examples are Vector DB companies such as *Pinecone* and *Qdrant* and LLM-framework companies such as *Langchain* and *Llamaindex*.
- A smaller subset aims to train and develop their own LLMs and will compete with *OpenAI* for their place among top-tier "Industrial LLMs." From the moment I started writing this book to now, we have seen many players enter the market, some new and some old ones: Google (Gemini), Mistral (Mistral models), Meta (Llama) and Microsoft, to name a few.

Naturally, venture capital (VC) funding escalates exponentially in alignment with the mentioned startup types. The initial category, abundant in numbers, tends to fade away quickly due to their ease of replication and lack of distinctiveness.

In contrast, the second category holds significant intrigue. Here, startups engage closely with enterprise customers and other emerging companies, often adopting a consultative approach to assist them in developing LLM solutions. Many of these startups focus on creating APIs and Software as a Service (SaaS) products that integrate LLMs into their ecosystems, rather

than relying solely on ChatGPT. Such startups have successfully secured VC funding ranging from 5 million to over 25 million dollars.

Finally, the last group represents a few notable organizations that we would call "GPU-rich" companies. They have funding to the tune of BILLIONS with thousands of H100s. The H100 is a high-performance graphics processing unit (GPU) designed by NVIDIA. It's specifically tailored for data center environments and is renowned for its exceptional computational power, particularly in accelerating artificial intelligence (AI) and high-performance computing (HPC) workloads

These organizations are building the "next gen" of LLMs and intend to compete directly with OpenAI. Interestingly, the founders of most of these startups were authors of the much-celebrated paper "*Attention is all you need*". I recently had the opportunity to test out *Inflection.ai*'s LLM and it reminded me of the movie "HER".

Spoiler alert: this book won't cover much of what the "GPU-rich" startups are doing.

As I mentioned earlier, our focus for this book will be on building LLM Applications, delving into the intricacies of constructing sophisticated language models to address real-world challenges. We will explore not only the theoretical foundations of LLMs but also provide practical insights into their implementation and deployment.

In the next chapter we will take a deep dive into the building components of LLMs and what makes them so good.

1.5 Summary

- Large language models (LLMs) are the latest breakthrough in natural language processing after statistical models and deep learning. LLMs stand on the shoulders of this prior research but take language understanding to new heights through scale.
- Pretrained on massive text corpora, LLMs like GPT-3 capture broad knowledge about language in their model parameters. This allows them

to achieve state-of-the-art performance on language tasks.

- Applications powered by LLMs include text generation, classification, translation, and semantic search to name a few.
- LLMs utilize multi-billion parameter Transformer architectures. Training such gigantic models requires massive computational resources only recently made possible through advances in AI hardware.
- Bias and safety are key challenges with large models. Extensive testing is required to prevent unintended model behavior across diverse demographics.
- Numerous startups are offering LLM model APIs, democratizing access and allowing innovation in the realm of Generative AI.

2 An in-depth look into the soul of the Transformer Architecture

This chapter covers

- The advantage of the Transformer Architecture over Recurrent Neural Networks
- Dive deep into the key components of the Transformer architecture: Self-Attention Mechanism, Multi-Head Attention, and Positional Encoding
- A comprehensive understanding of the Encoder and Decoder models and their roles within the Transformer architecture
- Real-world use cases and examples showcasing the power and versatility of the Encoder-Decoder model over existing architectures.

In the last chapter, we embarked on a journey to explore the transformative impact of Large Language Models (LLMs) and the remarkable advancements they have brought forth. Now, it's time to peel back the layers and dive into the core architecture that sets LLMs apart from their predecessors. Allow me to introduce you to the *Transformers Architecture*. The key takeaways from this chapter revolve around a trio of essential concepts: *the Self Attention Mechanism*, *Encoder models*, and *Decoder models*. We will thoroughly examine each of these elements and the significant roles they play. While Self Attention Mechanism is only covered in this chapter, Encoder and Decoder models will be explored in greater depth throughout the rest of the book, as they form the cornerstones of building powerful LLM Applications such as semantic search, machine translation, summarization and many more.

As we navigate through this chapter, my aim is not to overwhelm you with a barrage of complex equations but rather to equip you with a solid grasp of the crucial components that enable Transformers to excel at contextual understanding.

Attention is All You Need: The paper that started it all

[Attention is All You Need](#) is a groundbreaking paper in the field of natural language NLP that introduced the Transformer architecture. This architecture, based solely on attention mechanisms, revolutionized how machine translation models are built.

Key points from the paper:

- Attention mechanism: A way to weigh the importance of different parts of an input sequence when processing a specific part.
- Transformer architecture: A sequence-to-sequence model that uses self-attention to relate different positions of an input sequence.
- No recurrent neural networks (RNNs) or convolutional neural networks (CNNs): The Transformer relies solely on attention mechanisms, eliminating the need for recurrent or convolutional operations.
- Parallel computation: The Transformer allows for parallel computation, making it more efficient than RNN-based models.
- State-of-the-art results: The Transformer achieved state-of-the-art results on various NLP tasks, including machine translation, text summarization, and question answering.

Language Models have existed for many years. The pioneers of Language Models harnessed the power of techniques such as *n-grams* and *Markov chains*, which generated text by leveraging probabilities and predefined rules. While these early models displayed glimmers of potential, they were inherently limited in their ability to comprehend the nuances of language and grasp the intricacies of context. As a result, the outputs produced by these models often fell short, riddled with inaccuracies and nonsensical statements that failed to capture the true essence of human communication. This led to the development of the neural network models.

The inception of neural networks transformed language modeling, yielding a noteworthy enhancement in producing coherent and contextually relevant text. Recurrent neural networks (RNNs), a subset of neural language models, played a pivotal role in this transformation. Crafted to capture long-term dependencies and discern sequential patterns in text, these models excelled in generating more natural and fluent sentences.

A significant advancement in RNNs came with the introduction of Long

Short-Term Memory (LSTM) networks. LSTMs addressed some of the limitations of traditional RNNs, particularly in handling long-range dependencies in sequences. This made them especially effective for tasks involving language processing and generation.

Apple's Siri and Google's voice search both use RNNs and LSTMs, which were the state-of-the-art methods for sequential data at that time. These models were revolutionary because of their internal memory, which allowed them to remember and utilize information from earlier in a sequence when processing later parts. This capability made them particularly adept at handling tasks involving sequential data in machine learning, such as speech recognition, language translation, and text generation.

In practical terms, this meant that these models could maintain context over longer sequences of text or speech. For instance, in a conversation, they could better understand and respond to queries that depended on earlier parts of the dialogue. In text generation, they could produce more coherent paragraphs by maintaining consistency in topic and style throughout the text. This ability to process and generate sequential data with maintained context was a significant leap forward in making AI interactions more natural and human-like.

Although RNNs were truly state-of-art, they weren't without challenges. These models faced challenges in grappling with the intricacies of capturing long-range dependencies and contending with notorious issues like vanishing or exploding gradients during training (explained in the section). We begin this chapter by understanding the advantage of Transformers over RNNs.

2.1 The Transformers Architecture improvements over Recurrent Neural Networks

The Transformer architecture has brought significant improvements to the field of natural language processing, addressing the limitations of previous state-of-the-art models like RNNs and Gated Recurrent Units (GRUs). While RNNs and GRUs have been widely used for sequence modeling tasks, they often struggle with capturing *long-range dependencies* and suffer from *vanishing or exploding gradients* during training. The Transformer model

tackles these issues head-on, enabling more efficient and effective processing of sequential data.

Facing difficulties in capturing *long-range dependencies* means that the model struggles to include information from faraway parts in a sequence. This problem makes it hard for the model to understand connections and relationships that stretch over a long sequence of steps.

The phrase "prone to vanishing or exploding gradient problems during training" talks about a common issue while the model is learning. *Gradients*, which show how much and in which direction the model's settings should change, can either become very tiny (vanishing) or extremely large (exploding). This shaky behavior in gradient values makes the learning process complicated, making it tough for the model to find the best solution

To illustrate the difference between RNNs, GRUs, and Transformers, let's consider the sentence: "The quick brown fox jumps over the lazy dog." An RNN or GRU would process this sentence sequentially, one word at a time, maintaining a hidden state that carries information from previous words. However, if the sentence is long or contains complex dependencies, these models might struggle to accurately capture the relevant information. In contrast, a Transformer model processes the entire sentence in parallel, using self-attention mechanisms to identify and focus on the most relevant words and their relationships. This allows the Transformer to better understand the context and capture long-range dependencies, resulting in more accurate and coherent outputs.

The Transformer architecture offers several advantages over RNNs and GRUs:

- Parallel processing, enabling faster training and inference times.
- Ability to capture long-range dependencies effectively.
- Scalability to handle larger datasets and more complex tasks.
- Transfer learning capabilities through pre-training on vast amounts of unlabeled text data. Transfer learning is a machine learning technique where a model pre-trained on one task is used as a starting point for a new task, often related but different. This helps improve performance and reduces training time.

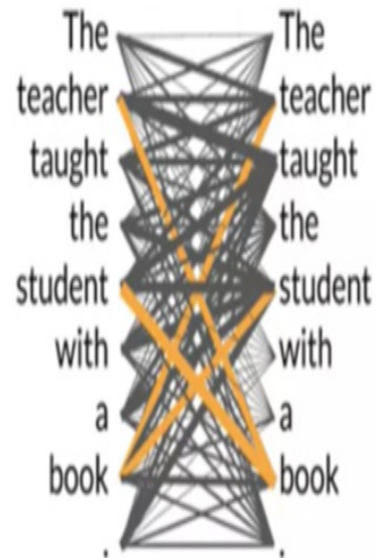
Transformers addressed this gap by introducing a fully parallelizable **self-attention mechanism**, allowing them to efficiently process sequences in parallel, overcoming the limitations of both RNNs and GRUs. To fully grasp this concept, let's consider an example: Consider reading a book word by word, and at each word, you update your understanding of the story. That's how RNNs and GRUs work—they process information sequentially, one step at a time. But there's a catch. Imagine you're reading a paragraph, and a crucial word at the beginning affects the meaning of a word at the end. RNNs and GRUs struggle here. They tend to forget early information as they go along, making it tough to capture long-distance connections in language.

Now, imagine if you could read the entire paragraph at once, grasping all connections in a single glance. That's what the Transformer does. Instead of reading sequentially, it uses self-attention. It checks out all words simultaneously, giving attention to the important ones. This way, it captures long-range relationships effortlessly.

Figure 2.1 The strength of the transformer model is in its ability to understand the significance and context of every word in a sentence, unlike RNNs which primarily assess the connection of a word with its immediate neighbor.



Recurrent Neural Network



Transformers

Transformers have found numerous applications in various aspects of NLP, improving upon the performance of previous models:

- Language translation, delivering high-quality and fluent translations compared to RNN and GRU-based models
- Text summarization, generating more coherent and informative summaries than previous approaches
- Sentiment analysis, achieving higher accuracy in determining the sentiment expressed in text data
- Named entity recognition, identifying and classifying named entities within text with improved precision and recall. Essentially it is a task in NLP that involves identifying and classifying named entities in text, such as persons, organizations, locations, dates, and times.
- Question answering, providing more accurate and contextually relevant answers to user queries compared to earlier models

Researchers are continuously working on enhancing the Transformer architecture, focusing on several key areas:

- Model compression, aiming to reduce the size of Transformer models without compromising performance, making them more efficient than RNNs and GRUs
- Interpretability, striving to make Transformer models more transparent and understandable compared to the black-box nature of previous models
- Multimodal learning, extending Transformers to handle multiple modalities like text, images, and speech, surpassing the capabilities of RNNs and GRUs.
- Domain adaptation, developing techniques to adapt Transformer models to specific domains or tasks with limited labeled data, outperforming previous approaches.

In essence, the Transformer's attention mechanism enables the model to "concentrate" on various aspects of the input data simultaneously, akin to how one navigates different speakers at a bustling cocktail party. This mechanism can assess the significance of each input data segment, regardless of its position in the sequence.

Let's explore the Transformer architecture in a little more detail.

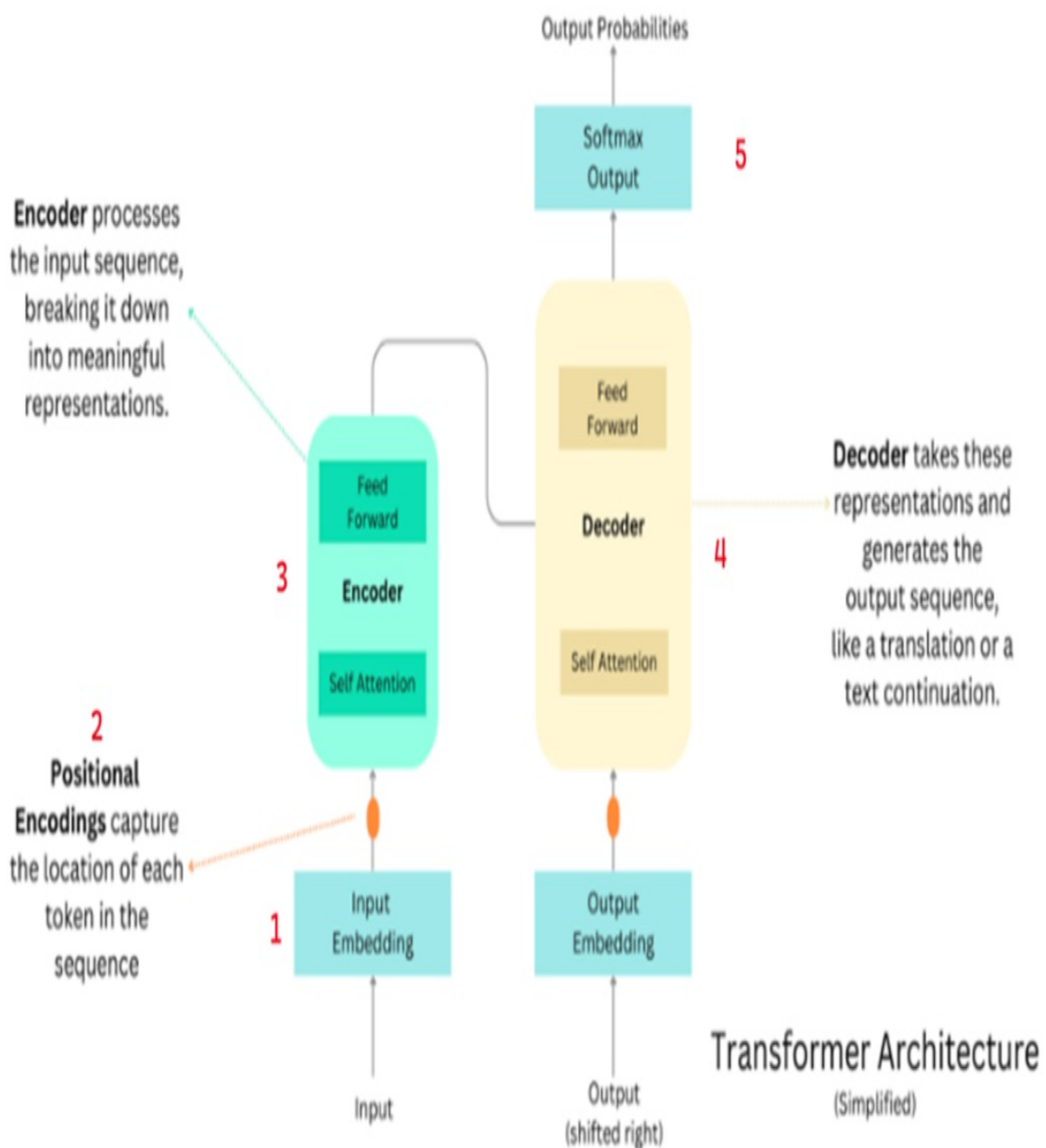
2.2 Digging into the Transformer's underlying architecture

Transformers feature a multi-layered architecture that centers around two primary components: an encoder and a decoder. The encoder functionally transforms an input sequence into a complex, higher-dimensional representation, meticulously preserving key contextual relationships within the data. Meanwhile, the decoder leverages this enriched representation to produce outputs, such as translations or text generation (which we will delve into more comprehensively shortly).

Nested just beneath the encoder/decoder layer, at the heart of the Transformer, lies a mechanism known as self-attention, or scaled dot-product

attention. This crucial feature sets Transformers apart from earlier models, granting both the encoder and decoder the ability to concurrently focus on various segments of their input and output sequences throughout the processing phase.

Figure 2.2 The interplay of the Transformer Architecture lies in two towers, the Encoder and Decoder layers which are made up of numerous layers.



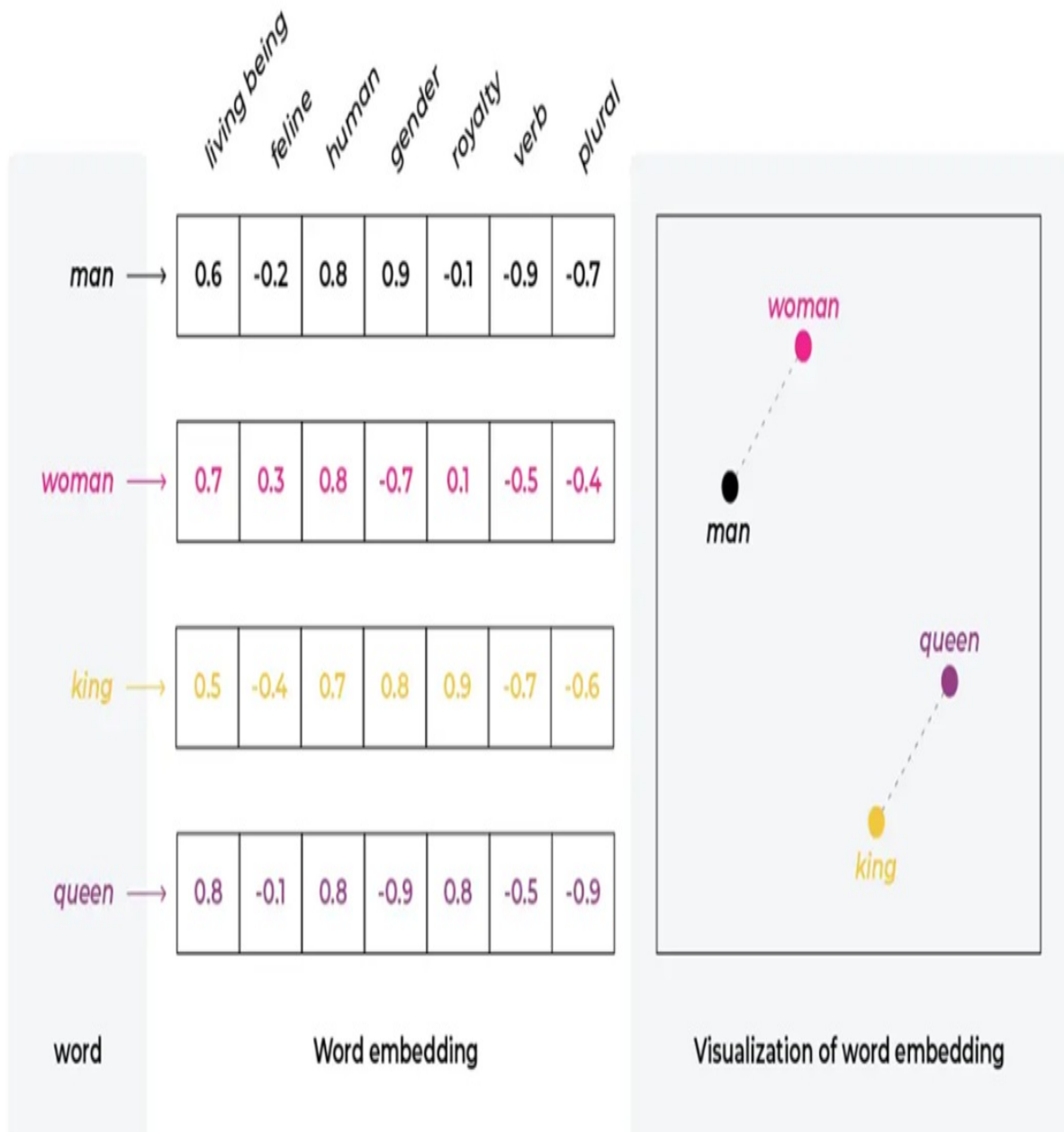
Let's break down the various layers and components of the Transformer architecture with detailed explanations:

1. Input Embedding:

Continuing from the encoder and decoder framework, the initial stage of the Transformer architecture begins with the input embedding layer. This foundational layer takes raw input data—typically in the form of text tokens (Tokens are the smallest units of text that LLMs process. They can be individual words, parts of words, or subwords, depending on the tokenization method used. The choice of tokenization can significantly impact the performance of an LLM)—and converts each token into a high-dimensional vector. These embeddings are crucial as they provide a dense, substantive representation of the input data, incorporating both the semantic and syntactic essence of each token.

To convert raw text into embeddings, we employ a method where words are translated into numerical arrays, often referred to as vectors. These vectors or “embeddings,” essentially serve as digital representations of words or even entire sentences. Within these numerical arrays, the meaning and nuanced context of the text are encapsulated, creating a multi-dimensional space where the richness of language is encoded in a numerical form. So, *input embedding* is turning words into a numerical form, helping the computer make sense of what each word means in a sentence.

Figure 2.3 illustrates the vector representation of words. Although the numerical values may not be immediately interpretable, the visualization effectively showcases the proximity of words in their numeric representation, highlighting their close association based on contextual meaning.



This initial step of input embedding involves transforming input sequences into high-dimensional vector embeddings. These embeddings encode the semantic meaning of each element in the sequence.

There are two popular approaches to how words are converted into vectors, Sparse Vectors and Dense Vectors. Let's explore them in detail.

One-Hot Encoding or Sparse Vectors: Imagine you have a list of words: "apple," "banana," and "orange." To convert these words into numbers, we use a technique called one-hot encoding. Each word gets a unique number, and a vector is created where all positions are zero except for the one corresponding to the word's number. For example:

- "apple" might be [1, 0, 0]
- "banana" might be [0, 1, 0]
- "orange" might be [0, 0, 1]

Dense Vectors: Using dense vectors provides a more nuanced way to represent words, utilizing a series of numerical values instead of simplistic binary 0s and 1s. This approach allows similar words to be closer together in the representation, offering a more accurate depiction of their relationships.

For example:

- "apple" might be [0.9, 0.3, 0.2]
- "banana" might be [0.8, 0.7, 0.1]
- "orange" might be [0.7, 0.5, 0.9]

Comparison:

- Sparse vectors are high-dimensional and mostly filled with zeros, representing words individually with little semantic information.
- Dense vectors are lower-dimensional and capture semantic relationships between words, providing a more contextually rich representation.

The transition from sparse to dense vectors has been essential for improving language models, allowing them to understand and process language more effectively. In Chapter 3, we explore the benefits when we build our first search engine.

Essentially, these vectors enable computers to understand language by

capturing relationships between words and contexts. Encoder models, in this process, are responsible for generating dense vector embeddings that serve as representations of sentences.

2. Positional Encoding:

After the input embedding layer processes the initial textual tokens into dense, high-dimensional vectors, the next critical step involves the application of positional encodings. Positional encodings are essential because, unlike RNNs, Transformers process input data in parallel rather than in sequence. This parallel processing offers efficiency gains but doesn't intrinsically account for the position or order of words in a sequence, which is crucial for understanding language.

Consider the Transformer as an advanced computational entity, akin to a sophisticated algorithmic system endeavoring to comprehend a narrative. However, this computational agent encounters a fundamental challenge—it lacks an inherent understanding of the sequential arrangement of words in a sentence. To overcome this hurdle, we intervene by assigning a distinct tag or code to each word, providing the system with positional information.

This unique code is termed “positional encoding.” Analogous to affixing small tags onto individual words, this encoding mechanism assists the computational entity in discerning the sequential structure of the words. With these positional tags, the computational agent can enhance its comprehension of the interrelation between words in the narrative. In essence, we equip our computational agent with a metaphorical map, enabling it to adeptly navigate through the intricate sequence of words within the textual domain. With the incorporation of positional encoding, the Transformer gains a comprehensive map to navigate through the intricate sequence of words. Importantly, this step in the process follows the initial *input embedding* and precedes the engagement with the *encoder layer*.

For example, consider a sequence of three words: "the," "cat," and "sat." It generates the following word embeddings:

"the": [0.1, 0.2, 0.3]

"cat": [0.4, 0.5, 0.6]

"sat": [0.7, 0.8, 0.9]

Assume a simplified positional encoding matrix for three positions and three dimensions:

[[0.1, 0.2, 0.3],

[0.4, 0.5, 0.6],

[0.7, 0.8, 0.9]]

The Combined Representation of the positional encoding matrix is added element-wise to the word embeddings:

"the": $[0.1 + 0.1, 0.2 + 0.2, 0.3 + 0.3] = [0.2, 0.4, 0.6]$

"cat": $[0.4 + 0.4, 0.5 + 0.5, 0.6 + 0.6] = [0.8, 1.0, 1.2]$

"sat": $[0.7 + 0.7, 0.8 + 0.8, 0.9 + 0.9] = [1.4, 1.6, 1.8]$

The combined representation now includes both word information (from the word embeddings) and positional information (from the positional encoding matrix).

The positional encoding helps the model understand the order of words in the sequence, which is essential for tasks like machine translation or text summarization.

Note: In practice, the positional encoding matrix is typically calculated using sine and cosine functions, and the number of dimensions is often larger to capture more complex positional relationships.

3. Encoder:

At the heart of the Transformer architecture lies the critical component known as the encoder. Before delving into the intricate workings of the encoder, it's crucial to understand its role in the broader context.

The encoder serves as the initial processing unit for input sequences, and it is pivotal in the Transformer's ability to comprehend and represent complex information. It functions through a series of meticulously crafted layers, each comprising essential sub-layers that collectively contribute to the model's overall understanding.

- Multi-Head Self-Attention Mechanism:

Within the encoder's layers, a standout feature is the incorporation of the multi-head self-attention mechanism. Imagine you're reading a story, and different characters and events are happening. The Multi-Head Self-Attention Mechanism is like having many pairs of eyes, each focusing on different parts of the story at the same time.

This helps the computer understand how words relate to each other, especially when they are far apart in the story. It's like paying attention to different parts of the tale simultaneously, allowing the computer to figure out complex connections and relationships across the entire story. This sophisticated mechanism is designed to address the challenge of capturing dependencies between distant elements in the input sequence. By simultaneously assigning varying weights to different positions, the model excels at discerning intricate relationships and dependencies, particularly those extending over extended ranges within the data.

- Position-wise Feed-Forward Networks:

Building upon the insights gleaned through the attention mechanism, the encoder further refines its understanding through the application of position-wise feed-forward networks.

Think of reading a book again. After paying attention to different parts of the story, the Position-wise Feed-Forward Networks are like a super-smart brain that thinks really hard about what's happening in each part. It works independently on each part of the story, making sure to catch all the important details and complex patterns. This step helps the computer understand the story even better by carefully thinking about each piece of information and creating a full picture of the entire sequence.

Within the encoder layer, these networks operate independently on each position within the sequence, allowing the model to capture and process complex patterns present in the data. This step enhances the encoder's ability to discern nuanced information and contributes to a more comprehensive representation of the input sequence.

Essentially, the encoder plays a pivotal role in processing input sequences through its multiple layers. The integration of the multi-head self-attention mechanism and position-wise feed-forward networks exemplifies the model's commitment to capturing both local and global dependencies within the data, marking a critical phase in the Transformer's journey toward robust comprehension and representation of sequential information.

4. Decoder:

The decoder is a crucial component in the Transformer architecture responsible for generating the output sequence. It follows a structure akin to the encoder but focuses on synthesizing information and producing a coherent output based on the knowledge acquired during the encoding phase.

To gain a comprehensive understanding, let's explore each component in detail:

- **Masked Multi-Head Self-Attention Mechanism:**

One of the decoder's distinctive features is the inclusion of the masked multi-head self-attention mechanism. This mechanism plays a crucial role during training in autoregressive tasks. This involves predicting the next element in a sequence based on the previously generated elements. This is often used in tasks like machine translation or text generation, where the model must generate a complete sequence one token at a time. Its primary function is to prevent the decoder from peeking into future positions in the sequence, ensuring that predictions are made based only on information available up to that point. This meticulous attention to temporal order enhances the accuracy of predictions during the training process. The "masked" part refers to a mechanism that prevents the model from attending to future tokens during the decoding process.

- **Multi-Head Attention Mechanism:**

Building on the knowledge acquired from the encoder's output, the decoder employs the multi-head attention mechanism. This mechanism enables the decoder to selectively attend to relevant portions of the input sequence. By leveraging information from the encoder, the decoder gains a deeper understanding of the context, significantly enhancing its ability to generate precise and contextually appropriate outputs.

- **Position-wise Feed-Forward Networks:**

Following the attention mechanisms, the decoder further refines its understanding through the application of position-wise feed-forward networks. Similar to the encoder, these networks operate independently on each position within the decoder. This step facilitates the capturing of intricate patterns and relationships in the data, contributing to the decoder's capacity to generate accurate and nuanced outputs.

In essence, the decoder in the Transformer architecture is a sophisticated module that plays a pivotal role in synthesizing information from the encoder and generating coherent output sequences. The integration of the masked multi-head self-attention mechanism, multi-head attention mechanism, and position-wise feed-forward networks showcase the depth of its functionality in achieving accuracy and context-awareness in the output generation process.

5. Output Layer:

Think of the Output Layer as the conclusion or final answer of the Transformer's work. After the Transformer has processed all the information and understood the story or data, it needs to tell us its findings.

- **Linear Transformation:**
This is like putting the finishing touches on the information. The Transformer reshapes and refines the data one last time before giving us the answer.
- **Softmax Activation Function:**
Imagine the Transformer assigning scores to different possibilities, like

how confident it is about different answers. The softmax activation function is like a filter that ensures the scores are easy to understand. It makes the final answer clearer and helps us see which option the Transformer is most certain about, especially in tasks like classification.

In simple terms, self-attention allows words to understand each other, positional encoding helps keep track of their order, and multi-head attention provides a more comprehensive view of their relationships, making transformers powerful in understanding and generating language.

Simplified Explanation of Transformer Components:

- **Self-Attention:**
Role: Imagine reading a story. Self-attention, which sits both in the encoder and decoder model, lets each word pay varying degrees of attention to all other words in the story, understanding their relationships.
Example: In the sentence "The cat sat on the mat," when focusing on "cat," self-attention helps understand its connection to other words like "mat."
- **Positional Encoding:**
Role: Since transformers read words in any order, they need to understand word positions. Positional encoding adds unique information to each word, telling the model where it is in a sentence.
Example: Without positional encoding, the model might mix up the meaning of "sat" if it appears at the beginning or end of a sentence.
- **Multi-Head Attention:**
Role: It's like having multiple sets of eyes. Instead of one attention mechanism, we have several, each looking at the input from different angles. This helps capture various aspects of relationships between words.
Example: In "The cat sat on the mat," one head might focus on the cat's action, while another pays attention to the location on the mat.
Bottom of Form

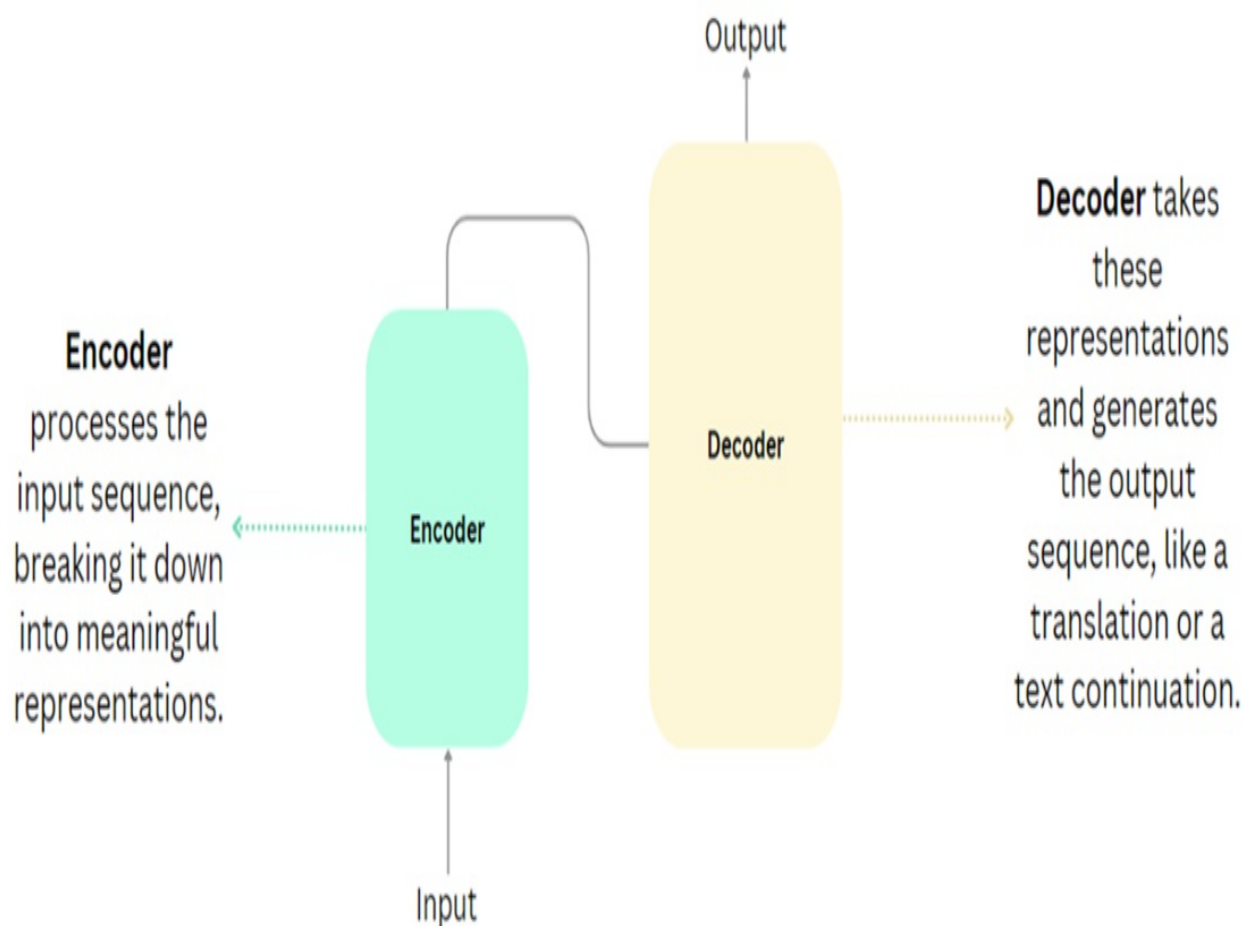
Now, moving slightly beyond the core of the Transformer architecture, we will explore the application layer of the encoder and the decoder model

layers. As stated earlier, these layers are fundamental in leveraging the attention mechanism, enabling the model to efficiently process and produce sequences of information that are both coherent and contextually aligned.

2.3 Encoder and Decoder Models

The original Transformer architecture manifested as a dual-functional entity, housing both an encoder and a decoder. Yet, within the expansive realm of Natural Language Processing (NLP), practitioners discovered utility in employing either or both of these components, contingent upon the specific task at hand.

Figure 2.4 A simplified illustration of the Transformer architecture focusing on the two major layers.



- **Encoder-Only Transformers (e.g., BERT):**
This variant exclusively utilizes the encoder segment of the architecture. Its purpose is to process a sequence of tokens, generating a dense representation for each token in the sequence. Models like BERT (explained in detail below), trained bidirectionally, engage in predicting intentionally masked words within a sentence. The efficacy of this approach has solidified BERT's foundational role in diverse tasks such as text classification and named entity recognition.
- **Decoder-Only Transformers (e.g., GPT):**
Conversely, this model focuses solely on the decoder component. Primarily designed for text generation, models like Generative Pretrained Transformers (GPT) take a given prompt and intricately craft coherent narratives or responses. Trained unidirectionally, they predict the next token in a sequence based on preceding tokens, making them adept at creative language generation.

2.3.1 The Encoder Models

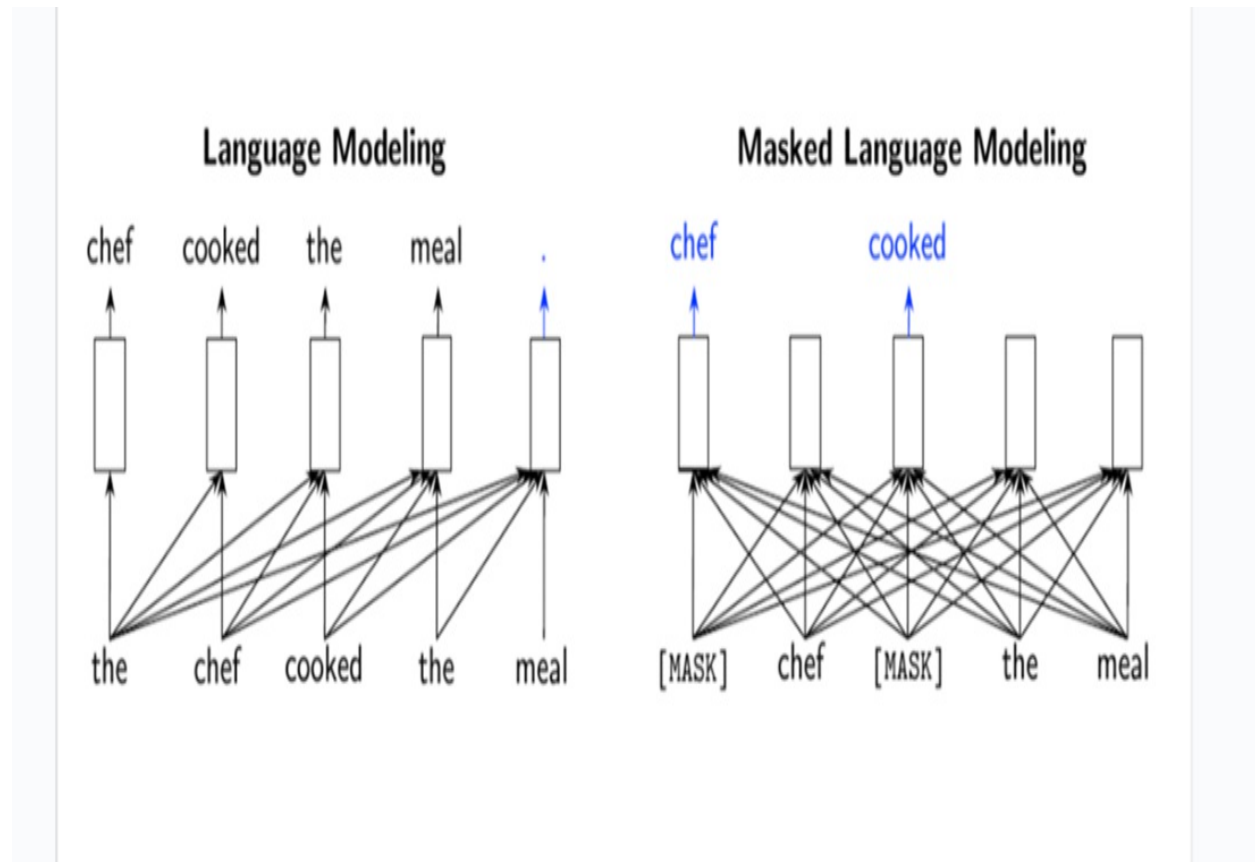
A significant breakthrough in NLP came with the advent of models like BERT (Bidirectional Encoder Representations from Transformers) and SBERT (Sentence-BERT), each representing a distinct phase in the evolution of encoder models.

BERT revolutionized the field of NLP by introducing a bidirectional approach to language understanding. Traditional language models, including the original Transformer, processed text in a unidirectional manner, considering context only from left to right or vice versa. BERT, on the other hand, leveraged bidirectional context by training on both left and right contexts simultaneously. This bidirectional pre-training allowed BERT to capture more comprehensive contextual information, leading to superior performance in various NLP tasks.

BERT's training involved predicting missing words in a sentence, known as the masked language model (MLM) objective. By considering context from both directions, BERT excelled at capturing intricate relationships between words, enabling it to understand the nuances and semantics of language more effectively. The contextual embeddings produced by BERT became valuable

representations for downstream NLP tasks, such as sentiment analysis, named entity recognition, and question answering.

Figure 2.5 The two images depict the difference between language modeling and masked language modeling which gives BERT an advantage over traditional left-to-right sequences.



Language Modeling: In the first image, the caption reads, "The chef cooked the meal". This exemplifies a basic language model, where each word is predicted based on the previous word.

Masked Language Modeling: The second image shows the caption, "The chef cooked [MASK] the meal". Here, the masked language model is trying to predict the missing word, "[MASK]". In this case, the most likely word is "the", based on the context of the sentence.

Building upon the success of BERT, Sentence-BERT (SBERT) introduced enhancements specifically tailored for semantic textual similarity tasks. While BERT was proficient in understanding the context of individual words,

SBERT aimed to capture the semantic similarity between entire sentences. The key innovation in SBERT was the introduction of siamese and triplet network architectures (both explained in callout section below) during training, emphasizing the creation of sentence embeddings that emphasize semantic relationships.

A brief intro to Sentence-BERT

In Sentence-BERT (SBERT), a significant advancement was made by introducing siamese network architecture and triplet loss function during the training process. This innovation specifically focused on generating sentence embeddings that highlight semantic relationships between sentences.

Siamese Network Architecture:

- A siamese network is a type of neural network that consists of two identical subnetworks (or twins) that share the same parameters and weights.
- The primary objective of a siamese network is to measure the similarity between two inputs, which in the case of SBERT, are sentences.
- In the context of SBERT, the siamese architecture is employed to compare pairs of sentences. The network is trained to understand the similarity or dissimilarity between two sentences by learning to distinguish between similar and dissimilar sentence pairs.
- The shared parameters enable the network to extract features and representations from each sentence independently while encouraging the model to produce embeddings that capture semantic relationships.

Triplet Loss Function:

- The triplet loss is a loss function applied to siamese networks to improve training, not a separate network architecture.
- It involves training on triplets of inputs: an anchor sentence, a positive sentence (similar to the anchor), and a negative sentence (dissimilar to the anchor).
- The objective is to minimize the distance between embeddings of the anchor and positive sentences while maximizing the distance between embeddings of the anchor and negative sentences.

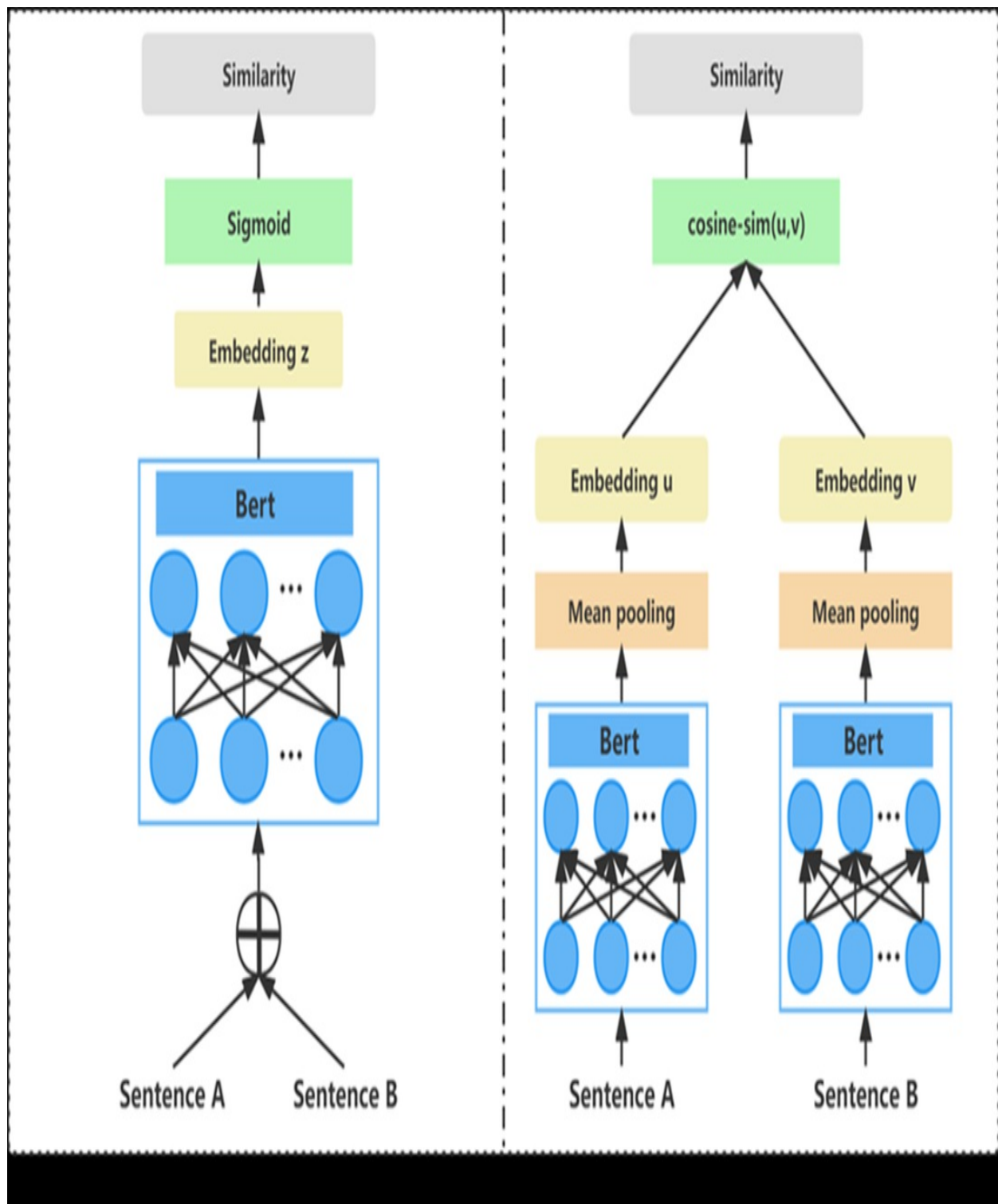
- This approach enhances the model's ability to generate embeddings that not only distinguish between similar and dissimilar pairs but also capture subtle semantic nuances.

Overall Objective:

- The introduction of siamese network architecture and triplet loss function in SBERT's training methodology is geared toward refining the creation of sentence embeddings.
- By training the model with pairs or triplets of sentences, SBERT is designed to understand and emphasize semantic relationships between sentences. This approach goes beyond merely considering individual sentence meanings and focuses on encoding the nuanced similarity or dissimilarity in meaning between pairs or triplets of sentences.
- The ultimate goal is to improve the model's ability to measure and represent the similarity between sentences, which is crucial for various natural language processing tasks.

In Siamese network training, SBERT learned to distinguish between similar and dissimilar sentence pairs, encouraging the model to generate embeddings that reflect semantic similarity. The triplet network approach further refined this process by incorporating triplets of sentences, optimizing the model to minimize the distance between similar sentences and maximize the distance between dissimilar ones. This training strategy enhanced SBERT's ability to understand and represent the semantic content of sentences more effectively than traditional models.

Figure 2.6 A distinction between Bert (on the left) and Sentence-Bert (on the right) lies in their approach to semantic textual similarity. In BERT, the two sentences are initially concatenated and then fed into the network. Subsequently, the similarity is computed using the activation function. Conversely, in Sentence-Bert, the two sentences are independently input into Bert, and their cosine similarity is then calculated.



The journey from the traditional Transformer architecture to BERT and subsequently to SBERT represents a trajectory marked by continuous improvement in capturing contextual and semantic information. While the

Transformer laid the foundation with its attention mechanisms, BERT introduced bidirectional context understanding, and SBERT specialized in semantic similarity. These advancements highlight the evolving landscape of encoder models, each iteration contributing to a deeper and more nuanced understanding of natural language.

2.3.2 The Decoder Models and its Meteoric

While Encoders have seen major innovation, Decoders have become more center stage in the recent past and will continue to play a major role in the future too. The evolution of decoder models within Transformer architectures offers a fascinating narrative of growth and increasing complexity, reflecting broader trends in artificial intelligence and machine learning. The most popular example of a decoder model is the GPT-3.5, developed by OpenAI. In this subsection, we will dive into the intricacies of the Decoder model, its architecture, and how it has significantly contributed to the success of text generation applications.

The Decoder model is an essential part of the Transformer architecture, working with the Encoder model to generate coherent and contextually relevant text. While the Encoder model processes the input sequence and captures its contextual information, the Decoder model takes this encoded representation and generates the output sequence one token at a time.

Traditionally, the decoder in a Transformer architecture works by taking the output of the encoder and transforming it step-by-step into a final output sequence. However, decoders have evolved beyond their initial designs to accommodate more sophisticated tasks and challenges in NLP and beyond. These models are particularly crucial in tasks that involve the generation of text, such as machine translation, text summarization, and even in generative tasks that require nuanced understanding and creation of human-like content.

The Decoder model consists of a stack of identical layers, each comprising several sub-components:

- Self-Attention Mechanism
- Encoder-Decoder Attention Mechanism

- Feed-Forward Neural Network

These components work together to enable the Decoder to generate text that is both grammatically correct and semantically meaningful.

The Self-Attention mechanism in the Decoder model plays a vital role in capturing the dependencies between the generated tokens. Unlike the Encoder's Self-Attention, which attends to all positions in the input sequence, the Decoder's Self-Attention is masked to prevent attending to future positions during the generation process. This masked Self-Attention ensures that the Decoder only considers the previously generated tokens when predicting the next token in the sequence. By attending to its own previous outputs, the Decoder can maintain coherence and generate contextually relevant text.

The technological advancement in decoder models has been characterized by enhancements in self-attention mechanisms, which allow these components to better handle longer sequences with more nuanced relationships. Improvements in training methodologies, such as more effective use of transfer learning and fine-tuning practices, have also enabled decoders to achieve state-of-the-art results in numerous benchmarks.

The Encoder-Decoder Attention mechanism is a crucial component that allows the Decoder to focus on relevant parts of the input sequence while generating the output. It takes the encoded representations from the Encoder and uses them to guide the generation process in the Decoder. At each generation step, the Decoder attends to the Encoder's outputs, assigning different attention weights to different positions in the input sequence. This attention mechanism enables the Decoder to selectively focus on the most relevant information from the input, ensuring that the generated text is coherent and aligned with the input context.

The Feed-Forward Neural Network (FFN) in the Decoder model is responsible for processing the attended information and producing the final output representation. It consists of two linear transformations with a ReLU activation function in between. The FFN takes the output of the Encoder-Decoder Attention mechanism and applies these transformations to generate a high-dimensional representation. This representation captures the relevant

information needed for generating the next token in the output sequence.

The Decoder model has significantly advanced the field of text generation by enabling the generation of coherent, fluent, and contextually relevant text. Some of the key applications and benefits of the Decoder model in text generation include language translation, text summarization, dialogue systems, and creative writing assistance. The success of the Decoder model in text generation can be attributed to its ability to capture long-range dependencies, maintain coherence, and generate contextually relevant output. By leveraging the power of attention mechanisms and deep neural networks, the Decoder model has pushed the boundaries of what is possible in text generation.

Since the introduction of the Transformer architecture, there have been numerous advancements and variations of the Decoder model. One significant development in the realm of decoders has been their increasing independence. In setups where detailed comprehension of complex context is not required, or the output needs to be particularly creative and diverse, architectures employing only the decoder have shown promising results. This is evident in models like GPT (Generative Pre-trained Transformer), where a stacked decoder handles both understanding the context and generating outputs based solely on that understanding, without the traditional encoder components.

The rise of decoder models has profound implications for the future of technology. With the increasing consumer demand for more intelligent, responsive, and context-aware systems, the decoder's role in AI becomes ever more crucial. Applications in automated content generation, real-time translation services, interactive bots, and sophisticated data analysis tools are just the beginning.

Looking ahead, the trajectory of decoder models points towards greater autonomy and insight. Innovations such as increased adaptability to different domains without extensive retraining, enhanced ability to handle ambiguity in human language, and improved ethical considerations in automated content generation are anticipated. Moreover, as we venture into more integrated AI systems, decoder models will likely play a critical role in unified architectures that are capable of handling diverse tasks across

different media types simultaneously. This will not only increase the efficiency of such systems but also open up new avenues for human-machine interaction.

Decoder models like those found in GPT-3.5 and GPT-4 have broad implications across various text-related applications, leveraging their advanced generation capabilities and deep understanding of context. Some significant applications include:

- **Content Creation:** These models can generate creative writing, such as poems, stories, and even full-length novels, by understanding and mimicking various writing styles and genres.
- **Copywriting:** Businesses use these AI models to produce high-quality marketing copy, product descriptions, and advertisements tailored to specific audiences.
- **Text Summarization:** They are capable of summarizing long documents, articles, or books into concise versions, saving time for readers and making information more accessible.
- **Chatbots and Virtual Assistants:** Decoder models power sophisticated conversational agents that can engage users in natural, context-aware dialogues, provide customer support, and automate responses in messaging platforms.
- **Email Drafting and Response Generators:** AI can help draft emails or suggest responses based on the context and content of received messages, enhancing communication efficiency.
- **Language Translation:** While primarily the domain of models that include both encoders and decoders, purely decoder-based models can also contribute by generating fluent text in the target language after an initial context-aware encoding step.
- **Academic and Technical Writing:** These models assist in generating structured text for use in academic papers, technical documentation, and reports, adhering to specific formatting and content guidelines.
- **News Generation:** AI can write articles and reports based on data inputs like financial reports or sports statistics, providing updates and insights almost instantaneously.
- **Scriptwriting for Videos and Podcasts:** Decoder models can create scripts for videos, movies, and podcasts, understanding genre-specific

nuances and audience expectations.

- **Educational Content:** They can generate educational materials, craft exam questions, provide detailed explanations on complex topics, and tutor students based on their learning needs.

The Decoder model in the Transformer architecture has revolutionized the field of text generation, enabling the production of coherent, fluent, and contextually relevant text. By leveraging the power of attention mechanisms and deep neural networks, the Decoder model has achieved remarkable success in various applications, including language translation, text summarization, dialogue systems, and creative writing assistance.

These applications highlight the versatility of decoder models in processing and generating text, showcasing their potential to transform how content is created and consumed across various sectors.

2.3.3 Combining the power of Encoders and Decoders

Is there a world where we can use the power of both? The answer is yes! I believe the true power of these models is unleashed when they collaborate as part of an Encoder-Decoder architecture (remember they can also be used individually). In this configuration, the Encoder processes the input sequence and generates embeddings, which are then fed into the Decoder to generate the desired output sequence. This collaboration is particularly powerful in sequence-to-sequence tasks.

Example: Semantic Search Use Case for enhanced retrieval

Semantic search represents a paradigm shift in information retrieval by focusing on the meaning and context of words rather than just keywords.

Suppose we have a vast collection of climate change articles written by multiple authors, including Dr. Jane Scientist. In a traditional keyword search, one might input "climate change" and "Jane Scientist" to find relevant articles. However, a semantic search dives deeper. Imagine a user enters the query, *"Impact of climate change on marine ecosystems Jane Scientist."* A semantic search engine, understanding the user's intent, retrieves articles

where Dr. Jane Scientist discusses the specific impact of climate change on marine ecosystems. It considers her unique insights, the context of each article, and the relationships between various concepts.

In this approach, the encoder plays a pivotal role in understanding the semantic content of the input data, such as user queries or document contents. The encoder processes and transforms text into dense numerical vectors, capturing the nuanced relationships and context between words.

Important

Encoders are needed when we deal with vast amounts of data. For instance, if we have hundreds of documents in our climate change repository, we will need to convert them into embeddings so that we can build a retrieval engine. However, if we need to summarize just one document, we necessarily don't need an encoder model, the decoder model can do that for us, given its context length can fit the length of the article. We will cover context length in upcoming chapters.

Once the information is encoded into vectors, the decoder takes center stage. It decodes these vectors, generating coherent and contextually relevant responses or representations. In the context of semantic search, this could involve retrieving documents or responses that not only match keywords but also align with the underlying meaning or intent of the user's query. Let's explore a real-life case study of this very semantic search. I give you my very own creation: *Travelle*.

2.4 Case Study: A Hotel Search Engine Utilizing Encoder & Decoder Models

To deepen our understanding, we're introducing our first case study: Travelle. This search engine, which I have developed, exemplifies the combined strength of Encoder and Decoder models in a practical application. We will cover individual aspects of building this engine in the upcoming chapters.

So, what sets Travelle apart? Unlike conventional search platforms that depend significantly on fixed parameters such as location, check-in, and

check-out dates, Travelle initiates a more dynamic and user-friendly search experience. Here, users can express their specific hotel preferences and needs through natural language queries. These queries can range from general descriptions like "quiet hotels with ocean views and a spa that offers couples massages" to more precise requests like "a hotel which has a room with a view of the Eiffel Tower". This flexibility and intuitiveness in search functionality fundamentally alter how users interact with hotel booking platforms.

Figure 2.7 The landing page of the actual website Travelle, currently it provides recommendations for hotels in Paris. You can visit this site at <https://travelle.traversaal.ai>

travelle

A new way to search for hotels!

Be as descriptive as possible and we will try to find the
best possible match for you in Paris.

Q A hotel which has a room with a view of the Eiffel Tower

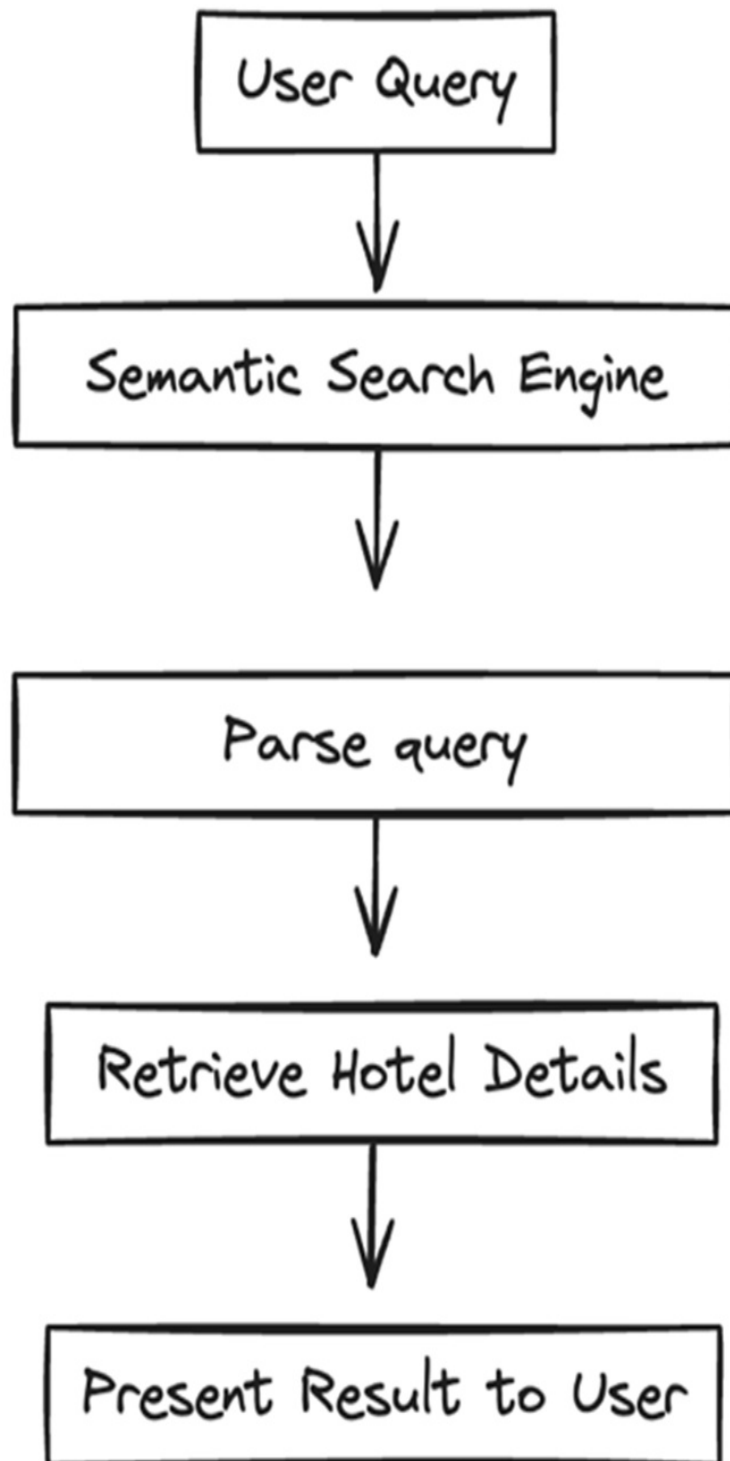
Search



At the core of Travelle's functionality is the encoder model, which interprets and analyzes the hotel descriptions and extensive user reviews housed in its database. This model is intricately designed to grasp the semantic essence of both the content in the database and the user queries. By deploying techniques discussed in the next chapter on Semantic Search, the encoder effectively maps out the deep, nuanced meanings embedded within texts rather than merely identifying superficial keyword matches. This capability allows it to understand, for instance, that a "peaceful" hotel implies a lack of noise and bustling activity, even if the reviews do not explicitly use that exact

word.

Figure 2.8 A simplified overview of the functionality of Travelle's Semantic Search Engine



After processing the input data with its sophisticated encoder, Travelle employs a decoder model to generate personalized hotel recommendations. This process, termed 'Retrieval Augmented Generation' (RAG), combines the innovative process of retrieving relevant information with the creative capacity of generating text-based content. The decoder accesses encoded representations of hotel characteristics and user reviews, matching these against the user's query to find the most suitable options.

Here's how the system works in practice: Suppose a user types "looking for a tranquil weekend getaway within 50 miles from Los Angeles, must have a spa and accept dogs". The encoder first processes this input to understand the underlying request, while also mapping available hotel data to detect which properties meet these criteria. The decoder then kicks in, synthetically generating a list of recommendations. Each suggestion is accompanied by a tailored explanation, detailing why each hotel is a match — perhaps highlighting the property's award-winning spa, its proximity to good dog parks, or its special quiet-zone floors.

The outputs provided by the decoder are unique not only in their relevance but also in their presentation. Building on the encoder's semantic understanding, the decoder can craft responses that reflect an understanding of context and user intentions. This creates a more engaging user experience, making the user feel understood and well-guided in their search for the perfect hotel. In our example in the figure above, "a hotel which has a room with a view of the Eiffel Tower" – we can see how good the result is since the very first "relevant" review mentions a room with a view of the Eiffel Tower.

Figure 2.9 Search result from the query about finding a hotel with a room that has view of the Eiffel Tower

Search Results



Hotel Duquesne Eiffel

Relevance Score: 85.25%

Description:

This cosy hotel offers numerous advantages such as its geographic location, appreciated as much for "leisure" vacations as it is for business meetings. Between "les Invalides" and the Eiffel Tower, the Duquesne Eiffel Hotel is located in one of Paris' most pleasant boroughs on the famous left bank of the Seine. The team welcomes you with a smile 24/7, while taking all the necessary measures to ens...[Read More](#)

Relevant Reviews:

1. The photos for this hotel do not do it justice. I have a front facing room with a view of the Eiffel Tower and it's just spectacular for what it cost. You don't need the Shangri La with this hotel. Very friendly staff with quick check in. I was supposed to be here with my husband but his passport didn't have long enough (make sure you have at least 3 months if you're coming from the UK) and they S...
2. The location of this hotel couldn't be beat. We had the top corner room which had a window literally opening up on the Eiffel Tower with a wraparound balcony to boot. Exceptionally comfortable bed, walking distance to the Tower and a great neighborhood. I found this hotel using the search "Eiffel Tower" and "Balcony" and this couldn't have been more on point to meet that criteria. Also the breakf...

[Close Reviews](#) ^

As Travelle continues to refine its use of encoder and decoder models, potential enhancements include more granular personalization, considering past searches and user preferences, and integrating real-time availability and pricing data to make the platform even more robust and user-friendly. The continual improvement in natural language understanding and generation models also promises to further elevate the effectiveness and intuitiveness of platforms like Travelle, ultimately making them indispensable tools for

modern travelers. In summary, Travelle, through its innovative application of encoder and decoder models, not only enhances the efficiency of hotel search but fundamentally alters the way users interact with travel platforms, offering a deeply personalized and context-aware search experience.

Hopefully, we now have a clearer understanding of the pivotal roles played by encoder and decoder models. These applications will be covered in greater detail as we move through our upcoming chapters. In this chapter, we covered the important components that make Transformers stand out and the important roles played by encoder and decoder models. In our next chapter, we will focus on what it takes to build a basic LLM application.

2.5 Summary

- The Transformer architecture is an outcome of various iterations and evolution of language models, starting from rule and probability-based models to neural models to finally their current state.
- Neural models such as Recurrent Neural Nets and Gated Recurrent Nets, while extremely revolutionary struggled with computational inefficiencies and the inability to parallelize processing effectively.
- These limitations led to the search for a more efficient and scalable architecture, culminating in the development of the transformative Transformer model.
- At the heart of it, the Transformer architecture has a “Self-Attention Mechanism” which provides comprehensive context consideration, capturing long-range dependencies.
- Self-attention allows words to understand each other, positional encoding helps keep track of their order, and multi-head attention provides a more comprehensive view of their relationships, making transformers powerful in understanding and generating language.
- The original Transformer architecture manifested as a dual-functional entity, housing both an encoder and a decoder. Yet, within the expansive realm of Natural Language Processing, practitioners discovered utility in employing either or both of these components, contingent upon the specific task at hand.
- Encoder Model extracts meaningful representations, crucial for capturing semantic and syntactic features with application span text

summarization and sentiment analysis.

- A significant breakthrough in natural language processing came with the advent of models like BERT (Bidirectional Encoder Representations from Transformers) and SBERT (Sentence-BERT), each representing a distinct phase in the evolution of encoder models.
- Decoder Model, on the other hand, generates output sequences leveraging self-attention, cross-attention, and feed-forward networks. Applied in language translation and text generation. These models have become center stage in the recent past and have shown meteoric growth in a very small passage of time.
- Combining the power of the Encoder and Decoder models allows us to create phenomenal products such as Search Engines that can truly understand user preferences and create a new sense of personalization.

3 Encoder models in action:

Semantic-Based Retrieval Systems

This chapter covers

- A general overview and evolution of information retrieval systems
- Introduction to keyword-based search
- Building semantic search with encoders
- Semantic similarity metrics

The previous chapter provided a comprehensive overview of the transformer architecture, including explorations into the encoder and decoder models, followed by an overview of self-attention mechanisms, positional encodings, and multi-head attention. Building upon those core concepts, this chapter delves deeper into encoder models and one of their most impactful applications: semantic-based information retrieval (IR) systems.

An IR system is a software system that stores and manages information (documents, web pages, articles, and others) and provides interfaces for users to find and access the information they need. At a high level, information retrieval systems have three core functions:

- Indexing - Analyzing the content of documents/information and creating an index to represent them. This includes extracting important keywords, concepts, etc. that can help retrieve the documents later.
- Storage - Storing the documents/information and the associated indexes into databases that allow fast lookup and retrieval. This storage must be scalable as the amount of information grows.
- Search and retrieval - Providing a search interface to users where they can specify queries and using the indexes to quickly find and return the relevant information. The matches are ranked by relevance.

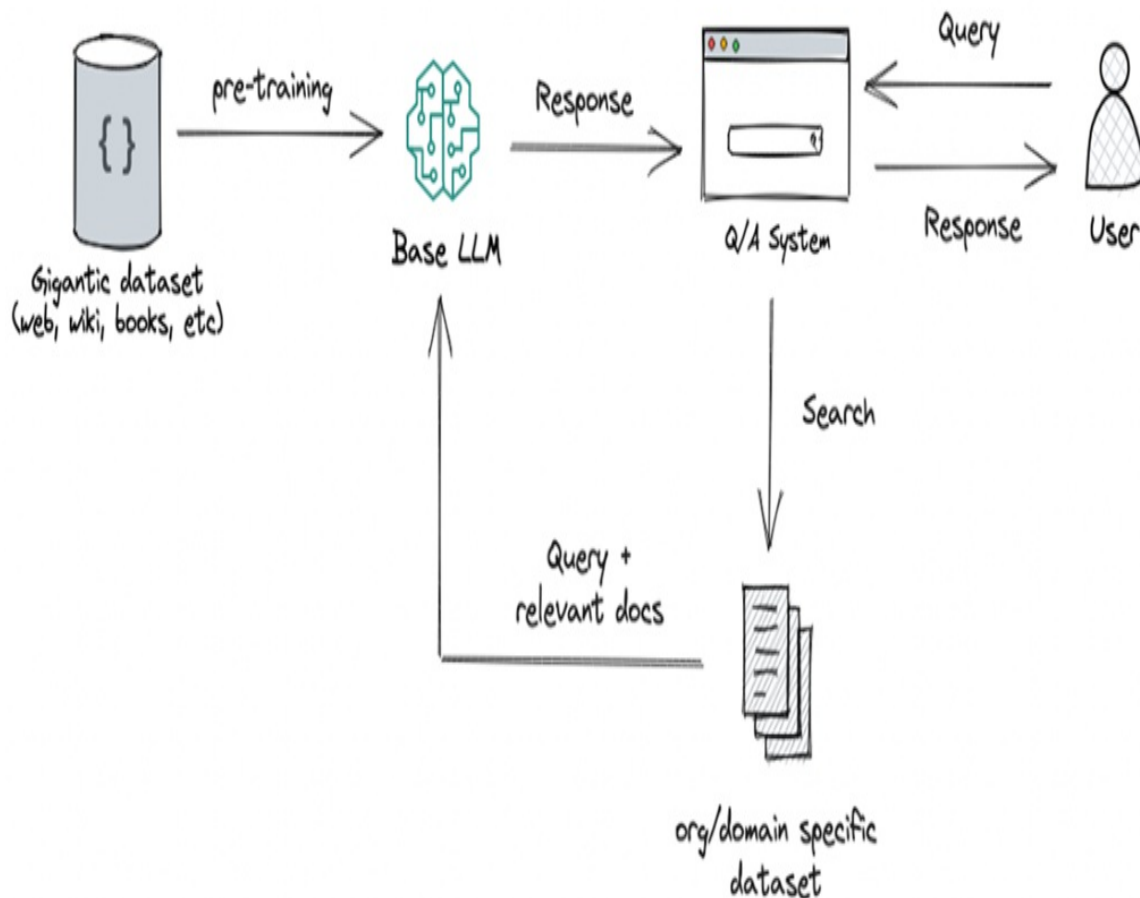
Some examples of commonly used information retrieval systems:

- Web search engines like Google that index web pages and allow searching for relevant content.
- Enterprise search systems for searching through internal company documents and data.
- E-discovery systems are used by lawyers to search large collections of legal documents.
- Library catalogs that manage metadata and indexes for finding books.

In essence, an IR system ingests information, analyzes it, stores it in an easily searchable format, and makes it available for users to search through quickly and conveniently even as the content scales up. Retrieving precise information from large corpora is the central goal.

In contrast, semantic-based IR, is a search engine that combines pre-trained language models (both encoders and decoders) with document retrieval to enhance the generation of responses in question-answering and other natural language processing tasks. This form of search is more commonly known as Retrieval augmented generation (RAG).

Figure 3.1 Recap from Chapter 1: Retrieval Augmentation Generation is used to enhance the capabilities of LLMs, especially in generating relevant and contextually appropriate responses.



It operates in two stages:

- first, given a query, it retrieves relevant documents or passages from a large corpus of text using a neural retrieval method. This retrieval is semantically driven, meaning that it looks for material based on the contextual meaning rather than just keyword matching.
- Second, the retrieved documents are provided as additional context to a language generation model (like GPT-4), which synthesizes the information into a coherent response. By leveraging both retrieved knowledge and the model's pre-existing language understanding capabilities, RAG systems aim to produce more accurate, informative, and contextually relevant answers.

While we focus on semantic IR systems in this chapter, it is important to note that the encoder model has been pivotal to advances in other domains as well. One other prominent application is machine translation (MT), where the encoder maps input in one language to vector representations that the decoder then transforms into another language. State-of-the-art MT models like Google's Transformer rely entirely on the encoder-decoder framework.

Outside of MT, the Encoder model underpins natural language understanding capabilities in dialogue systems and question-answering systems. By encoding inputs into vector representations that encapsulate semantic meaning, the Encoder allows downstream tasks to leverage this understanding. In later chapters, we will highlight more applications where Encoders have driven breakthroughs. For now, our focus resides on how Encoders facilitate semantic matching in IR systems.

3.1 Information Retrieval Systems: A Historic Overview

IR systems have become deeply integrated into modern life, most visibly manifesting as internet search engines. These systems have undergone tremendous evolution over the past few decades to meet the demands of delivering relevant results at a massive scale to billions of users globally.

Note

Consider the world's most popular search engine, Google, which processes approximately 8.5 billion queries per day. As internet users have grown reliant on search engines for information access, search queries have become more sophisticated in complexity and intent. In turn, semantic search has rightfully taken center stage, as contextual understanding promises superior relevance compared to keyword matching alone. The most capable semantic search models that can truly grasp language nuance and meaning stand to become the most powerful information retrieval tools in existence today.

The foundations of IR systems and search technology trace back to the post-World War II era when visionaries first conceptualized advanced computers

and information networks. In July 1945, Vannevar Bush published an article titled "As We May Think" in "The Atlantic Monthly." In this article, he proposed a system called the "Memex" (short for "Memory Extender"), which was envisioned as a desk-like device where an individual could store and retrieve vast amounts of information, including books, personal records, and other materials. Even in those pioneering days, early researchers realized that managing the storage and indexing of rapidly growing information would enable more efficient retrieval and knowledge discovery. This planted the seeds for the first primitive experimental IR systems in the 1950s and 1960s.

Over the next few decades, IR systems evolved from specialized tools for indexing scientific literature and documents into far more expansive search engines powering the Internet age. The origins of modern search behemoths like Google and Bing can be traced back to 1990s systems like Archie, Veronica, and Lycos that introduced web-scale keyword indexing and text matching. These early search engines laid the groundwork for a revolution in public access to information.

As internet adoption grew in the 2000s, IR systems became deeply integrated into digital life for billions worldwide. People increasingly relied on search engines like Google to instantly meet their information needs. In response, IR innovated rapidly over a short span to keep pace with skyrocketing demands for relevance, speed, and scale. Search queries grew more sophisticated over time. In the next two sections, we will cover two major examples of approaches to implementing IR, keyword search and semantic search using encoder models.

3.2 Keyword Search using Inverted Index and TF-IDF

Keyword search has long been the most prevalent approach for information retrieval from document collections and databases. At its core, keyword search aims to match a user's query terms to relevant documents solely based on overlapping keywords or phrases present in both, such as *Nike blue shirt*. Keyword queries tend to be succinct, typically less than four words, with the

search engine placing focus on individual terms instead of holistic meaning. We will implement a step-by-step example of a popular approach, *Term Frequency - Inverse Document Frequency (TF-IDF)* in the upcoming section, in Python.

The key data structure that enables efficient keyword search is called an *inverted index*. This creates essentially a lookup table that maps unique words or tokens to documents containing them. For example, the word "neural" may map to documents with IDs 1, 3, and 5, indicating those documents mention the word, "neural". This data structure allows quick scanning for matching documents when a query contains that term, rather than having to scan through every single document's text to find matches.

A Primer on Inverted Index

We call this index "inverted" because instead of mapping documents to their tokens like a traditional index, it maps terms or *tokens* to the documents containing them. This inverted structure allows fast lookups of documents matching a term during the search.

Here are the steps used to build a basic inverted index:

Start with text documents to be searched:

Doc 1: "Machine learning approaches leverage neural networks".

Doc 2: "Classical AI relied on rules and decision trees not neural networks".

1. Tokenize the documents:

Tokenization refers to splitting the text into individual words or tokens that become the terms to be indexed. Tokens can be words, phrases, or any meaningful chunks, depending on the context. After tokenizing, the documents look like:

Doc 1 Tokens: ["Machine", "learning", "approaches", "leverage", "neural", "networks"]

Doc 2 Tokens: ["Classical", "AI", "relied", "on", "rules", "and", "decision", "trees", "not", "neural", "networks"]

2. Create an empty inverted index data structure mapping tokens to

document IDs:

Inverted index = { "machine": [], "learning": [], "approaches": [], ... }

3. Populate the index by iterating through the tokens of each document and appending the document's ID to the token's list.

For the token "neural" token in Doc 1, append 1 to index["neural"].

Inverted index = { "neural": [1], ... }

4. Repeat for all tokens across documents.

If "neural" also appears in Doc 2, append 2 to the list.

Inverted index = { "neural": [1, 2], "machine": [1], ... }

5. Now to find documents with "neural", we can directly scan the list under that term, without needing to scan sequentially through all documents!

Once an inverted index is constructed, the system needs an algorithm to score and rank search results. This is where TF-IDF comes into play. This technique assigns numeric weights to terms based on two statistics - term frequency (TF) and inverse document frequency (IDF).

TF measures how often a term appears within a document. A higher term frequency indicates greater relevance of that word to the document text. TF is often normalized by the document's length, so it does not unfairly favor longer documents. The following function illustrates the corresponding calculation:

$$TF = \frac{\text{number of times the term appears in the document}}{\text{total number of terms in the document}}$$

Suppose we have a document that consists of the following text: "The quick brown fox jumps over the lazy dog. The fox is quick and the dog is lazy."

Let's say we want to calculate the term frequency for the term *fox* within this document. First, we count the number of times the term "fox" appears in the document. In the given text, "fox" appears twice. Now, we count the total

number of terms (words) in the document. The document contains the following words (after breaking it into tokens and assuming no filtering like stop-word removal):

"The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog", "The", "fox", "is", "quick", "and", "the", "dog", "is", "lazy"

That gives us a total of 18 terms. Now, we apply the term frequency formula:

TF = (Number of times the term "fox" appears in the document) / (Total number of terms in the document)

TF = 2 / 18 $\approx$ 0.1111 (rounded to four decimal places)

So, the term frequency for "fox" in this example document is approximately 0.1111, which means the term "fox" composes roughly 11.11% of the document's terms. Please remember that for real-world applications, typically additional preprocessing steps (like lowercasing, stop-word removal, stemming, etc.) would be applied to the text before calculating term frequencies. This ensures a more accurate and useful representation of term importance within the document.

On the other hand, *inverse document frequency*, the IDF part of TF-IDF, measures how common or rare a word is across the entire document collection, called corpus. Words occurring in many documents have lower IDF and less distinguishing significance compared to niche words contained only in a subset of documents. IDF reduces the weights of ubiquitous words like "the" which would otherwise dominate scoring.

$$IDF = \log \left(\frac{\text{number of the documents in the corpus}}{\text{number of documents in the corpus contain the term}} \right)$$

The final TF-IDF score combines these factors. Examples with higher TF-IDF scores for query terms get ranked higher, as they prominently and

distinctively feature those relevant keywords.

When a user enters a search query, the system tokenizes the text into keywords, looks up matching documents from the inverted index, computes TF-IDF scores between query keywords and potential matches, and ranks results by these relevance scores. The top-scoring search hits are returned to satisfy the user's query based on keyword matching. document also needs to be assigned a numeric score reflecting its relevance to potential search queries.

Let's review all the newly introduced terminologies:

Inverted index:

- It is a data structure that maps words/terms to documents that contain them
- Allows quick lookup of documents matching a term during the search
- Enables scalable and fast keyword-based search

TF-IDF:

- It is a scoring mechanism, not a data structure
- Used to compute a weight reflecting how important a word is to a document
- TF part measures how frequently a word occurs in the document
- IDF part measures how rare the word is across all documents
- Words with high TF-IDF are considered informative for that document

In summary:

- The inverted index enables efficient keyword lookups and matches
- TF-IDF scores each match to determine the most relevant results
- An inverted index facilitates fast matching of documents based on keyword search. TF-IDF then ranks the matching documents by how relevant they are likely to be to the search query. The inverted index provides speed and scalability, while TF-IDF provides smarter ranking. They work together to power keyword search engines.

3.2.1 Implementing keyword search using inverted index and TF-IDF

To emulate how most keyword search engines work and understand the implementation of TF-IDF and keyword search better, let's build a sample search step by step using Python.

Part 1:

In this step, we will build an inverted index from a set of example sentences. The inverted index will map each unique word to the sentences in which it appears, allowing for efficient text search and retrieval.

The code performs the following steps:

- It imports the necessary libraries: nltk for natural language processing and math for mathematical operations.
- It downloads the 'punkt' resource from NLTK, which is a pre-trained tokenizer.
- It defines a corpus called examples that contains six sentences as strings in a list.
- It tokenizes each sentence in the examples list using the word_tokenize function from NLTK, which splits the sentences into individual words. The resulting tokenized sentences are stored in the tokenized_docs list.
- It initializes an empty dictionary called index to store the inverted index

Listing 3.1 Create tokens and inverted index

```
import nltk #A
nltk.download('punkt')
from nltk.tokenize import word_tokenize
import math

examples= [
    "The cat is playing in the garden",      #B
    "A dog and a cat are good pets",
    "Cats love to chase mice",
    "Machine learning is based on algorithms",
    "Deep learning uses neural networks",
```

```

        "Recurrent networks have connections"
    ]

    tokenized_docs = [word_tokenize(d) for d in examples] #C

    index = {} #D

    for i, doc in enumerate(tokenized_docs):
        for word in doc:
            if word not in index:
                index[word] = []
            index[word].append(i)

    print(f'Inverted Index:\n {index}')

```

The print function will produce an output that will display the inverted index, which maps each unique word to the indices of the documents in which it appears. This inverted index can be used for efficient text search and retrieval tasks.

```

Inverted Index:
{'The': [0], 'cat': [0, 1], 'is': [0, 3], 'playing': [0], 'in':

```

Part 2:

Once we have created the inverted index, we define a function called `tfidf` that calculates the TF-IDF (Term Frequency-Inverse Document Frequency) weight for a given word in a specific document. TF-IDF is a numerical statistic that reflects the importance of a word in a document within a collection of documents. The function takes two parameters: `word` (the word for which to calculate the TF-IDF weight) and `id` (the index of the document in the `tokenized_docs` list). The function computes the term frequency (TF) by counting the number of times the word appears in the document, and the inverse document frequency (IDF) by taking the logarithm of the total number of documents divided by the number of documents containing the word. The TF-IDF weight is then calculated by multiplying the TF and IDF values. This weight represents the importance of the word in the specific document, considering both its frequency within the document and its rarity across the entire collection of documents.

Listing 3.2 Create TF-IDF Function

```
def tfidf(word, id):    #A
    tf = tokenized_docs[id].count(word)    #B
    df = len(index[word]) if word in index else 0    #C
    idf = math.log(len(examples) / (df+1))    #D
    return tf * idf
```

Part 3:

The final step is to perform a search query using the inverted index and calculate the relevance scores for each document based on the TF-IDF metric. The code follows these steps:

- The relevance scores are based on the TF-IDF (Term Frequency-Inverse Document Frequency) metric.
- The query is tokenized into individual words.
- For each query word, the code retrieves the documents containing that word from the inverted index.
- The TF-IDF score is computed for each query word in each document.
- The scores are accumulated for each document.
- The documents are ranked based on their relevance scores in descending order.
- The ranked examples are printed along with their scores.

Listing 3.3 Search and rank items

```
query = "machine learning" #A
query_words = word_tokenize(query)

scores = {id:0 for id in range(len(examples))} #B
for q in query_words:
    if q in index:
        for id in index[q]:
            scores[id] += tfidf(q, id)

ranked = sorted(scores.items(), key=lambda x: x[1], reverse=True)

for id, score in ranked:    #D
    print(f"Example {id}: { examples [id]}")
```

```
print(f"Score: {score}")
```

The final output from the above code will display a ranking of each example in descending order of relevance

```
Example 3: Machine learning is based on algorithms
Score: 1.791759469228055
Example 4: Deep learning uses neural networks
Score: 0.6931471805599453
Example 0: The cat is playing in the garden
Score: 0
Example 1: A dog and cat are good pets
Score: 0
Example 2: Cats love to chase mice
Score: 0
Example 5: Recurrent networks have connections
Score: 0
```

Based on the TF-IDF scores computed above for the query *Machine learning*, we were able to retrieve example #3 as the most relevant result from the corpus. The scoring algorithm appropriately surfaced this example to the top since it contains multiple occurrences of the exact search phrase.

However, this rudimentary keyword-matching approach is far from foolproof. We notice that example #4 has the second-highest score since the word *learning* appears in it. This illustrates the inherent limitations of TF-IDF and keyword-based search-matching examples may have high term overlap and statistical relevance, yet no actual semantic relevance.

The root issue is that keyword searches lack true comprehension of linguistic context and meaning. It performs a simplistic pattern match, unaware that example #4 discusses neural networks rather than machine learning. For all its computational efficiency, keyword search still struggles with the nuances, meaning, and implications of human language.

The more advanced semantic search techniques powered by Transformers have emerged to address these gaps. By encoding the contextual meaning and position of words and phrases, semantic models can retrieve results based on conceptual relevance rather than just statistical keyword matches. However, these complex neural models trade the performance and frugality of keyword

searches for accuracy. In the next section, we will explore semantic search and how it aims to deliver search experiences closer to human understanding.

3.3 Semantic Search from Scratch

Semantic search is an approach to IR that seeks to improve search accuracy by understanding the searcher's intent and the contextual meaning of terms as they appear in the searchable database. Unlike traditional search engines, which only look for exact matches of query terms, semantic search systems consider the context of the search and the location and interlinking of words within a document to deliver more accurate results.

The foundation of semantic search lies in the semantics, or meaning, of the words and phrases used in the search query. For example, a traditional search for *dog* would return all examples containing that exact term. A semantic search, however, may also return results related to puppies, canines, and pets because the system understands these terms are related. This is achieved by calculating the vector similarity of the words, dog and puppies, canines, and pets.

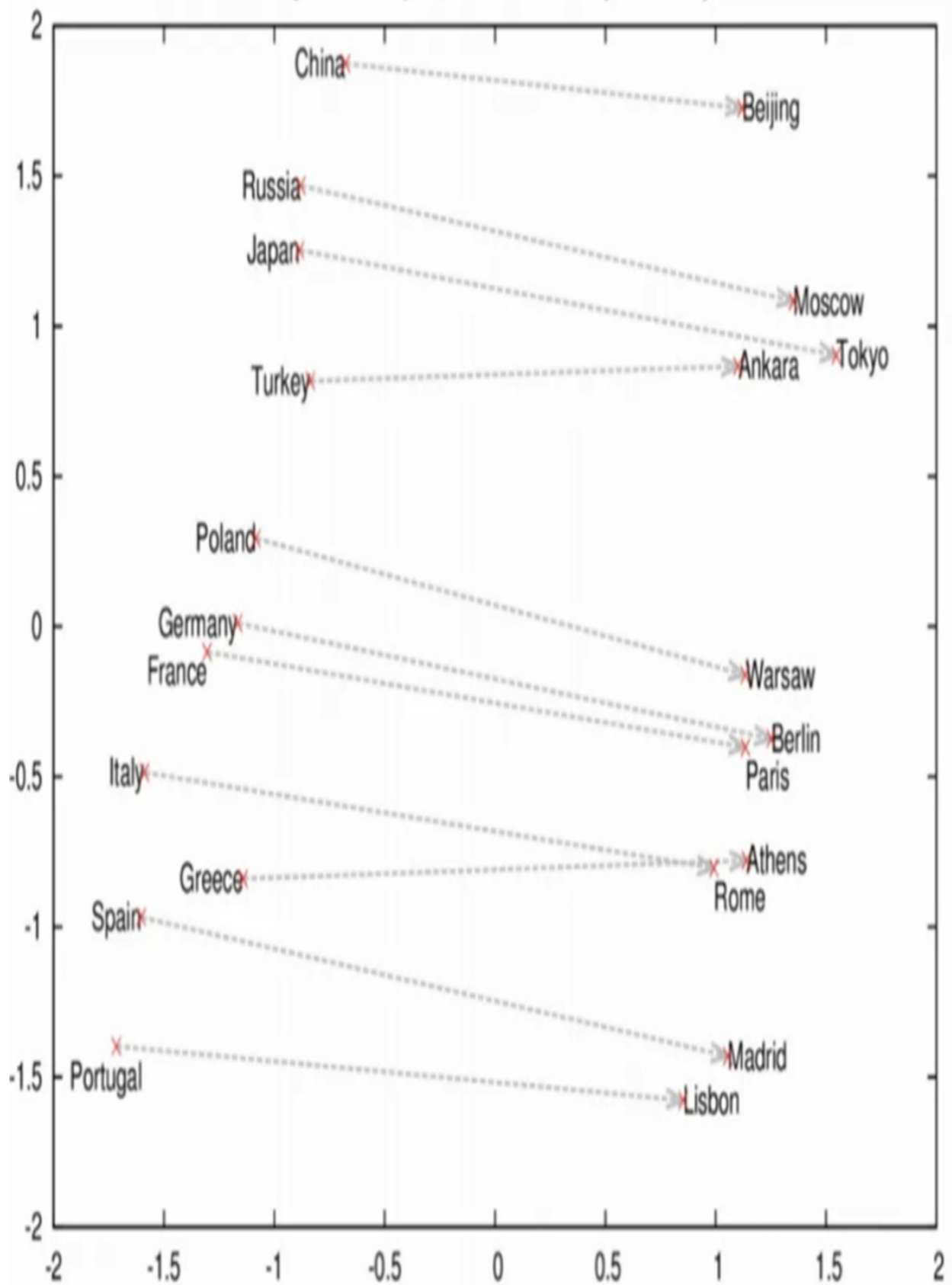
A vector is just a fancy word for a list of numbers that a computer uses to represent different examples. So, when you ask the search engine for dog, it doesn't just look for the word that says *dog*, it also looks at all the words that are similar to it. This is vector similarity

This means, thanks to vector similarity, you'll get search results that are about all things related to *dog*, not just the times when the exact word *dog* was used. The computer does this by measuring how close or similar the vectors (remember, lists of numbers) for words like *puppies*, *canines*, and *pets* are to the vector for *dog*.

It's like having a super smart search engine that knows all about word families and can figure out what you're really interested in, even if you don't use the exact right word. Here is an illustration of how country – capital combinations all form similar vectors in a 2d space.

Figure 3.2 Imagine a map where countries are placed based on how similar their names are. This plot is like that map but for words instead of countries. Words close together have similar

meanings. For example, China is closer to Russia than France because their names are more related.



Semantic search holds significant potential for enhancing the way we find and use information. It can handle ambiguity and understands the context better than traditional search methods. This can be particularly useful in fields like healthcare, where a search for a condition like "heart attack" should also return results related to "myocardial infarction," a term used by medical professionals.

However, implementing semantic search is not without challenges. Understanding and interpreting human language is complex, and creating systems that can understand context and nuances in language is a difficult task. Despite these challenges, continuous advancements in technology are pushing the boundaries of what is possible in semantic search, making it an exciting area to watch in the coming years.

3.4 Implementing Semantic Search with Python and Sentence Transformers

In this section, we will build our semantic search system. Implementing semantic search involves a myriad of steps, from data collection and pre-processing to embedding creation and search execution. This section will delve into the process of implementing semantic search using Python and Sentence Transformers, a powerful library for computing dense vector representations for sentences and paragraphs, using the Encoder models.

The first step in implementing semantic search is installing the necessary libraries. You will need *Sentence Transformers*, *SciPy*, and *Pandas* for this implementation. The *Sentence Transformers* library as mentioned above, is used for creating sentence and paragraph embeddings, *SciPy* for scientific computations including cosine distance measure (more on this in the next steps), and *Pandas* for handling our data. Recall: we covered BERT and SBERT in the previous chapter as different variations of Encoder models. Sentence Transformers have many pre-trained encoder models under its umbrella, and it was created by the creators of SBERT.

Figure 3.3 Simplified steps to follow in building semantic search.

Step 1: Install & Import Python Libraries



Step 2: Load Encoder Model



Step 3: Create Embeddings on examples



Step 4: Embed query and run similarity search



Step 5: Return results

Step 1A: Install the required libraries

You can install these libraries using pip, which is a package installer for Python. Run the following command in your terminal:

```
pip install sentence-transformers scipy pandas
```

Step 1b: Import Libraries and Load Data

After installing the required libraries, you need to import them into your Python script. The import statement in Python allows you to use the functions and methods defined in these libraries.

```
from sentence_transformers import SentenceTransformer
import scipy.spatial
import pandas as pd
```

Note: If you are running this code in a Jupyter notebook, you might encounter the following error that can be ignored:

```
"TqdmWarning: IPProgress not found. Please update jupyter and ipyw
```

To load our data, we will use the same documents we used in the keyword search.

```
examples = [ #A
    "The cat is playing in the garden",
    "A dog and a cat are good pets",
    "Cats love to chase mice",
    "Machine learning is based on algorithms",
    "Deep learning uses neural networks",
    "Recurrent networks have connections"]
```

Step 2: Load Pretrained Encoder Model

The next step is to load a pre-trained model from Sentence Transformers. This model will be used to create embeddings for our data. For this tutorial, we will use the 'all-MiniLM-L6-v2' model, which provides a good balance of performance and speed:

```
model = SentenceTransformer('all-MiniLM-L6-v2')
print(model)
```

The following output gives a great overview of the model:

```
>>> SentenceTransformer(
>>> (0): Transformer({'max_seq_length': 256, 'do_lower_case': Fal
>>> (1): Pooling({'word_embedding_dimension': 384, 'pooling_mode_
>>> (2): Normalize()
>>> )
```

This is showing the architecture of the Sentence Transformer model, which has 3 main components:

- Transformer
 - This is a BERT model that encodes text into embeddings.
 - `max_seq_length`: The maximum number of tokens per sequence the BERT model can handle. Sentences longer than that will not be read.
 - `do_lower_case`: Whether to convert all text to lowercase before feeding into BERT.
- Pooling
 - Aggregates the token embeddings from BERT into a sentence embedding.
 - `word_embedding_dimension`: The length of each token embedding vector output by BERT. Here it is 384. Think of this as different dimensions, that is quite a bit more than our 2d example with countries and corresponding capitals above.
 - `pooling_mode_mean_tokens`: Applies mean pooling, which averages BERT token embeddings.
- Normalize
 - L2 normalizes (see below) the sentence embeddings for better similarity matching.

In summary, it feeds text sequences into a BERT Transformer to generate token embeddings, aggregates via mean pooling and normalizes the sentence embeddings before output.

Primer on Normalization

L2 normalization for sentence transformers is a simple but important technique to improve the quality and usefulness of sentence embeddings.

Here is a basic explanation:

- The sentence transformer model generates an embedding vector for each input sentence. This is a vector with a fixed length, like 384 numbers representing the semantic meaning of the sentence.
- These raw embedding vectors have no constraints on the scale or distribution of the values. Some may have large values, others small values. Their magnitudes may be inconsistent.
- L2 normalization rescales each vector to make its length = 1, while keeping direction same. This puts them all on equal footing.
For example, raw vectors could be:
 $A = [2, 0]$
 $B = [6, 0]$
- L2 normalization does:
 $A_{\text{normalized}} = A / |A| = [1, 0] \ (|A| = 2)$
 $B_{\text{normalized}} = B / |B| = [1, 0] \ (|B| = 6)$
Now A and B both face the same direction $[1, 0]$ and length 1.
- This normalization can improve similarity matching between vectors allowing focus on direction rather than magnitude. L2 normalization also sometimes referred to as L2 regularization helps sentence embeddings discover more nuanced relationships by balancing them out.

In summary, L2 forces consistency which improves the quality of semantic search in the vector space.

Step 3: Create Embeddings

Now, it's time to create embeddings for each document in your corpus. Embeddings, as explained before, are a way of representing text data in a form that computers can process more easily. In this case, the `encode()` function from the Sentence Transformers library is used to transform sentences into 384-dimensional vectors:

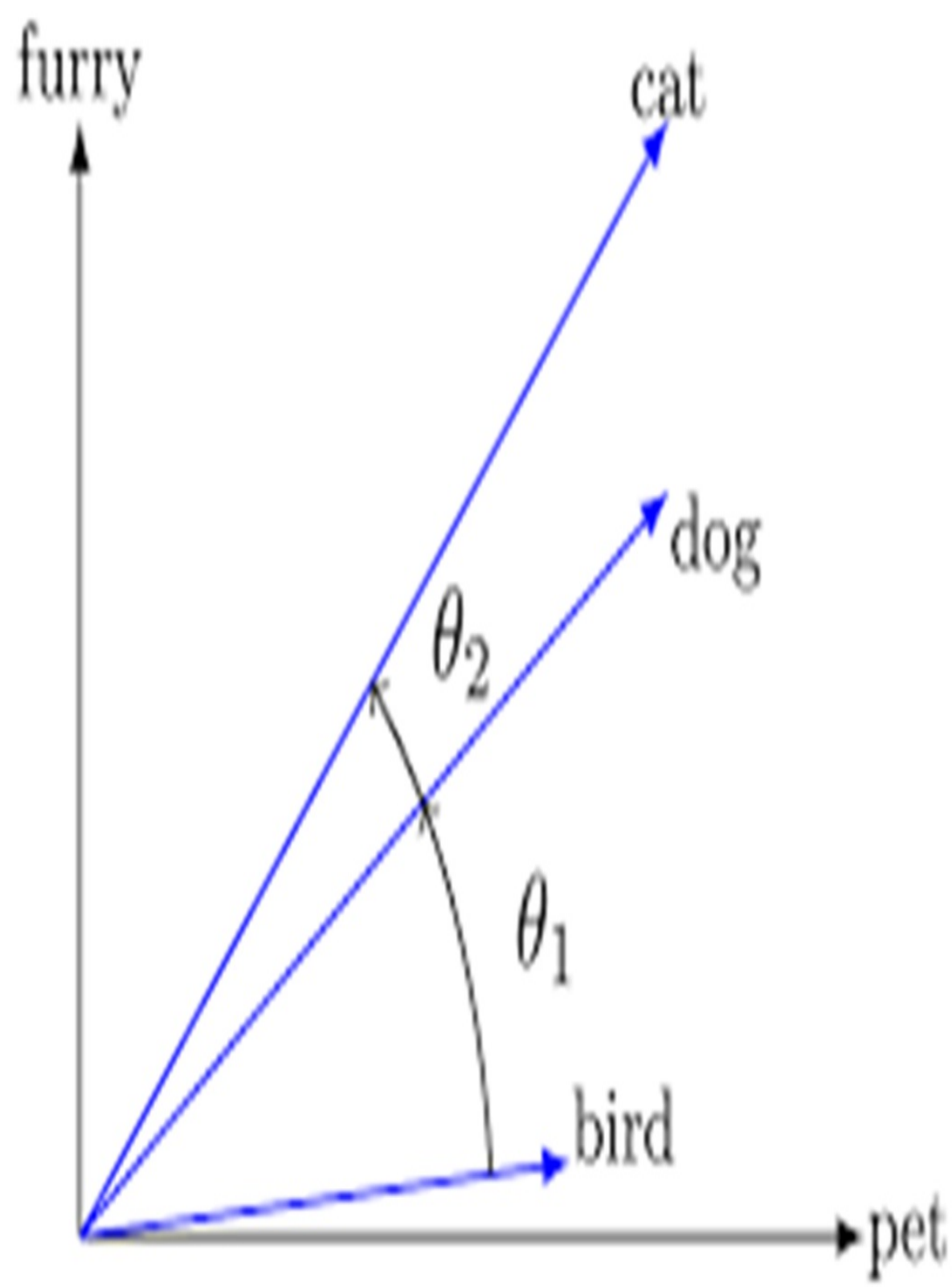
```
embeddings = model.encode(examples)
embeddings.shape
```

(6, 384) #A

Step 4: Semantic Search Function

With your embeddings ready, you can now create a function to perform a semantic search. In keyword search, we used TF-IDF to find the most relevant document to the query. In semantic search, *cosine similarity* is used to compare the query's vector representation (or embedding) with the embeddings of all the documents in the database. Simplified, the cosine similarity represents how two vectors (arrows) in a space of 384 dimensions follow a similar direction. Our capital–country vectors all were similar, therefore, having a high cosine similarity. Likewise, the documents with the highest cosine similarity values are then returned as the most relevant results.

Figure 3.4 A graphical demonstration of cosine similarity using two dimensions ("pet" and "furry"). Three words are drawn as vectors into the coordinate system. θ_1 denotes the angle between bird and dog, θ_2 depicts the angle between dog and cat, respectively. The smaller angle θ_2 illustrates a higher similarity of dogs with cats compared to birds.'



This function will take a query as input and return the most similar documents, using the using *cosine-similarity*:

```
def semantic_search(query, embeddings, examples):  
    query_embedding = model.encode(query) #A  
  
    scores = [scipy.spatial.distance.cosine(query_embedding, doc)  
              for i, doc in enumerate(examples): #C  
                print(f"Example {i}: {examples[i]}")  
                print(f"Score: {1 - scores[i]:.4f}")  
  
    print("Most similar example:", examples[scores.index(min(score
```

In the above function, we first create an embedding for the query. Then, we calculate the cosine distance between the query embedding and the embeddings of all the documents in our dataset. The documents with the smallest cosine distances are the ones most similar to our query.

Similarity Measures

Cosine similarity is a measure of similarity between two non-zero vectors of an inner product space that measures the cosine of the angle between them. Its values range from -1 to 1, where -1 means exactly opposite, 1 means the same, and 0 indicates orthogonality or decorrelation. In the context of semantic search, cosine similarity is used to compare the similarity between the query and document embeddings, with higher cosine values indicating greater similarity.

Apart from cosine similarity, there are several other similarity metrics used in semantic search. These are three famous distance calculations used in many computational use cases:

- **Euclidean Distance:** This metric calculates the straight-line distance between two points in a space. However, it may not be ideal for high-dimensional spaces that are common in NLP.
- **Manhattan Distance:** Also known as city block distance, it computes

the absolute difference between the coordinates of pairs of objects. Like Euclidean distance, it can be less effective in high-dimensional spaces.

- **Jaccard Similarity:** This measures the similarity between finite sample sets and is defined as the size of the intersection divided by the size of the union of the sample sets.

Step 5: Run a Search Query

Finally, you can run a search query using the semantic search function:

```
semantic_search("Machine learning", embeddings, examples)
```

We finally have our output, similar to the keyword search

```
Example 0: The cat is playing in the garden
Score: -0.0488
Example 1: A dog and cat are good pets
Score: 0.0513
Example 2: Cats love to chase mice
Score: 0.0171
Example 3: Machine learning is based on algorithms
Score: 0.6723
Example 4: Deep learning uses neural networks
Score: 0.4380
Example 5: Recurrent networks have connections
Score: 0.3609
Most similar document: Machine learning is based on algorithms
```

As a recall, let's compare the result with the keyword search for the same query we got previously:

```
Example 3: Machine learning is based on algorithms
Score: 1.791759469228055
Example 4: Deep learning uses neural networks
Score: 0.6931471805599453
Example 0: The cat is playing in the garden
Score: 0
Example 1: A dog and cat are good pets
Score: 0
Example 2: Cats love to chase mice
Score: 0
Example 5: Recurrent networks have connections
Score: 0
```

Most similar document: Machine learning is based on algorithms

We see that with the keyword search, only Examples 3 and 4 have non-zero relevance scores. This is because they are the only documents containing the exact words from the phrase "Machine learning" from the query. The other documents are considered irrelevant as there is no lexical overlap.

In contrast, the semantic search engine assigns graded relevance scores to all documents. Interestingly, Example 3 with the phrase match does still come out with the highest semantic score of 0.67. But Example 4 is now ranked second with a score of 0.43 even without an exact keyword match.

This indicates the semantic search model was able to infer conceptual relevance between "deep learning" and the query phrase using its learned language understanding. Examples 0 and 1 with no technical overlap get the lowest scores as expected.

While both methods ranked Example 3 at the top, the semantic search achieved a more nuanced relevance ranking. By utilizing deep language models like Sentence Transformers, semantic search systems can go beyond surface form keyword matching to better model meaning. This allows discovering relevant content even without word-for-word query matches. This approach of capturing and calculating semantic meaning is used in Transformers at large scale and makes them superior to previous exact-match keyword systems.

Semantic search achieves higher accuracy and relevance than traditional keyword-based methods by incorporating aspects of language comprehension. This makes such systems incredibly useful for search applications.

In the next chapter, we will explore a more advanced version of keyword search, called BM 25, and will try to combine semantic search with keyword search to explore a new approach called Hybrid Search. We will also explore some advanced topics on similarity search measures.

3.5 Summary

- An Information Retrieval (IR) system is a software system that stores and manages information (documents, web pages, articles, etc.) and provides interfaces for users to find and access the information they need.
- A very popular example of IR is Keyword search using inverted indices and Term Frequency Inverse Document Frequency (TF-IDF) scoring. It provides a scalable baseline for search relevancy. However, keyword matching is mostly based on word frequency and has limited semantic understanding beyond the exact phrases mentioned in the query.
- Recent advances in deep learning have enabled a more sophisticated form of search, called semantic search. The foundation of semantic search lies in the semantics, or meaning, of the words and phrases used in the search query.
- Using a pre-trained model framework like Sentence Transformers, we encode language into vector representations capturing meaning.
- Similarity metrics such as cosine similarity is a crucial metric in semantic search that measures the angle between two vectors - in this case, the vectors representing the query and the document. It effectively measures how closely related two pieces of text are, regardless of their size.
- Semantic search performs substantially better than keyword search in retrieval as it tries to achieve the context of the query and can go beyond surface form keyword matching to better model meaning. This allows discovering relevant content even without word-for-word query matches.

4 Semantic Search from Scratch

This chapter covers

- Building a hotel search engine from scratch: *Travelle*
- The hotel data and reviews used to build the search engine
- Converting data into Embeddings using Sentence Transformers
- Using distance metrics to rank results using Bi-Encoders
- Introduction to FAISS and the World of Vector Databases

In Chapter 3, we introduced the concept of semantic search and highlighted its advantages over traditional keyword-based search. While keyword search focuses on matching specific words, semantic search goes beyond by understanding the meaning and context of the text, enabling more accurate and relevant results.

We found when a user searches for "furry animal" using a keyword search, the results would only include documents that explicitly contain the words "furry" and "animal." However, many relevant documents that mention specific furry animals like "dogs," "cats," or "rabbits" might be missed. In contrast, semantic search understands that "dog" is a furry animal and would include documents mentioning "dog" in the search results, even if the words "furry" and "animal" are not explicitly present.

In this chapter, our goal is to build upon our previous learning and dive into the practical application of Semantic Search: Build A Hotel Search Engine from scratch (like Expedia), which we will call *Travelle*, built by me (you can see this engine in action here: <https://travelle.traversaal.ai>).

As we mentioned in Chapter 2, Travelle is quite unlike conventional hotel search engines, it allows a more dynamic and user-friendly search experience. Here, users can express their specific hotel preferences and needs through free-form text queries. These queries can range from general descriptions like "quiet hotels with ocean views and a spa that offers couples massages" to more precise requests like "a hotel that has a room with a view of the Eiffel

Tower". This flexibility and intuitiveness in search functionality fundamentally alter how users interact with hotel booking platforms.

Figure 4.1 The landing page of the actual website Travelle; currently it provides recommendations for hotels in Paris. You can visit this site at <https://travelle.traversaal.ai>

travelle

A new way to search for hotels!

Be as descriptive as possible and we will try to find the best possible match for you in Paris.

Q A hotel which has a room with a view of the Eiffel Tower

Search



We will begin by discussing the first step, the data used for this semantic search. We will then discuss the Encoders model used to convert this data into Embeddings, forming Semantic Search's foundation. Here we will also discuss the importance of finding the right Encoder Model and using an

innovative library, Sentence Transformers. Next, we will introduce distance metrics, which are crucial in determining the similarity between embeddings. We will explore common distance metrics such as cosine distance and Euclidean distance and understand how they measure the proximity or dissimilarity between vectors in a high-dimensional space. Following distance metrics, we will delve into two popular architectures for semantic similarity: bi-encoders and cross-encoders. Bi-encoders consist of two separate encoder models, one for encoding the query and another for encoding the documents, allowing for efficient similarity search. Cross-encoders, on the other hand, take both the query and document as input and consider their interaction during the encoding process, capturing fine-grained relationships but requiring more computational resources.

After exploring distance metrics and encoding architectures, we will introduce the Facebook AI Similarity Search (FAISS), a popular library for efficient similarity search and clustering of dense vectors and go deeper into Vector Databases. FAISS provides various indexing techniques and algorithms to optimize the search process, enabling fast retrieval of the most similar documents even in large-scale datasets. Each of these steps, from data preparation to embedding generation, distance metric selection, encoding architectures, and re-ranking models, will play a major role in building Travelle from scratch.

What about langchain, llamaindex or other frameworks?

It's worth mentioning that there are existing frameworks like Langchain and LlamaIndex that provide high-level abstractions for implementing semantic search and other language model-based applications. These frameworks offer convenient tools and pre-built components that can simplify the development process.

However, the focus of this chapter is to teach you how to build semantic search systems from the ground up. By understanding the underlying principles and implementing the components yourself, you'll gain a deeper insight into how these systems work. This approach will enable you to:

- Develop a thorough understanding of each component in the semantic search pipeline.

- Customize and fine-tune your system to meet specific requirements.
- Troubleshoot and optimize your implementations more effectively.
- Create novel solutions by combining or modifying existing techniques.
- Adapt to new developments in the field more easily.

While frameworks can be incredibly useful in production environments, building from scratch ensures you have the knowledge and skills to create, modify, and extend semantic search systems as needed. We will also cover “chunking” strategies in the upcoming chapters.

By understanding and applying these concepts, we aim to create a powerful and efficient semantic search system tailored to the needs of the travel domain. These respective tools will be applied to building Travelle: (1) We collect and embed Paris’ hotel reviews and will store the embedded data (2) in the FAISS vector database. Then, whenever a user makes a hotel search request, we embed that query (3) and match its embeddings with the closest reviews of Paris hotels, to display those to the user.

Figure 4.2 An Overview of the various steps involved in creating Semantic Search from scratch for Hotel Finder in Paris: Travelle

1.Collate Data for all the relevant hotels from various cities

2.Generate Embeddings for each review from each hotel

3.Generate Embedding of Query

4.Apply Similarity Metrics for Retrieval of the most relevant embeddings/ documents to the query

4.1 Loading the data for semantic search

To begin building our search engine, the initial step is to gather and organize the data that will be utilized. In this particular use case, we have sourced hotel reviews and descriptions from TripAdvisor, a popular travel website that permits the scraping of such information.

A note: We will put all of the code in this chapter together in Subchapter 4.5, so you don't need to code along but you are welcome to do so. Also, please use Jupyter Notebook application to run this code

The dataset encompasses details from hotels across five cities, with each city featuring a varying number of hotels and corresponding reviews. However, for the purpose of our current exploration, we will focus solely on hotels located in Paris. As we progress through the subsequent chapters and develop more advanced retrieval systems, we will gradually expand our scope to include additional cities from the dataset. It would also be possible to create semantic search for a wide range of other use cases such as corporate documents, restaurant reviews or medical reports. Oftentimes, the complexity is in preparing the right data at a high quality, but fortunately this has been taken care of in this scenario.

In the first section of our code, we will walk through the process of loading a hotel dataset using the Hugging Face Datasets library and filtering it to focus on hotels located in Paris. We will use the `pandas` library to manipulate and analyze the data.

Step 1: Install the Hugging Face Datasets Library

First, let's install the Hugging Face Datasets library, which provides easy access to a wide range of datasets. Open your terminal or command prompt and run the following command:

```
pip install huggingface datasets
```

if this installation is too noisy you can append the flag `-q` to make it more quiet

Step 2: Import the Required Libraries

Next, we need to import the necessary libraries. We will use `pandas` for data manipulation and the `load\_dataset` function from the Hugging Face Datasets library to load the hotel dataset.

```
import pandas as pd
from datasets import load_dataset
```

Step 3: Load the Hotel Dataset

Now, let's load the hotel dataset using the `load\_dataset` function. We will use the "traversaal-ai-hackathon/hotel_datasets" dataset, which contains hotel reviews and descriptions.

```
dataset = load_dataset("traversaal-ai-hackathon/hotel_datasets")
```

Step 4: Convert the Dataset to a Pandas DataFrame

To work with the dataset using `pandas`, we need to convert it to a DataFrame. We will extract the 'train' split of the dataset and create a DataFrame named `df`, short for dataframe.

```
df = pd.DataFrame(dataset['train'])
```

Step 5: Explore the DataFrame

Let's take a look at the first few rows of the DataFrame using the `head()` function to get an idea of the data structure. This will display the first five rows of the DataFrame, showing the columns and their corresponding values.

```
df.head() #A
```

The data should look something like this:

1 df.head()

[6] ✓ 0.0s

	hotel_name	hotel_description	review_title	review_text	rate	tripdate	hotel_url	hotel_ir
0	Romance Istanbul Hotel	Romance Istanbul Hotel has 39 rooms.Every room...	An exceptional boutique hotel, great value for...	None	NaN	February 2020	https://www.tripadvisor.com/Hotel_Review-g2939... cdn.tripadvisor.com/media/pho	https://m
1	Romance Istanbul Hotel	Romance Istanbul Hotel has 39 rooms.Every room...	You can't get better than this.	None	NaN	March 2021	https://www.tripadvisor.com/Hotel_Review-g2939... cdn.tripadvisor.com/media/pho	https://m
2	Romance Istanbul Hotel	Romance Istanbul Hotel has 39 rooms.Every room...	Exceeds all expectations	None	NaN	March 2021	https://www.tripadvisor.com/Hotel_Review-g2939... cdn.tripadvisor.com/media/pho	https://m
3	Romance Istanbul Hotel	Romance Istanbul Hotel has 39 rooms.Every room...	Great Location, Fantastic Accommodations	None	NaN	August 2021	https://www.tripadvisor.com/Hotel_Review-g2939... cdn.tripadvisor.com/media/pho	https://m
4	Romance Istanbul Hotel	Romance Istanbul Hotel has 39 rooms.Every room...	Perfection. It is all in the details.	None	NaN	June 2021	https://www.tripadvisor.com/Hotel_Review-g2939... cdn.tripadvisor.com/media/pho	https://m

Step 6: Filter the DataFrame by Locality

Since we want to focus on hotels in Paris, we will filter the DataFrame based on the 'locality' column (not visible in screenshot). We can use the `loc[]` accessor to create a new DataFrame named `df_paris` that contains only the rows where the 'locality' is 'Paris'.

```
df_paris = df.loc[df.locality == 'Paris']
```

Step 7: Explore the Filtered DataFrame

Let's again use the `head()` function to view the first few rows of the filtered DataFrame `df_paris`.

```
df_paris.head() #A
```

Optionally, you can also use `df_paris.shape` which will return a tuple of (1200, 14) indicating that we have 1200 review entries for our hotels in Paris and 14 different columns of information ('hotel_name', 'hotel_description', 'review_title', 'review_text', 'rate', 'tripdate', 'hotel_url', 'hotel_image', 'price_range', 'rating_value', 'review_count', 'street_address', 'locality', 'country') on it.

Step 8: Analyze the Hotel Names

To get a count of the unique hotel names in the filtered DataFrame, we can use the `value_counts()` function on the 'hotel_name' column.

```
df_paris.hotel_name.value_counts() #A
```

The output from our last code indicates that we have 30 unique hotels in Paris, each with 40 reviews. While we know that Paris has more than 30 hotels, this dataset provides a sufficient starting point for building our search engine. Since each review offers a unique perspective on the hotel, we will generate embeddings for each review of a given hotel. Essentially, we have 1,200 distinct reviews in this dataset, which we will treat as separate documents. Although the dataset also includes hotel descriptions and other details for each hotel, we will not be utilizing them at this stage. Our current focus will be solely on the reviews. This lays the groundwork for developing a search engine specifically tailored to hotels in Paris.

Building upon this foundation, the next section will delve into various approaches for converting our hotel reviews into embeddings. Embeddings are crucial for capturing the semantic meaning and context of the reviews, enabling us to perform efficient and accurate similarity searches. By representing each review as a dense vector embedding, we can effectively compare and retrieve relevant reviews based on their semantic similarity.

4.2 Generate Embeddings from Hotel Reviews

Now that we have our data, the next step for Semantic Search is to convert text into Embeddings. Embeddings are crucial for semantic search as they enable the representation of text data in a dense, continuous vector space, capturing the semantic relationships between words, sentences, and documents. This allows us to perform similarity comparisons and retrieve relevant information based on semantic meaning rather than exact keyword matches. Our first task is selecting an appropriate pre-trained transformer encoder model. However, these models primarily work with word or token-level embeddings rather than sentence-level embeddings, which may not be the most effective approach for building sentence vectors and calculating accurate sentence similarity. (see explanation below)

Sentence Transformers (we covered SBERTs in Chapter 2) address these limitations by providing a more efficient and flexible approach to generating sentence embeddings. Instead of using a cross-encoder structure, which can be computationally expensive and does not generate standalone sentence embeddings, Sentence Transformers fine-tunes the transformer models on sentence-level tasks, allowing them to capture the overall meaning and semantics of sentences directly. This results in a single, fixed-size embedding vector for each sentence, which can be easily stored, indexed, and compared for various downstream tasks. Sentence Transformers has become a go-to library for generating embeddings due to its comprehensive ecosystem, wide range of pre-trained models, and focus on sentence-level embeddings, making it particularly valuable for tasks like semantic search. For even better results models with a higher dimensionality – but also a fee – can be used such as [OpenAI](#)'s text-embedding-3-small. Using SBERT is sufficient for our use case.

Why do we prefer to use the Sentence Transformer Library

One of the major advantages of Sentence Transformers is its ability to perform both word-level and sentence-level tokenization. Word-level tokenization involves breaking down a text into individual words or subwords, which is the standard approach used by many transformer-based models like BERT.

For example, the sentence "The quick brown fox jumps over the lazy dog" would be tokenized into ["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog"]. On the other hand, sentence-level tokenization treats the entire sentence as a single unit, allowing the model to capture the overall meaning and context of the sentence.

Sentence Transformers leverages this capability to generate fixed-size embeddings for entire sentences, regardless of their length. For instance, the sentences "The movie was fantastic!" and "I absolutely loved the film and would highly recommend it to anyone." would each be represented by a single, dense vector that encapsulates their semantic meaning. This sentence-level tokenization enables efficient similarity comparisons and retrieval of semantically related sentences without the need for complex cross-encoder architectures.

The library's focus on sentence-level embeddings makes it particularly valuable for tasks like semantic search, where the goal is to find semantically similar documents based on their meaning rather than exact keyword matches. By providing high-quality sentence embeddings out-of-the-box, Sentence Transformers simplifies the process of building semantic search systems and other applications that rely on accurate text representations. Moreover, the active development and community support behind Sentence Transformers ensures that the library stays up-to-date with the latest advancements in transformer-based models.

Let's explore the code for creating embeddings.

Generate Embeddings using Sentence Transformers:

Sentence Transformers makes it straightforward to generate embeddings for your text data. The process typically involves the following steps:

- **Installation:** Install the Sentence Transformers library using pip:

```
pip install sentence-transformers
```

- **Model Selection:** Choose a pre-trained Sentence Transformers model that suits your needs. Models are available for various tasks and

languages.

- **Encoding:** Pass your input sentences or paragraphs to the selected model's `encode` function, which handles tokenization, encoding, and pooling internally. The function returns the generated embeddings as a numpy array.

Here's a simple example of generating embeddings using Sentence Transformers:

Listing 4.1 Generate Embeddings on sample data

```
from sentence_transformers import SentenceTransformer #A
model = SentenceTransformer("all-MiniLM-L6-v2") #B
sentences = ["This is an example sentence.", "Another example sen
embeddings = model.encode(sentences) #C
print(len(embeddings)) #D
```

The resulting `embeddings` variable contains the generated embeddings for each input sentence. These embeddings can be stored and used for various downstream tasks, including semantic search. By representing text as dense vectors, we can capture the semantic meaning and perform similarity comparisons for effective information retrieval. While we have covered Sentence Transformers in detail, the question remains: Which encoder model for Sentence Embeddings should we use?

4.2.1 Selecting the right Encoder Model

The landscape of encoder models for generating sentence embeddings is vast, there are 100s if not thousands of models available, each with unique characteristics and capabilities. When selecting the most suitable model, the first decision often revolves around the choice between open-source and closed-source models. Proprietary models, such as OpenAI's embedding v3, boast impressive performance metrics and extensive multilingual support but present limitations due to their closed-source nature. In contrast, the open-source community has been driving the development and dissemination of

cutting-edge embedding models, prioritizing transparency, reproducibility, and collaborative innovation.

Note

The choice between open-source and closed-source models also involves considering the computational costs. Open-source models provide the flexibility to run on local machines or cloud infrastructure, giving users control over the allocated resources. However, the model's performance depends on the user's hardware capabilities. In contrast, closed-source models, often offered as API services, abstract away the computational complexities. These models run on the provider's infrastructure, and users access them through API calls, following a pay-as-you-use model. While this approach eliminates the need for users to manage hardware and allows for scalability, it also means limited control over the model's performance and potential costs based on API usage.

Among the recently published open-source models in 2024, a few notable ones stand out:

- E5-mistral-7b: Developed by Microsoft, initialized from Mistral-7B-v0.1, and fine-tuned on multilingual datasets.
- Multilingual-e5-large-instruct (ML-E5-large): Another E5 model from Microsoft, designed to handle multilingual data effectively.
- BGE-M3: Created by the Beijing Academy of Artificial Intelligence, supporting over 100 languages.
- Nomic-embed-text-v1 (Nomic-Embed): Developed by Nomic, claiming superior performance compared to OpenAI's Ada-002 and text-embedding-3-small while being significantly smaller (0.55GB) and fully reproducible and auditable.

For this chapter, we will use a very popular Encoder Model, all-MiniLM-L6-v2, which can be easily identified in Sentence Transformer.

Step 1: Import and install the necessary libraries

We start by installing/importing three different libraries

- We import SentenceTransformer class from the sentence_transformers library, which provides the functionality for loading and using the Nomic Embedding Model.
- We also import the torch library to check for CUDA availability (CUDA is important for GPU usage which speeds up the progress for generating embeddings)

```
from sentence_transformers import SentenceTransformer
import torch
```

Step 2: Load the Nomic Embedding Model

We create an instance of the `SentenceTransformer` class, specifying the model name `"all-MiniLM-L6-v2"` to load the model:

```
model = SentenceTransformer("all-MiniLM-L6-v2") #A
```

Note

You will be prompted that running above code will execute remote code. It is safe to confirm this with “y” twice.

Step 3: Check if CUDA is available

We use `torch.cuda.is\_available()` to check if CUDA is available on the system. If CUDA is available, we move the model to the GPU using `model.to('cuda')` for faster computation and print a message indicating that the model has been moved to the GPU. If CUDA is not available, we print a message indicating that the model will run on the CPU.

```
if torch.cuda.is_available():
    model = model.to('cuda')
    print("CUDA is available. The model has been moved to GPU.")
else:
    print("CUDA is not available. The model will run on CPU.")
```

Step 4: Extract the review text from the DataFrame

Using our DataFrame from previous section `df_paris`, we extract the review text from the DataFrame using `df_paris['review_text'].tolist()`. This converts the review text column into a list of strings, which we store in the `reviews` variable.

```
reviews = df_paris['review_text'].tolist()
```

Step 5: Generate embeddings for the reviews

We use the `encode` method of the loaded model to generate embeddings for the reviews. We pass the `reviews` list to `model.encode()` and set `show_progress_bar=True` to display a progress bar during the encoding process. The generated embeddings are stored in the `review_embeddings` variable.

```
review_embeddings = model.encode(reviews, show_progress_bar=True)
```

Step 6: Print the shape of the embeddings

Printing then gives us information about the number of reviews and the embedding dimension.

```
print(f"Embeddings shape: {review_embeddings.shape}")
```

By following these steps, you can load the MiniLM-v6, check for CUDA (GPU) availability, extract the review text from the DataFrame, generate embeddings for the reviews, and print the shape of the embeddings. You can now use the `embeddings` variable for further analysis, similarity calculations, or any other downstream tasks in your semantic search pipeline.

Embedding is the first crucial step in semantic search, allowing us to represent documents as dense vectors that capture their semantic meaning. Once we have generated embeddings for our documents, the next step is to measure the similarity between these embeddings to determine how semantically related the documents are to a given query, i.e. request like “Hotel with a view of the Eiffel tower”). This is where distance metrics come into play. We will explore more of them in the next section.

4.3 Similarity scores using Cross Encoders and Bi-Encoders

In the previous sections, we covered how to generate embeddings using the Sentence Transformer model. These embeddings serve as dense vector representations of the documents, capturing their semantic meaning. To compare our query to the most relevant documents, we utilize similarity scores or distance metrics.

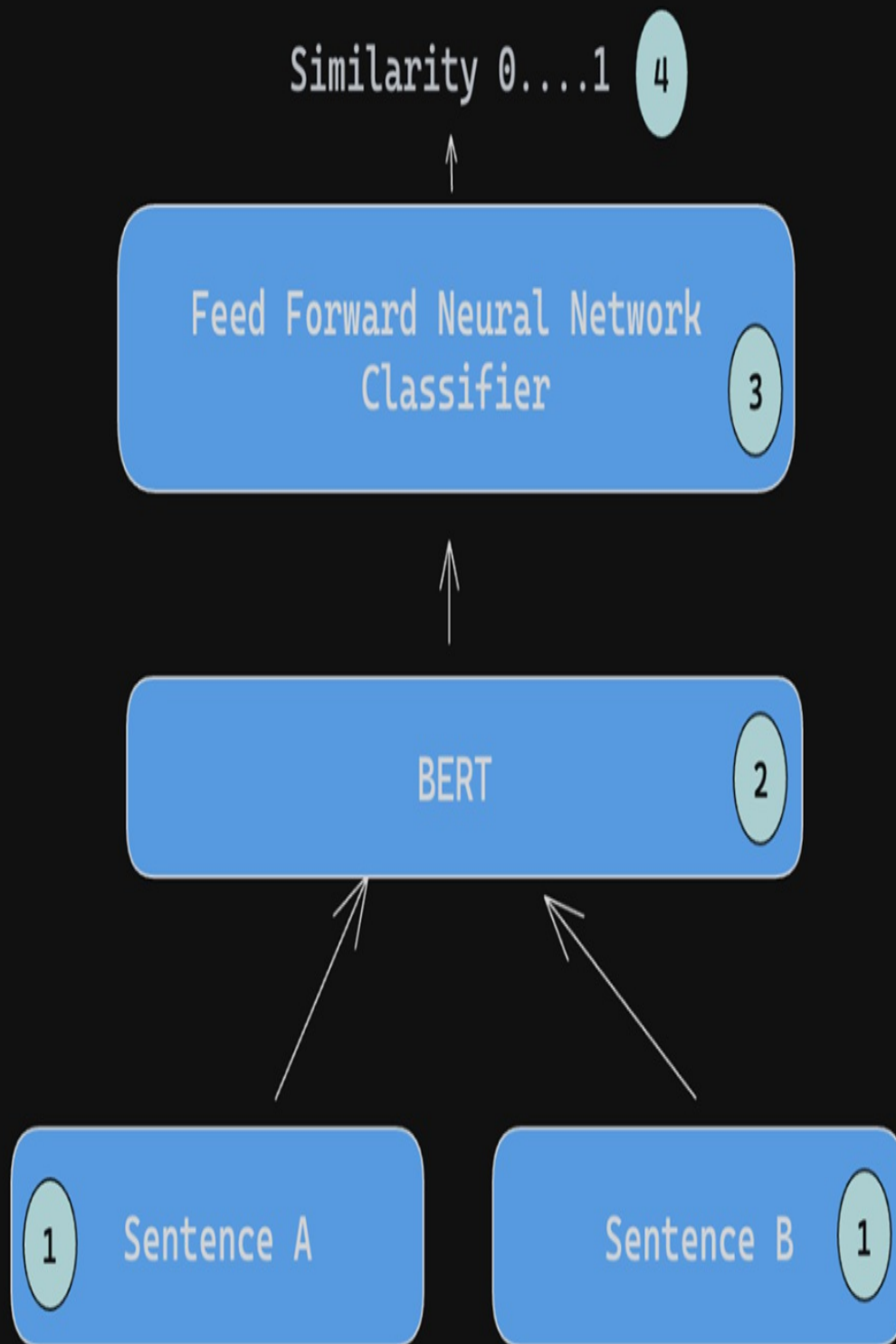
Distance metrics are essential for determining the similarity between embeddings. They measure the proximity or dissimilarity between two vectors in a high-dimensional space. The choice of distance metric can significantly impact the quality and interpretability of the search results. Two commonly used distance metrics are cosine distance and Euclidean distance, which we covered in Chapter 3. Cosine distance measures the cosine of the angle between two vectors, focusing on their orientation rather than magnitude, while Euclidean distance measures the straight-line distance between two points in a multi-dimensional space.

When it comes to semantic similarity, two popular architectures are cross-encoders and bi-encoders. Cross-encoders take two input sentences, concatenate them, and pass them through a transformer model followed by a classification layer. They consider the interaction between the sentences during the encoding process, capturing fine-grained relationships. The common approach to calculating sentence similarity using BERT based models is to employ a cross-encoder structure. In this setup, two sentences are passed to the BERT model, and a classification head is added on top of the BERT architecture. The cross-encoder structure processes both sentences in the same sequence, separated by a special token called [SEP]. The output of the BERT model is then fed into a feedforward neural network classifier, which produces a similarity score indicating how similar the two sentences are. While this is a quite technical description, it is straightforward to use this functionality in practice as you will learn in a bit.

To illustrate this process, the BERT cross-encoder architecture works as follows:

1. Sentence A and Sentence B are concatenated with a [SEP] token in between, forming a single input sequence.
2. The input sequence is passed through the BERT model, which generates contextualized embeddings for each token in the sequence.
3. The output embeddings from BERT are then fed into a feedforward neural network classifier.
4. The classifier produces a similarity score, typically ranging from 0 to 1, indicating the degree of similarity between Sentence A and Sentence B.

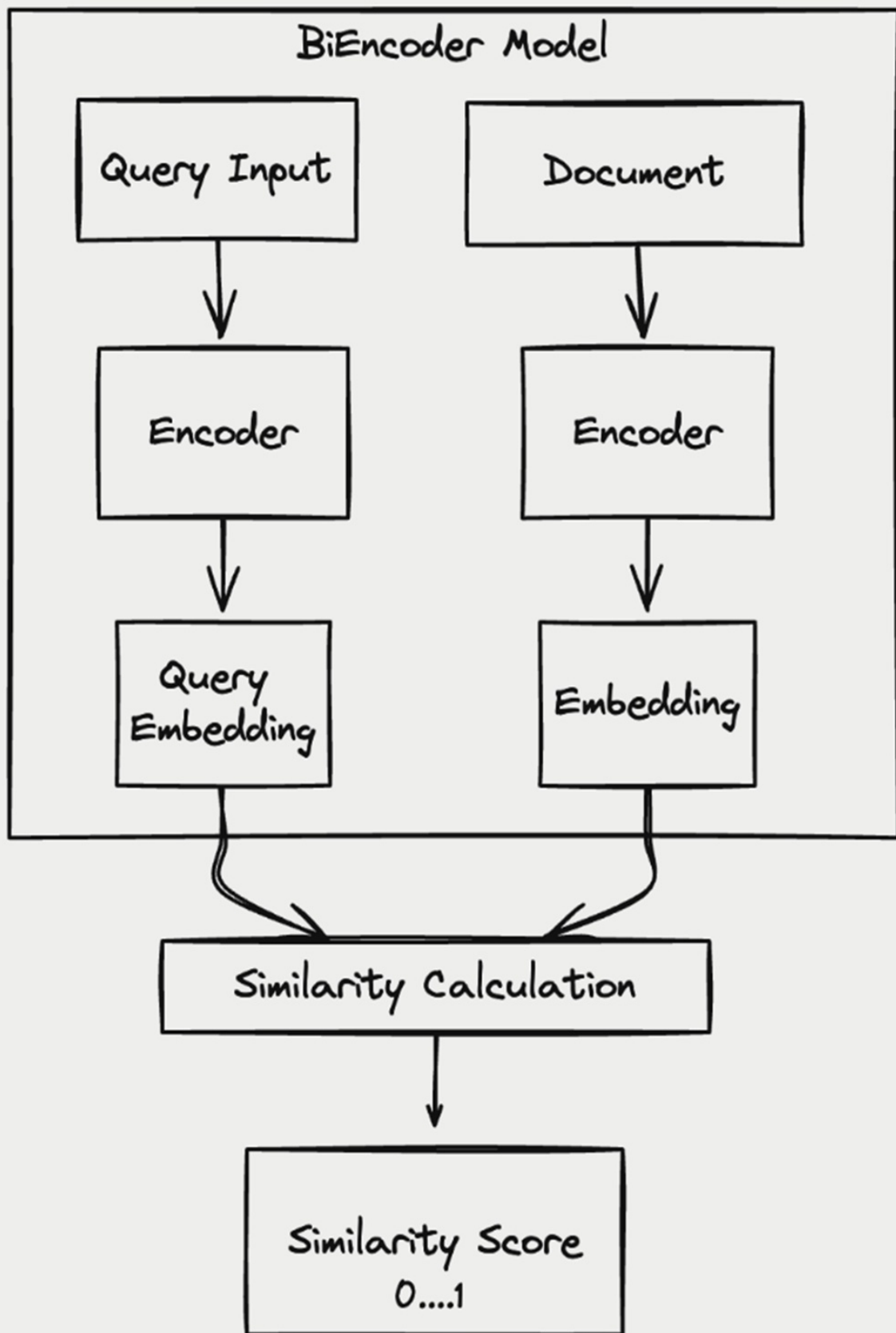
Figure 4.3 An Overview of the Cross-Encoder Model to measure the similarity between two sentences.



However, cross-encoders have a limitation in terms of computational efficiency. They require computing the similarity score for each pair of sentences, which can be computationally expensive, especially when dealing with a large number of documents. This is where bi-encoders come into play.

Bi-encoders consist of two separate encoder models: one for encoding the query sentence and another for encoding the candidate sentences. Each encoder independently maps the input sentence to a fixed-size embedding vector. The key advantage of bi-encoders is that the embeddings for the candidate sentences can be pre-computed and stored. During the similarity search, only the query sentence needs to be encoded, and its embedding can be compared with the pre-computed embeddings of the candidate sentences using a similarity metric like cosine similarity. This is much faster as the pre-computation can be done ahead of time and requires only the query to be encoded.

Figure 4.4 An overview of the Bi-Encoder Model to measure the similarity between two sentences.



By leveraging distance metrics and encoding architectures like cross-encoders and bi-encoders, we can effectively compare the query embedding with the document embeddings and retrieve the most semantically similar documents. The choice between cross-encoders and bi-encoders depends on the specific requirements of the application, such as the desired trade-off between accuracy and efficiency. Bi-encoders are more efficient for similarity search because the embeddings for the candidate sentences can be pre-computed and stored.

Let's implement Bi-Encoder Semantic Search using Cosine Similarity for Review Embeddings

Step 1: Import the necessary library

We import the NumPy library, which provides functions for numerical operations and array manipulation.

```
import numpy as np
import scipy.spatial
```

Step 2: Define the cosine function

We define a function called `cosine\_search` that takes three parameters:

- `query\_embedding`: The embedding of the query text.
- `review\_embeddings`: The embeddings of all the reviews.
- `k`: The number of similar reviews to retrieve.

Inside the function:

- We compute the Cosine distances between the query embedding and all review embeddings
- We get the indices of the top-k smallest distances using `np.argsort(-cosine_similarities)[:k]` This sorts the distances in ascending order and returns the indices of the top-k smallest distances.
- We retrieve the distances of the top-k similar reviews using the obtained indices.

- Finally, we return the top-k indices and their corresponding distances.

```
def cosine_search(query_embedding, review_embeddings, k):
    query_embedding = query_embedding.reshape(-1) #A
    dot_products = np.dot(review_embeddings, query_embedding) #B

    query_norm = np.linalg.norm(query_embedding) #C
    review_norms = np.linalg.norm(review_embeddings, axis=1)

    cosine_similarities = dot_products / (query_norm * review_norms)
    top_indices = np.argsort(-cosine_similarities)[:k] #E
    top_cosine_similarities = cosine_similarities[top_indices] #F

    return top_indices, top_cosine_similarities
```

Step 3: Perform a similarity search for a query

We will now identify a query, encode it and call the `cosine\_search` function with the query embedding, review embeddings, and `k` to obtain the top-k indices and distances.

```
query = " Hotel near the Louvre with great food nearby." #A
query_embedding = model.encode([query]) #B

k = 5 # Number of similar reviews to retrieve #C
indices, distances = cosine_search(query_embedding, review_embeddings)
```

Step 4: Print the top-k similar reviews with distances

We print the Cosine distance between the query embedding and the review embedding using `distance`, formatted to display four decimal places.

```
print(f"Query: {query}") #A
print("Top hotel with similar reviews matching the query:") #B
for i, (idx, distance) in enumerate(zip(indices, distances), 1):
    print(f"{i}. {df_paris.iloc[idx]['hotel_name']}") #D
    print(f"Review: {df_paris.iloc[idx]['review_text']}") #E
    print(f"Distance: {distance:.4f}") #F
    print()
```

Here's a screenshot of the responses:

Figure 4.5 Cosine Similarity Search provides us with a great set of results and hotels related to the query

Query: Hotel near the Louvre with great food nearby.

Top hotel with similar reviews:

1. Grand Hotel du Palais Royal

Review: Great hotel located in the best part of town just next to the Louvre. Walking distance to the Louvre, Norte Dame, and shops. Very clean and the service was wonderful. Even though we didn't speak French everyone at the hotel was very nice and helpful. Almost all of the staff spoke English. The spa is small though which can make things awkward if using the spa amenities.

Distance: 0.7877

2. Hotel Malte - Astotel

Review: Excellent hotel all round. Ideal location for The Louvre, city centre shops, and plenty of metro stations within a few minutes walk to get to other areas of Paris. There is no restaurant for evening meals and lunch and no bar in the hotel but that doesn't matter as there are numerous restaurants and cafes close by so you don't have to go far to eat out. Nothing was too much trouble for the staff and they were all so polite and helpful. The room was very comfortable. The bed was as good as you could wish for and the en-suite bathroom had plenty of room. The room and the hotel in general was spotlessly clean. A nice touch was the mini bar fridge with soft drinks and water in the room at no extra charge, as was the food buffet with hot drinks and soft drinks that was

Distance: 0.7877

3. Hotel Moliere

Review: Travelled with another couple and we stayed for 3 nights and an additional night after visiting the countryside. Highly recommend. Excellent location near Louvre and other key sites. We really enjoyed the helpfulness of the staff and the hospitality there including the delicious breakfast and refreshments offered. Rooms have good amenities but are small and varied. We didn't love the bathroom layout with a glass wall that didn't offer much privacy.

Distance: 0.7869

4. La Maison Favart

Review: Great location - peaceful and very central - opposite the Opera House. Fifteen minutes walk down to the Louvre and many good restaurants close by. Very comfortable beds and pretty breakfast room. Front of House staff couldn't be more helpful especially with directions and great restaurants !

Distance: 0.7856

5. Hotel Square Louvois

Review: What a great hotel. The staff was over the top nice. Every single one of them. Location was great. Room was clean and nice. Went with 3 teen boys and the adjoining room was perfect in every way. Close to metro walking distance to Louvre and dozens of restaurants.

Distance: 0.7829

The query "**Hotel near the Louvre with great food nearby**" indicates that we are looking for a hotel conveniently located near the Louvre Museum in Paris, with excellent dining options in the surrounding area. The cosine similarity search results have provided a fantastic list of hotels that closely match your preferences, ensuring that we have a wonderful stay in the city of lights.

Note

A higher cosine score in the function reflects a higher similarity.

The top results, including Grand Hotel du Palais Royal, Hotel Malte - Astotel, Hotel Moliere, La Maison Favart, and Hotel Square Louvois, all boast prime locations within walking distance to the Louvre and other popular attractions. The reviews consistently praise the helpfulness and friendliness of the staff, the cleanliness of the rooms, and the overall positive experiences of the guests. While some hotels may not have their restaurants or bars, the abundance of dining options in the nearby areas more than makes up for it. You can rest assured that you'll have a comfortable and enjoyable stay at any of these highly recommended hotels.

To efficiently store and search through the embeddings generated from the hotel reviews, we can leverage FAISS (Facebook AI Similarity Search) and vector databases. FAISS is a popular library developed by Facebook AI Research that provides efficient algorithms for similarity search and clustering of dense vectors. It is designed to handle large-scale datasets and enables fast retrieval of the most similar vectors to a given query. Vector databases, on the other hand, are specialized databases optimized for storing and searching high-dimensional vectors like embeddings. By combining FAISS with a vector database, we can build a high-performance semantic search system capable of quickly finding the most relevant hotel reviews based on user queries.

4.4 Introduction to FAISS and Vector Databases

Just as traditional databases are essential for storing and managing structured data, vector databases play a crucial role in handling embeddings efficiently. Imagine you have a vast collection of customer records, each with various attributes like name, age, address, and purchase history. To store and retrieve this structured data effectively, you would typically use a relational database management system (RDBMS) like MySQL or PostgreSQL. These databases provide a structured way to organize the data, define relationships between tables, and perform efficient queries to retrieve specific records based on certain conditions.

However, when it comes to semantic search and working with embeddings, traditional databases fall short. Embeddings are high-dimensional vectors that capture the semantic meaning of documents, and they require specialized

databases optimized for storing and searching these vectors efficiently. This is where vector databases come into play.

Vector databases are designed to handle the unique characteristics of embeddings and provide efficient similarity search capabilities. They leverage indexing techniques like inverted indexes, trees, or hashing to enable fast retrieval of similar vectors. Unlike traditional databases that are built for structured data, vector databases are specifically tailored to work with high-dimensional vectors. We will cover vector database in detail in the next chapter. The fastest standard out there is HNSW, an algorithm, to yield fast [vector lookup](#). More on this later.

FAISS (Facebook AI Similarity Search) is a prominent library developed by Facebook AI Research that addresses the need for efficient similarity search and clustering of dense vectors. It is built to handle large-scale datasets and enables lightning-fast retrieval of the most similar vectors to a given query. FAISS offers a wide range of indexing and search methods, catering to various use cases and performance requirements. Faiss is also in-memory, so it simplifies setup and is ideal for this use case.

Figure 4.6 Updated flow of Semantic Search with FAISS included in the architecture

1.Collate Data for all the relevant hotels from various cities

2.Generate Embeddings for each review from each hotel

3.Index Embeddings in FAISS

4.Generate Embedding of Query

5.Apply Similarity Metrics for Retrieval of the most relevant embeddings/ documents to the query

The combination of FAISS and vector databases provides a powerful solution

for storing and searching embeddings at scale. FAISS can be used as a backend for vector databases, allowing them to efficiently index and search through large collections of embeddings. By integrating FAISS with a vector database, we can build high-performance semantic search systems that can handle millions or even billions of documents.

One of the key advantages of FAISS is its ability to perform both exact search and approximate nearest neighbor search. Exact search guarantees finding the most similar vectors to a query, but it can be computationally expensive, especially for large datasets. On the other hand, approximate nearest neighbor search trades a small amount of accuracy for significant speed improvements. It allows you to quickly retrieve highly similar vectors while maintaining a good balance between search quality and efficiency.

Just as you would choose an appropriate database system based on your structured data requirements, selecting the right vector database and indexing method depends on your specific needs. FAISS provides a flexible and scalable solution for storing and searching embeddings, making it a go-to choice for many applications in the realm of semantic search.

By leveraging the power of vector databases and FAISS, we can efficiently store and search through vast collections of embeddings, enabling high-performance semantic search systems that can handle the ever-growing volume of data in today's world.

Let's Index our hotel dataset in FAISS. This step-by-step explanation breaks down the process of using FAISS and vector databases to index and search embeddings created from the hotel dataset. Each step is accompanied by the relevant code snippet and a detailed explanation of what is happening in that step.

We begin with the `review_embeddings` variable from our previous section.

Step 0: Install FAISS Vectorstore Dependency

Let's get start by installing the FAISS dependency:

```
pip install faiss-cpu
```

Step 1: Convert the embeddings to float32

To work with FAISS, we need to convert the embeddings to float32 format. We use the `astype('float32')` function to convert the `review_embeddings` to float32.

```
review_embeddings = review_embeddings.astype('float32')
```

Step 2: Initialize the FAISS index

We Initialize the FAISS index with cosine similarity and specify the dimension of the embeddings as `review_embeddings.shape[1]`, which represents the length of each embedding vector.

```
import faiss #A

review_embeddings_normalized = review_embeddings / np.linalg.norm

index = faiss.IndexFlatIP(review_embeddings_normalized.shape[1])
```

We initialize a FAISS index using `faiss.IndexFlatIP`, (IP: Inner Product) which uses Cosine Distance as the similarity metric. However, it's important to note that the choice of similarity metric can have a significant impact on the search results. In Chapter 3, we covered two commonly used distance metrics: cosine distance and Euclidean distance. Cosine distance measures the cosine of the angle between two vectors, focusing on their orientation rather than magnitude. On the other hand, Euclidean distance, which is used by `IndexFlatL2`, measures the straight-line distance between two points in a multi-dimensional space. It takes into account both the orientation and magnitude of the vectors.

Step 3: Add the embeddings to the FAISS index

Here, we add the `review_embeddings_normalized` to the FAISS index using the `add()` method. This step indexes the embeddings and prepares them for efficient similarity search.

```
index.add(review_embeddings_normalized)
```

FAISS supports efficient similarity search and clustering of dense vectors. It provides various indexing strategies and distance metrics, making it compatible with both cross-encoders and bi-encoders.

Let's explore how we can use Cosine distance in FAISS for similarity search. Flat indexes are the simplest indexing strategy in FAISS. They store the embeddings as-is, without any compression or quantization. Flat indexes perform an exhaustive search over all the embeddings to find the nearest neighbors (bi-encoder). They are suitable for small to medium-sized datasets and provide exact search results. We pick up from the index variable we stored in the last section, which has embeddings stored in FAISS

Step 1: Encode the query and setting a threshold for reviews matching the query

We will embed the query to compare it with the embeddings and convert it to float32

```
query = " Hotel near the Louvre with great food nearby." #A
query_embedding = model.encode([query]).astype('float32') #B
query_embedding_normalized = query_embedding / np.linalg.norm(query_embedding) #C
k = 5 #D
```

Step 2: Perform Similarity Search

Use the `search()` method of the FAISS index to find the top-k similar reviews. We specify the `query_embedding` and the number of similar reviews to retrieve (`k`). In this example, we set `k = 5` to retrieve the top 5 similar reviews. The `search()` method returns the distances and indices of the top-k similar reviews.

```
distances, indices = index.search(query_embedding, k)
```

Step 3: Print the top-k similar reviews

The final step is to print the reviews closely related to the reviews

```
print(f"Query: {query}")
```

```

print("Top hotel with similar reviews using FAISS:")
for i, (idx, distance) in enumerate(zip(indices[0], distances[0]))
    print(f"{i}. {df_paris.iloc[idx]['hotel_name']}")
    print(f"Review: {df_paris.iloc[idx]['review_text']}")
    print(f"Distance: {distance:.4f}")
    print()

```

Finally, we print the query and the top-k similar reviews retrieved from the hotel dataset. We use the indices obtained from the search to access the corresponding reviews in the `df\_paris` DataFrame using `df\_paris.iloc[idx]['review\_text']`. We enumerate the indices starting from 1 to display the rank of each similar review.

Figure 4.7 Cosine Similarity Search using FAISS provides us with the same answer as generated using the cosine similarity with bi-encoders

Query: Hotel near the Louvre with great food nearby.

Top hotel with similar reviews using FAISS:

1. Grand Hotel du Palais Royal

Review: Great hotel located in the best part of town just next to the Louvre. Walking distance to the Louvre, Norte Dame, and shops. Very clean and the service was wonderful. Even though we didn't speak French everyone at the hotel was very nice and helpful. Almost all of the staff spoke English. The spa is small though which can make things awkward if using the spa amenities.

Distance: 0.7877

2. Hotel Malte - Astotel

Review: Excellent hotel all round. Ideal location for The Louvre, city centre shops, and plenty of metro stations within a few minutes walk to get to other areas of Paris. There is no restaurant for evening meals and lunch and no bar in the hotel but that doesn't matter as there are numerous restaurants and cafes close by so you don't have to go far to eat out. Nothing was too much trouble for the staff and they were all so polite and helpful. The room was very comfortable. The bed was as good as you could wish for and the en-suite bathroom had plenty of room. The room and the hotel in general was spotlessly clean. A nice touch was the mini bar fridge with soft drinks and water in the room at no extra charge, as was the food buffet with hot drinks and soft drinks that was

Distance: 0.7877

3. Hotel Moliere

Review: Travelled with another couple and we stayed for 3 nights and an additional night after visiting the countryside. Highly recommend. Excellent location near Louvre and other key sites. We really enjoyed the helpfulness of the staff and the hospitality there including the delicious breakfast and refreshments offered. Rooms have good amenities but are small and varied. We didn't love the bathroom layout with a glass wall that didn't offer much privacy.

Distance: 0.7869

4. La Maison Favart

Review: Great location - peaceful and very central - opposite the Opera House. Fifteen minutes walk down to the Louvre and many good restaurants close by. Very comfortable beds and pretty breakfast room. Front of House staff couldn't be more helpful especially with directions and great restaurants !

Distance: 0.7856

5. Hotel Square Louvois

Review: What a great hotel. The staff was over the top nice. Every single one of them. Location was great. Room was clean and nice. Went with 3 teen boys and the adjoining room was perfect in every way. Close to metro walking distance to Louvre and dozens of restaurants.

Distance: 0.7829

At this point, our main question would be “Why would I even use FAISS?” The answer lies in efficiency and scalability. As the number of documents in a dataset grows, the computational cost of performing similarity search using bi-encoders becomes increasingly expensive. With bi-encoders, the query embedding needs to be compared against the embedding of each document in the dataset. This process involves computing the similarity score (e.g., cosine similarity) between the query embedding and every document embedding, which requires a significant amount of computational resources, especially for large-scale datasets. As the dataset size increases, the time and memory required for this exhaustive search grows linearly, making it impractical for real-time applications.

This is where FAISS comes into play as a standard practice for efficient and scalable similarity search. FAISS is designed to handle massive datasets with millions or even billions of embeddings, making it an essential tool for real-world applications. By utilizing optimized algorithms and data structures, FAISS enables lightning-fast similarity search, even on large datasets. This is crucial for providing a seamless user experience and delivering relevant results in real-time.

Moreover, FAISS is highly memory-efficient, allowing you to store and search through vast amounts of embeddings without consuming excessive resources. It supports distributed search across multiple machines, enabling you to handle even larger datasets and higher query volumes. This scalability is particularly important as your semantic search system grows and needs to accommodate an increasing number of documents.

Another significant advantage of FAISS is its support for advanced indexing techniques. While we focused on flat indexes and cosine similarity in our previous examples, FAISS offers a wide range of indexing methods that can significantly boost the performance and efficiency of similarity search. These techniques include inverted file systems (IVF), product quantization (PQ), and hierarchical navigable small world graphs (HNSW), among others.

By leveraging these advanced indexing techniques, you can achieve faster search times, reduced memory usage, and improved accuracy compared to using cosine similarity alone. FAISS allows you to strike a balance between search quality and computational efficiency, making it suitable for various use cases and dataset sizes.

In the upcoming chapters, we will explore these advanced indexing techniques in more detail and demonstrate how they can be applied to enhance the performance of your semantic search system. By combining the power of FAISS with the effectiveness of bi-encoders and cosine similarity, you can build a robust and efficient search engine that delivers highly relevant results to your users.

4.5 Putting it all together: Travelle in action

Now that we have gone through all the steps in building the code from data to embeddings to indexing using FAISS, using distance metrics to rank the results, let's put the entire code together:

Listing 4.2 Travelle code from start to finish

```
import pandas as pd
from datasets import load_dataset
import numpy as np
from sentence_transformers import SentenceTransformer
import torch
import scipy.spatial
import faiss
import time

dataset = load_dataset("traversaal-ai-hackathon/hotel_datasets")

df=pd.DataFrame(dataset['train'])
df_paris = df.loc[df.locality=='Paris']

model = SentenceTransformer("all-MiniLM-L6-v2")

if torch.cuda.is_available():
    model = model.to('cuda')
    print("CUDA is available. The model has been moved to GPU.")
else:
    print("CUDA is not available. The model will run on CPU.")

reviews = df_paris['review_text'].tolist()

query = "Hotel near the Louvre with great food nearby."

review_embeddings = model.encode(reviews, show_progress_bar=True)
query_embedding = model.encode([query]).astype('float32')

review_embeddings = review_embeddings.astype('float32')

review_embeddings_normalized = review_embeddings / np.linalg.norm

index = faiss.IndexFlatIP(review_embeddings_normalized.shape[1])
index.add(review_embeddings_normalized)

query_embedding_normalized = query_embedding / np.linalg.norm(query

k = 5  # Number of similar reviews to retrieve
```

```

start_time = time.time()

distances, indices = index.search(query_embedding_normalized, k)

faiss_search_time = time.time() - start_time
print(f"Faiss search time: {faiss_search_time:.4f} seconds")

print(f"Query: {query}")
print("Top hotel with similar reviews using FAISS:")
for i, (idx, distance) in enumerate(zip(indices[0], distances[0])):
    print(f"{i}. {df_paris.iloc[idx]['hotel_name']}")
    print(f"Review: {df_paris.iloc[idx]['review_text']}")
    print(f"Distance: {distance:.4f}")
    print()

```

This code combines all the previous snippets into a single, long snippet.

While it may seem we have a fitting output of just 5 reviews, what if we want more reviews and a more robust answer which gives good evidence of what we are searching for? In the previous code, we limited the number of similar reviews to retrieve to just 5 using the variable `k`. However, to obtain a more comprehensive and reliable set of results, we can increase the value of `k` to retrieve a larger number of similar reviews.

By increasing the number of similar reviews, we can gather more evidence and insights related to our search query. This allows us to analyze a broader range of opinions, experiences, and perspectives from different reviewers. With a larger pool of reviews, we can identify common themes, patterns, and sentiments that strengthen the credibility and relevance of our search results.

Moreover, retrieving more reviews enables us to provide a more diverse and representative set of hotels that match our search criteria. It helps mitigate the impact of individual outliers or biased reviews and presents a more balanced view of the available options. To achieve this, let's modify the code to retrieve a larger number of similar reviews and process them to obtain a more robust answer.

Here's the updated code with more reviews:

Step 1: Perform similarity search using FAISS

Instead of using just 5 reviews, will now increase the number of reviews to 25 to potentially get more than one review for each hotel

```
k = 25  #A
distances, indices = index.search(query_embedding_normalized, k)
```

Step 2: Create a dictionary to store the hotel information.

We will initiate a dictionary to store different attributes about the hotel, including review and description

```
hotel_info = {}
```

Step 3: Iterate over the top-k similar reviews and populate the hotel_info dictionary.

Add hotel reviews with similarity score (distance) for each hotel

```
for idx, distance in zip(indices[0], distances[0]): #A
    hotel_name = df_paris.iloc[idx]['hotel_name'] #B
    if hotel_name not in hotel_info: #C
        hotel_info[hotel_name] = {
            'description': df_paris.iloc[idx]['hotel_description']
            'reviews': [],
            'distances': []
        }
    hotel_info[hotel_name]['reviews'].append(df_paris.iloc[idx]['
    hotel_info[hotel_name]['distances'].append(distance)
```

Step 4: Calculate the overall distance score for each hotel

We will calculate the average distance score of each hotel from the query by taking the average of the distances and counting the number of reviews:

```
for hotel_name in hotel_info: #A
    hotel_info[hotel_name]['overall_distance'] = np.mean(hotel_in
    hotel_info[hotel_name]['num_reviews'] = len(hotel_info[hotel_
```

Step 5: Remove hotels with less than two reviews.

Create a new dictionary called `filtered_hotels` that only includes hotels with at least two reviews

```
filtered_hotels = {hotel_name: hotel_data for hotel_name, hotel_d
```

Step 6: Sort the hotels based on their overall distance score in descending order.

Sort the hotels based on their overall distance score in descending order:

```
sorted_hotels = sorted(filtered_hotels.items(), key=lambda x: x[1
```

Step 7: Print the query and the top hotels with similar reviews using FAISS

Iterate over the sorted hotels and print their details, including the hotel name, description, overall distance score, number of reviews, and the reviews themselves.

```
print(f"Query: {query}")
print("Top hotels with similar reviews using FAISS:")
for i, (hotel_name, hotel_data) in enumerate(sorted_hotels, 1):
    print(f"{i}. {hotel_name}")
    print(f"Description: {hotel_data['description']}")
    print(f"Overall Distance: {hotel_data['overall_distance']:.4f")
    print(f"Number of Reviews: {hotel_data['num_reviews']}")
    print("Reviews:")
    for review in hotel_data['reviews']:
        print(f"- {review}")
    print()
```

In this updated code, we increased the value of `k` to 25 in Step 1, which means we will retrieve the top 25 similar reviews instead of just 5. This larger number of reviews will provide a more comprehensive and robust set of results. By retrieving and analyzing a larger number of similar reviews, we can obtain a more reliable and informative answer to our search query. This approach helps us make more informed decisions based on a broader range of evidence and opinions.

Let's take a look at the results:

Figure 4.8 Hotel Recommendations when we increase the number of reviews: I removed the related reviews because the output was crossing the threshold, but the code should be able to show them

Faiss search time: 0.0017 seconds
Query: Hotel near the Louvre with great food nearby.
Top hotels with similar reviews using FAISS:

- 1. Hotel Moliere**
Description: New four star boutique hotel in chic 1st arrondissement. Our 4* hotel sits in a prime central location in the heart of Paris. Within walking distance of the River Seine, Place Vendome, the Louvre and the Jardin de Tuileries and the Opera Garnier, the 1st arrondissement is considered Paris's most prestigious district. This historic area offers the best of culture and entertainment with traditional architecture style and elegant French formal gardens, world-famous museums, haute-couture and jewellery shops, antiques dealers and art galleries.
Overall Distance: 0.7672
Number of Reviews: 3
- 2. Grand Hotel du Palais Royal**
Description: Un hôtel 5 étoiles au coeur de Paris ! Grâce à sa situation privilégiée, à la fois historique et culturelle, le Grand Hôtel du Palais Royal bénéficie d'atouts uniques au cœur de Paris. A deux pas du Palais Royal et de son romantique jardin à la française, avoisinant le Louvre, et le Jardin des Tuileries, il offre un véritable havre de paix dans la capitale.
Overall Distance: 0.7627
Number of Reviews: 2
- 3. Hotel Square Louvois**
Description: Its location in the heart of Paris' 2nd arrondissement, minutes from the Louvre, Jardin des Tuileries, Palais-Royal, Opera and banks of the Seine, make it the perfect place to soak up the best of the French capital. Whether it's a business trip, romantic break or weekend away with friends, experience the beautiful spa and the quality of innovative services.
Overall Distance: 0.7571
Number of Reviews: 5
- 4. Hotel Malte - Astotel**
Description: Located in the 2nd district next to the Stock Exchange, the Louvre and Place des Victoires, Hotel Malta provides a superb location for a memorable stay. Whether you want to shop in the heart of the department store neighbourhood or visit one of the main business centres of Paris, Malta Hotel is the place to be. Elegantly concealed behind a facade dating from the seventeenth century, let yourself be seduced by our garden patio, a pleasurable experience on sunny days, our cozy breakfast room, and the relaxing atmosphere of our lobby. After the warm welcome by our professional team, always at your disposal, you will appreciate the comfort of our rooms decorated in an elegant and timeless design. A convenient location, caring personnel and warm atmosphere make Malta Hotel the perfect place for your stay in the capital, whether traveling with family or on your own.
Overall Distance: 0.7534
Number of Reviews: 8
- 5. Hotel Tourisme Avenue**
Description: Finding an ideal charming 4 stars hotel in Paris does not have to be difficult. Welcome to Hotel Tourisme Avenue, a nice option for travelers like you. For those interested in checking out popular landmarks while visiting Paris, Hotel Tourisme Avenue is located a short distance from Champs-Elysees (1.5 mi) and Arc de Triomphe (1.7 mi). Guest rooms offer amenities such as a flat screen TV, air conditioning, and a refrigerator, and guests can go online with free wifi offered by the hotel. Hotel Tourisme Avenue features a concierge, to help make your stay more enjoyable. The property also boasts breakfast buffet. If you're looking for a steakhouse, consider a visit to Le Relais de l'Entrecote, Il Etait Un Square, or La Poule au Pot, which are all conveniently located a short distance from Hotel Tourisme Avenue. During your visit, be sure to check out popular attractions like Eiffel Tower (0.6 mi) and Trocadero (1.0 mi), which are all within walking distance of the hotel. Hotel Tourisme Avenue looks forward to welcoming you on your visit to Paris.
Overall Distance: 0.7380
Number of Reviews: 2
- 6. Best Western Plus La Demeure**
Description: Best Western Plus La Demeure is housed in an elegant Haussmannian building in Paris's legendary Latin Quarter and has been entirely redecorated by two young designers, Flavie Papin and Paul Jouan. Their innovative approach created unusual, pleasing spaces that are practical, beautiful and full of light. This family-run establishment offers you a warm welcome and a totally unique experience, particularly in the remarkable themed suites. Book on our site for the best rates.

Wow, the output from the code above is truly impressive! It provides a

fantastic list of the top hotels with reviews similar to the query "Hotel near the Louvre with great food nearby" using the powerful FAISS similarity search. By increasing the number of reviews considered and calculating an overall distance score for each hotel, the results offer a much more comprehensive and reliable set of options compared to just looking at a handful of reviews. This approach ensures that we get a well-rounded view of each hotel, taking into account multiple perspectives and experiences.

The top hotels in the list, such as Hotel Moliere, Grand Hotel du Palais Royal, and Hotel Square Louvois, boast excellent locations near the Louvre and other key attractions, along with glowing reviews praising their helpful staff, clean and comfortable rooms, and delicious breakfast offerings. With a range of reviews for each hotel, we can feel confident in the quality and suitability of these recommendations. Hotel Malte - Astotel particularly stands out with its eight reviews, providing an even stronger endorsement of its superb location, elegant rooms, and complimentary amenities. It's exciting to see how much we can achieve by using just a few lines of code. As a next step, we could consider consolidating the reviews into a summary to reduce the amount we need to read to make an informed decision and remove redundancy in the reviews.

As we conclude this chapter, we have successfully built a solid foundation for our Paris Travelle hotel search engine. However, our journey is far from over. In the next chapter, we will explore even more advanced topics to further enhance the capabilities of our search system. We will dive deeper into FAISS indexing techniques, such as inverted file systems and product quantization, to optimize search performance and scalability. Additionally, we will investigate the use of vector databases to efficiently store and manage large-scale embedding data.

Moreover, we will examine how semantic search can be combined with traditional keyword-based search to provide a more comprehensive and flexible search experience. By leveraging the strengths of both approaches, we can offer users the ability to search for hotels based on specific amenities, locations, or other criteria, while still benefiting from the semantic understanding of the reviews.

4.6 Summary

- The first step towards building Semantic Search is to identify the data needed to build the engine. For Travelle (hotel search engine for finding relevant hotels in Paris based on free form search), we began by exploring the hotel dataset, which provided us with a rich collection of hotel information and reviews, serving as the foundation for our search engine.
- To capture the semantic meaning of the hotel reviews, we utilized state-of-the-art models like Sentence Transformers to convert the text into dense vector representations, also known as embeddings. Specifically, we employed the Nomic Embedding Model, which has shown impressive performance in capturing semantic similarity.
- It's worth noting that there is a vast array of encoder models available for generating embeddings. The choice of the most suitable model depends on various factors, such as the specific use case, the size of the dataset, and whether an open-source or closed-source model is preferred. Careful consideration should be given to selecting the model that aligns best with the requirements of the search engine.
- Distance metrics play a pivotal role in measuring the similarity between embeddings and the user's query. Two commonly used metrics are cosine similarity and Euclidean distance. These metrics allow us to quantify the proximity or similarity between the query and the hotel reviews in the vector space, enabling us to retrieve the most relevant results.
- When it comes to semantic similarity, two popular architectures are the Bi-Encoder and Cross-Encoder models. The Bi-Encoder model offers advantages in terms of efficiency and computational resources, making it a preferred choice for many applications. It allows for the pre-computation and storage of embeddings, enabling fast retrieval during the search process.
- To efficiently store and search through the embeddings at scale, FAISS (Facebook AI Similarity Search) is a powerful library designed for similarity search and clustering of dense vectors. FAISS provides a range of indexing techniques and algorithms that optimize the search process, ensuring fast and accurate retrieval of the most relevant hotel reviews based on the user's query.

5 Decoders in Action

This chapter covers

- Understanding Decoder model core principles
- A deeper dive into decoding algorithms
- An insight into using prompt engineering for our application
- Choosing between Open-Source vs Closed-Source LLMs
- Challenges within LLMs

In Chapter 4, we built a solid foundation for our semantic search engine by exploring the process of generating embeddings, utilizing distance metrics, and introducing the powerful FAISS library. We witnessed the effectiveness of semantic search in understanding user queries and retrieving relevant hotel information.

In this chapter, we will explore the most popular theme in today's Gen AI Landscape: Decoder models or simply large language models. As we have witnessed in previous chapters, Encoder models play an important role in generating embeddings and enabling us to search through a myriad of documents. Decoder models are responsible for generating text, which will become quite handy in the next chapter as we explore Retrieval Augmented Generation (RAG) explained in Chapter 1. I am also very excited to have a contributor for this chapter, Ali Shafique, a PhD candidate focused on building State of the Art LLMs.

While we've successfully built a semantic search system in the previous chapter that can effectively find relevant hotel information, we now face a new challenge: naturally presenting this information, engaging, and personalized for each user. Simply returning a list of search results, even if they're semantically relevant, doesn't provide the rich, interactive experience that today's users expect. This is where Large Language Models come into play, and why we need to take a brief detour to understand their capabilities before combining them with our search functionality.

Think of our journey as building a sophisticated travel assistant rather than just a search engine. While our encoder-based search system can find the perfect hotels based on user criteria, we need decoder models (LLMs) to transform these raw search results into natural conversations, explanations, and personalized recommendations. For instance, instead of just showing that a hotel matches a user's query for "romantic getaway near the Eiffel Tower," an LLM can explain why the hotel would be perfect for a romantic stay, describe the ambiance of its rooftop restaurant, and suggest nearby activities for couples - all in a natural, conversational manner. This detour into understanding LLMs will ultimately allow us to create a more complete and sophisticated system that not only finds relevant information but presents it in a way that truly enhances the user experience.

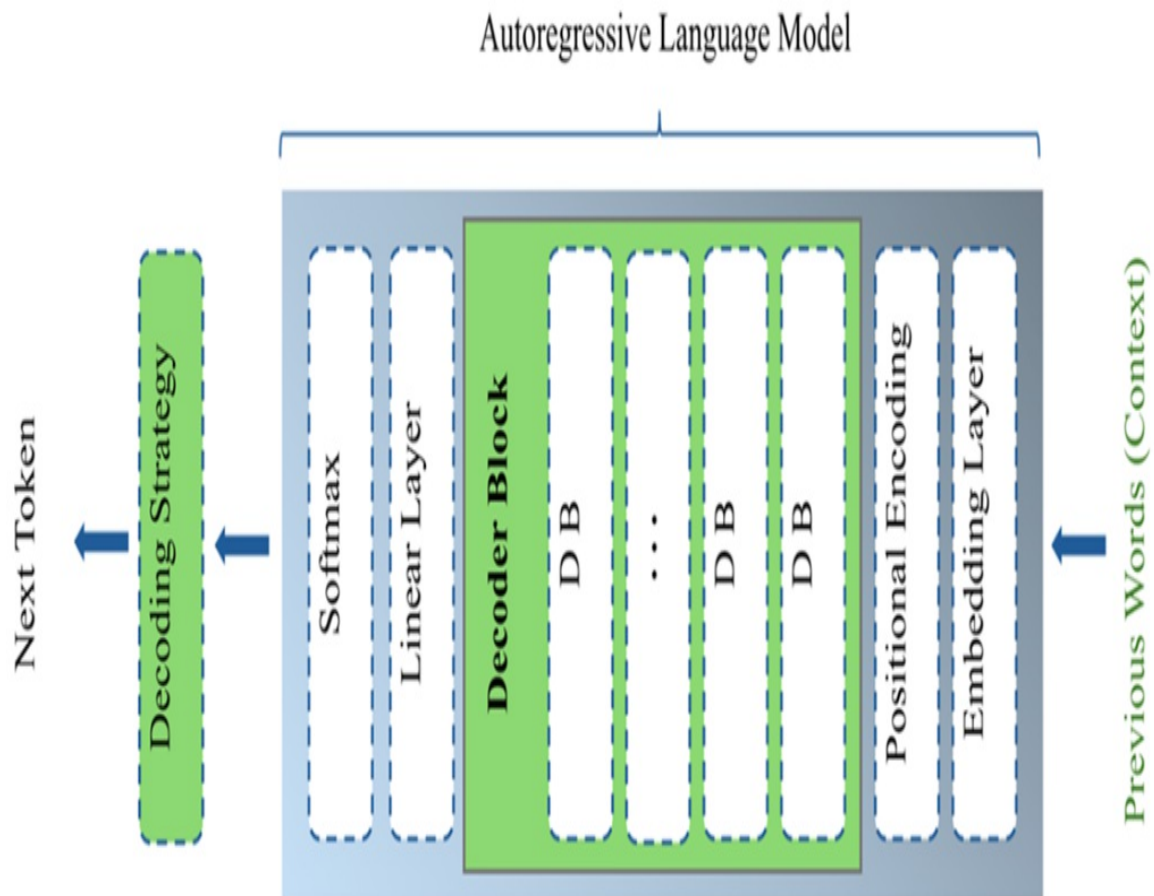
For this chapter, we will go deeper into the fundamental concept of autoregressive language models and explore well-known decoding strategies such as Greedy Decoding, Beam Search, Top-k Sampling, Top-p Sampling (Nucleus Sampling) and Temperature Scaling. By understanding these techniques, users can optimize their use of LLMs to achieve more effective and tailored text generation. From there, we'll explore various prompting techniques, including few-shot learning and chain-of-thought prompting, that help us extract the best performance from these models. Through practical examples, we'll demonstrate how different prompting strategies can dramatically affect the quality and relevance of the model's outputs, and how to optimize these prompts for specific use cases.

The landscape of Decoder models is diverse, ranging from open-source alternatives like Llama (by META) and Mistral to closed-source solutions such as GPT-4 and Claude. Towards the end of the chapter we will compare these different options, discussing their strengths, limitations, and appropriate use cases. Understanding these distinctions is crucial for developers and organizations when choosing the right model for their specific needs, considering factors such as cost, performance, and deployment requirements.

5.1 Understanding the Core Principles of Decoder Models

While we covered the core architecture of Decoders in Chapter 2, we will try to focus on the building blocks of Decoder models in this chapter. The emergence of ChatGPT at the end of 2022 revolutionized the world with its ability to generate human-like text responses. At its core, ChatGPT is a decoder-only transformer-based autoregressive language model that predicts the probability distribution of the next token (sub-word) one by one based on the input prompt. These models, now commonly referred to as Large Language Models (LLMs), have become increasingly popular. The quality and characteristics of their responses can be fine-tuned using various decoding algorithms with customizable settings.

Figure 5.1 Autoregressive Language Models predict the next word by processing a sequence of previous words. They use embedding, positional encoding, and multiple decoder blocks to generate a probability distribution for the next word.



At their heart, decoder models employ a sophisticated architecture designed for sequential processing and generation. The key components include:

- **Token Embeddings:** Converting input text into numerical representations.
- **Positional Encodings:** Maintaining sequence order information.
- **Multiple Decoder Layers:** Processing information through successive transformations.
- **Masked Self-Attention:** Ensuring causality in generation.
- **Feed-Forward Networks:** Processing transformed representations.

5.1.1 Autoregressive Nature of LLMs

The fundamental characteristic of LLMs is their autoregressive property, which allows them to predict the next token (sub-word) distribution in a sequence based on the preceding words with prompt and previously generated words.

This property enables the model to:

- Generate coherent text sequences.
- Maintain context across long passages.
- Produce contextually appropriate responses.
- Learn and apply patterns from training data.

For example, when generating the sentence "*The cat sat on the...*", the model uses the probability distribution of all possible next words, considering the context of the previous words to decide that "mat" might be a more appropriate completion than "cloud" or "computer". The autoregressive property can be expressed mathematically as a probability distribution, where each token is predicted based on all previous tokens in the sequence. This creates a chain of predictions that forms the basis for text generation.

Mathematically, the probability of generating a sequence

$$S = \{w_1, w_2, \dots, w_n\}$$

is expressed as:

$$P(S) = P(w_1) \times P(w_2|w_1) \times \dots \times P(w_n|w_1, w_2, \dots, w_{n-1})$$

where:

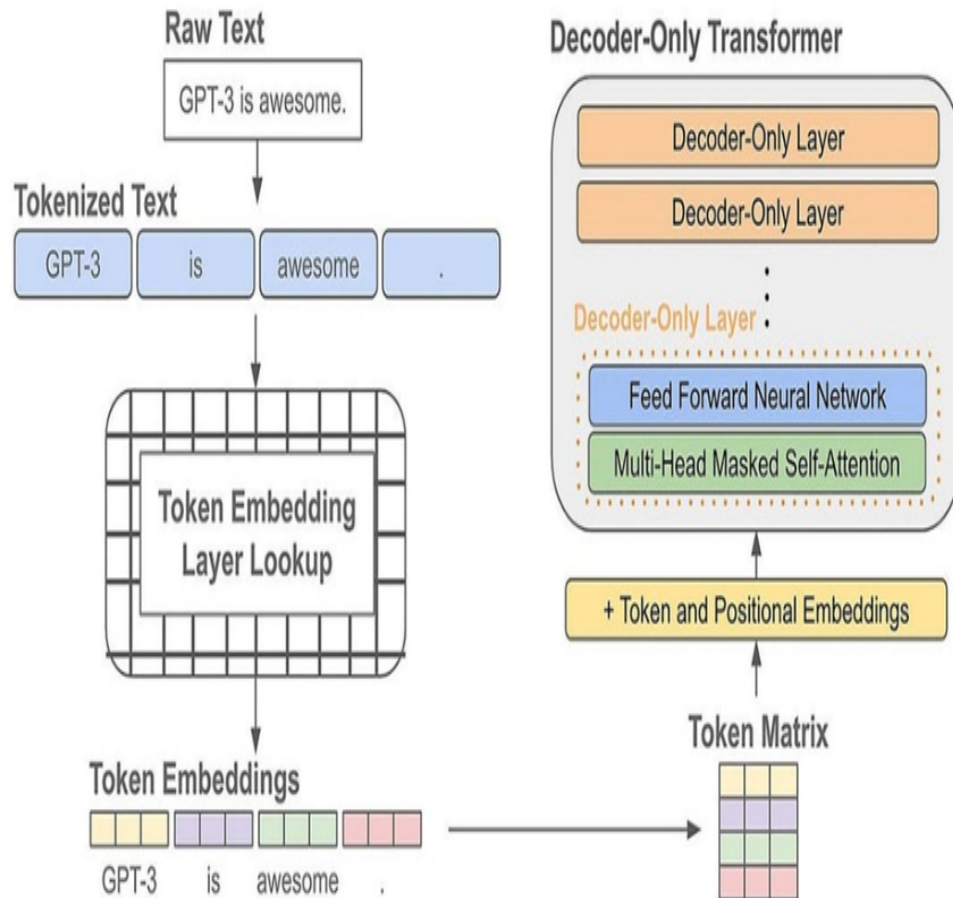
w_n is the next token to be predicted.

$w_1, w_2, \dots, w_{n-1}$ are the preceding tokens in the sequence.

This property enables the model to generate text one word at a time, conditioned on the context provided by the preceding words, leading to coherent and contextually relevant text. How do LLMs know how to generate words in a coherent and contextually relevant manner? ,Ä two words:

Decoding Algorithms

Figure 5.2 Process of how a Decoder-Only Transformer model, like GPT-3, processes text by converting raw text into tokenized representations, embedding them, and feeding them through multiple layers of self-attention and feed-forward networks to generate an output.



5.2 Decoding Algorithms

While LLMs generate probabilities for possible next tokens, decoding algorithms convert these probabilities into coherent text. The choice of algorithm directly influences the generated text's coherence, diversity, and style.

Here are some of the most well-known decoding algorithms, that we will introduce next:

- Greedy Decoding
- Beam Search
- Sampling Methods:
 - Top-k Sampling
 - Top-p Sampling (Nucleus Sampling)
 - Temperature Scaling

5.2.1 Greedy Decoding

Greedy Decoding is a simple and straightforward decoding algorithm used in natural language processing for LLMs. It's a method for generating text by selecting the most likely word at each step.

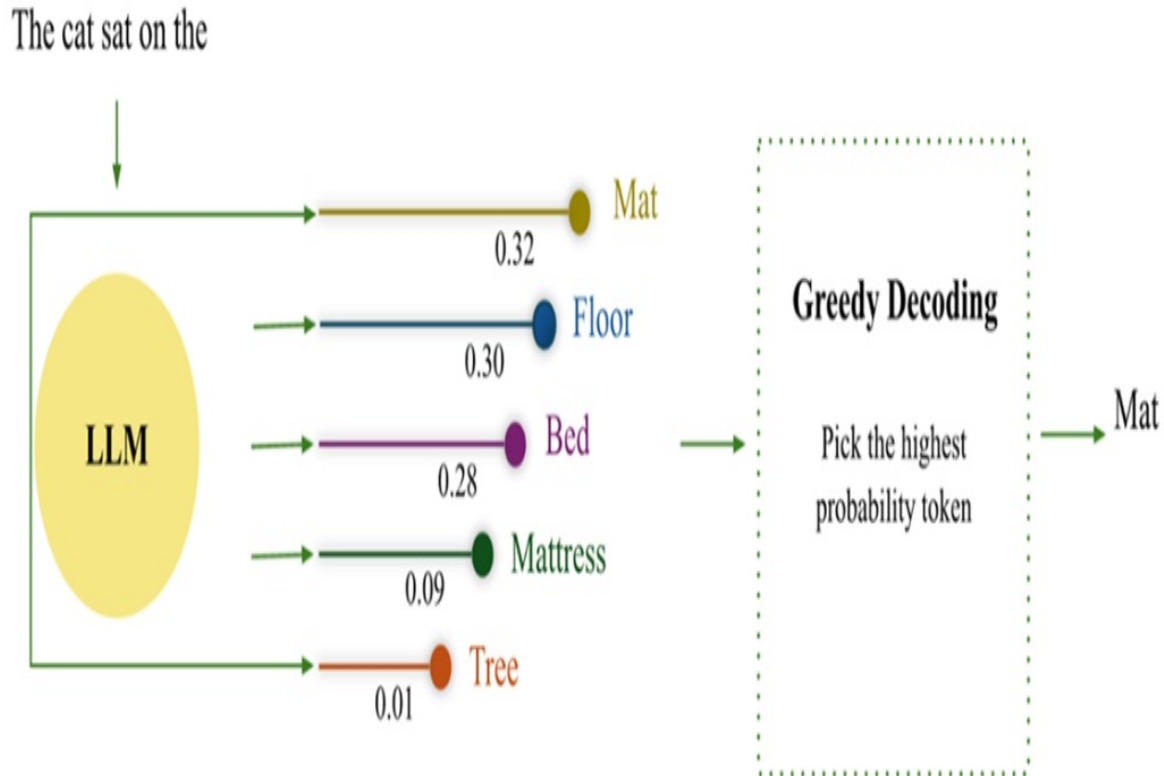
Imagine you have a sentence fragment like "The cat sat on the...". A language model can predict the probability of various words following this fragment, such as "jumps," "runs," "walks," and so on, Greedy Decoding would simply choose the word with the highest probability as the next word in the sequence.

Greedy decoding selects the word with the highest probability at each step:

$$w_t = \operatorname{argmax}_w P(w \mid w_1, w_2, \dots, w_{\{t-1\}})$$

where w_t is the next token to be predicted and $w_1, w_2, \dots, w_{\{t-1\}}$ are the preceding tokens in the sequence.

Figure 5.3 Greedy Decoding method used by Large Language Models. The LLM generates a probability distribution for the next word, and Greedy Decoding selects the word with the highest probability



How it works:

1. **Initialization:** Start with the input prompt or the first word of the sequence.
2. **Prediction:** The language model predicts the probability distribution of the next word given the current context.
3. **Selection:** Choose the word with the highest probability from the distribution.
4. **Append:** Add the selected word to the sequence.
5. **Repeat:** Repeat steps 2-4 until the desired sequence length is reached or a termination condition is met (e.g., encountering an end-of-sequence token).

Advantages:

- **Computational Efficiency:** Greedy decoding is computationally

efficient as it only needs to consider the most probable word at each step.

- **Implementation Simplicity:** It's relatively easy to implement due to its straightforward nature.

Limitations:

- **Repetitive Output:** Greedy decoding can often lead to repetitive and monotonous text because it always selects the most probable word, even if it results in less diverse or creative output.
- **Lack of Exploration:** It doesn't explore alternative paths that might lead to better or more creative sequences. This can limit the diversity and quality of the generated text.

Greedy Decoding is a basic and efficient decoding algorithm, but its limitations in terms of diversity and creativity often make it less suitable for applications where these factors are crucial. More sophisticated decoding methods, such as Beam Search, Top-k Sampling, and Nucleus Sampling, have been developed to address these limitations and generate more diverse and engaging text.

5.2.2 Beam Search

Beam Search explores multiple paths through the search space, selecting the one with the highest cumulative probability.

How it works:

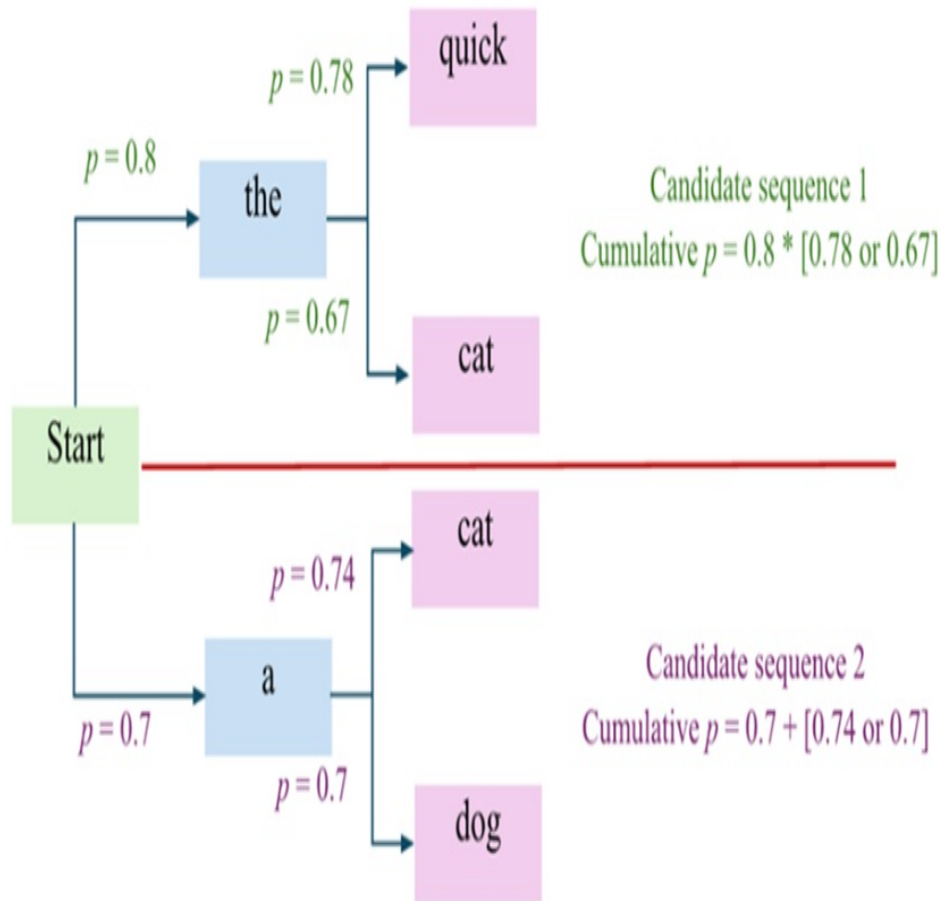
1. **Initialization:** Start with the top-k most likely initial tokens as candidate sequences.
2. **Iterative Growth:** For each candidate sequence, append the top-k most probable next tokens, creating new sequences.
3. **Selection:** Assign a score to each new sequence based on its cumulative probability and keep only the top-k scoring sequences.
4. **Termination:** Repeat this process until either a maximum token length is reached, or all candidate sequences end with an end-of-sequence token.

Example:

Let's consider generating the phrase "the quick brown fox" using Beam Search with a beam width (k) of 2.

- Initialization:
 - Start with two candidate sequences: "the" and "a".
- Step 1:
 - For "the", the top 2 probable next words might be "quick" and "cat".
 - For "a", the top 2 probable next words might be "dog" and "cat".
 - This creates four new candidate sequences: "the quick", "the cat", "a dog", and "a cat".
 - Select the top 2 scoring sequences (e.g., "the quick" and "the cat" based on their probabilities).

Figure 5.4 Beam search starting with two candidates, "the" and "a". Two candidate sequences are highlighted, each with a calculated cumulative probability based on the probabilities of the individual words within the sequence.

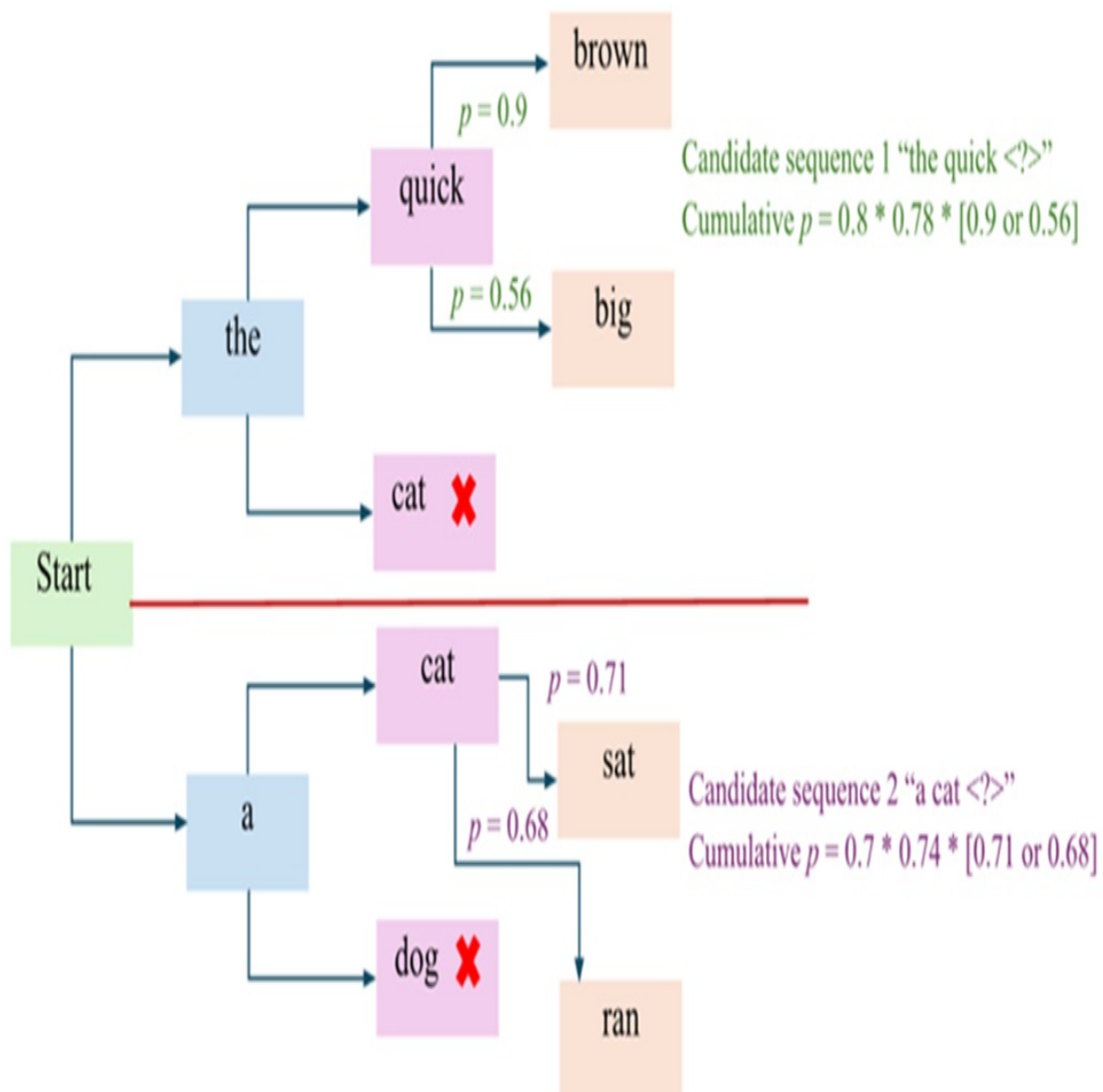


- Step 2:
 - For "the quick", the top 2 probable next words might be "brown" and "big".
 - For "the cat", the top 2 probable next words might be "sat" and "ran".
 - This creates four new candidate sequences: "the quick brown", "the quick big", "the cat sat", and "the cat ran".
 - Select the top 2 scoring sequences (e.g., "the quick brown" and "the cat sat").

As shown in figure 5.4, the network of words from the previous image has been expanded. New words and probabilities have been added. The original

candidate sequences have been updated to include these new words, and their cumulative probabilities have been recalculated. Additionally, two words ("cat" and "dog") have been marked with an "X" to indicate that they are not part of the correct sequence.

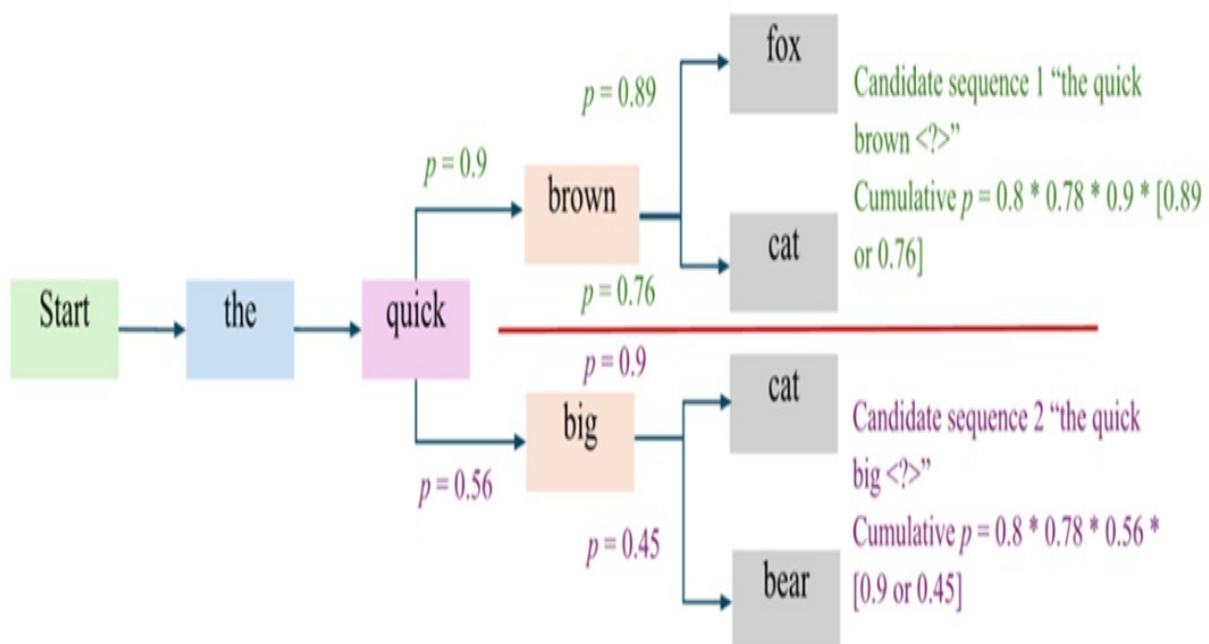
Figure 5.5 The word network has been expanded, with new words and probabilities added, resulting in updated candidate sequences and cumulative probabilities, while some words have been marked as incorrect



Continue this process until the desired sequence is generated or the termination condition is met. The result is depicted in the image below (Figure 5.5) which showcases a complete word network, where each word is connected to others with associated probabilities.

The two candidate sequences, "the quick brown..." and "a cat sat...", have been further evaluated. The cumulative probabilities for each sequence have been recalculated based on the updated word network. The red "X" marks indicate words that have been identified as incorrect in the sequences. The image represents the culmination of the analysis process, providing a comprehensive view of the word network and the evaluation of the candidate sequences.

Figure 5.6 The final image presents the complete word network with updated candidate sequences and their probabilities, along with identified incorrect words.



The three images illustrate the concept of beam search, an algorithm used

Decoders for finding the most probable sequence of words. Beam search maintains a set of "candidate sequences" and their associated probabilities.

At each step:

- **Expansion:** The algorithm expands each candidate sequence by adding a new word from the vocabulary.
- **Scoring:** Each expanded sequence is scored based on the probability of the new word given the previous words in the sequence.
- **Pruning:** The algorithm keeps only the top "k" sequences with the highest scores (where "k" is the beam width). This step effectively prunes lower-scoring sequences, reducing the search space.

The process continues until a termination condition is met (e.g., reaching a maximum length or encountering an end-of-sequence token). The final output is the highest-scoring sequence among the remaining candidates. In essence, beam search explores a wider range of sequences than greedy search (which only considers the highest-scoring word at each step) while still being computationally more efficient than exhaustive search.

Advantages:

- Balances exploration and exploitation, often resulting in more coherent and diverse text than greedy decoding.
- Considers multiple paths, which can lead to more globally optimal sequences.

Limitations:

- Can still suffer from repetition and may not capture the full diversity of possible outputs.
- Can be computationally expensive, especially for large values of k.

5.2.3 Sampling Methods

Sampling methods are probabilistic techniques for selecting the next word in a sequence generated by an LLM. Unlike deterministic methods like greedy decoding, sampling introduces randomness, allowing the model to explore a

wider range of possibilities and generate more diverse and creative outputs. There are three popular types of Sampling Methods:

Temperature Sampling

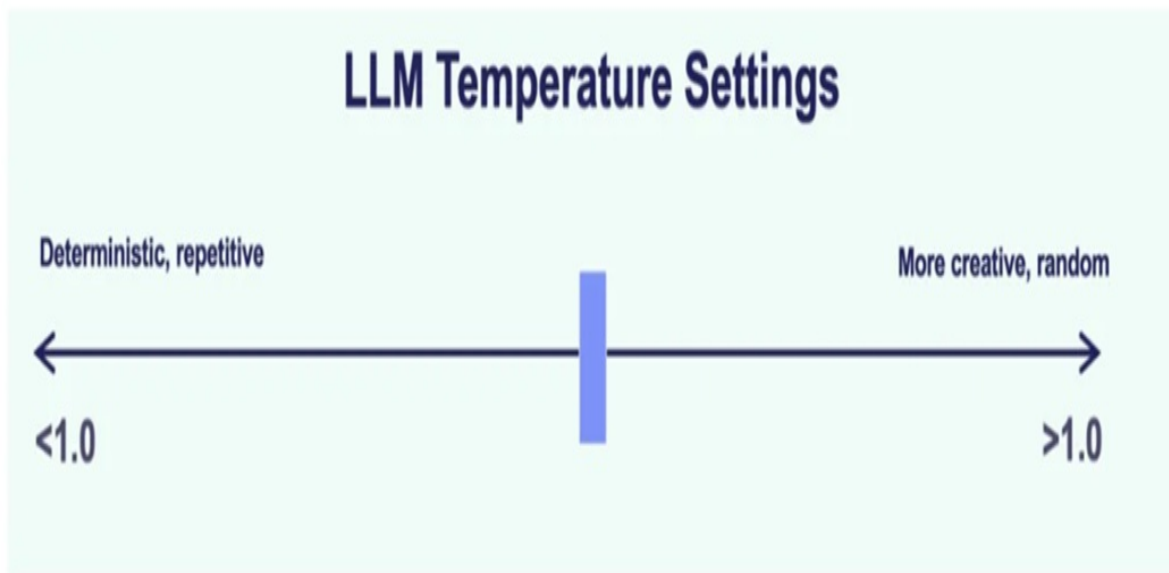
Adjusts the probability distribution of the next word by dividing the log-probabilities by a temperature parameter (T).

Formula: $P(w_i) = \exp(\log(P(w_i)) / T) / \sum_j (\exp(\log(P(w_j)) / T))$

Examples:

- High Temperature ($T > 1$): Increases randomness, leading to more surprising and creative outputs, but may also produce nonsensical or irrelevant text.
- Low Temperature ($T < 1$): Makes the model more deterministic, favoring the most probable words. T close to 0 approaches greedy decoding.

Figure 5.7 An overview of how temperature settings in LLMs control the balance between deterministic, repetitive output (low temperature) and more creative, random output (high temperature).



- Advantages: Simple to implement, allows for smooth control over randomness.
- Disadvantages: Can be sensitive to temperature settings. High temperatures may lead to gibberish, while low temperatures can limit creativity.

Top-k Sampling

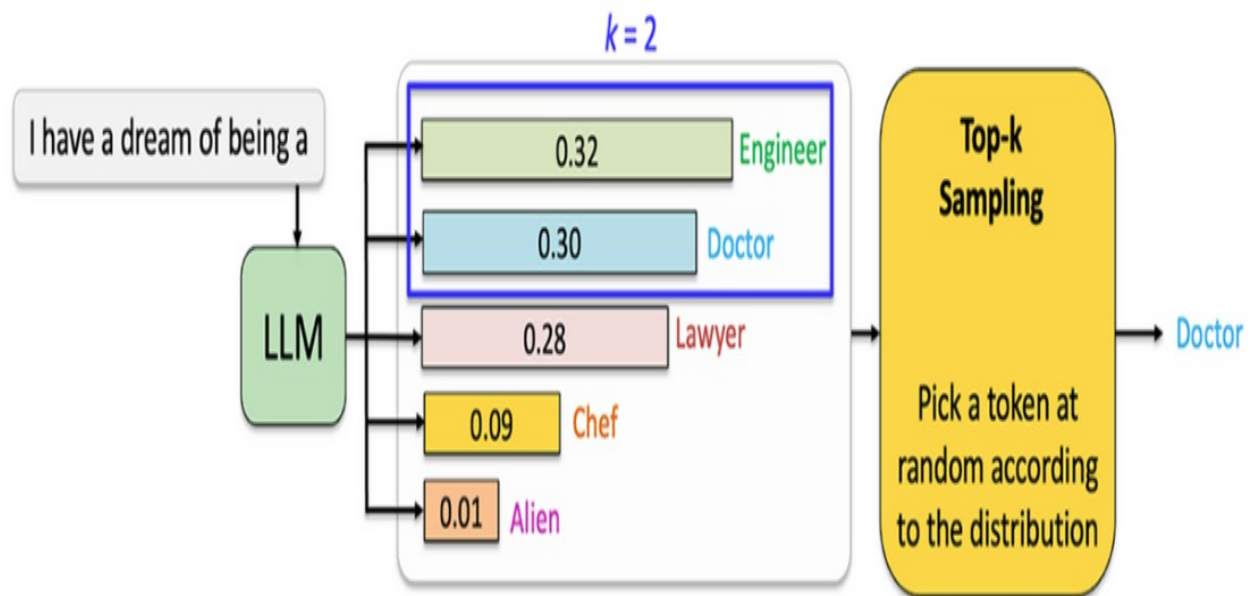
Selects the next word from the top-k most probable words in the vocabulary.

Examples:

- $k = 1$: Equivalent to greedy decoding.
- $k > 1$: Increases diversity by considering multiple high-probability words.
- Input: "The quick brown fox"
- Top-k Sampling ($k=3$) Output: "The quick brown fox jumps over the lazy dog...fox...jumps" (Only the top 3 most likely words are considered at each step)

Figure 5.8 Top-k sampling in LLMs, where the model generates a probability distribution for the

next word, and only the top-k most probable words are considered, with the final word chosen randomly based on their probabilities.



- Advantages: Easy to implement, provides direct control over the number of considered words.
- Disadvantages: May still miss out on less probable but relevant words, especially for small values of k .

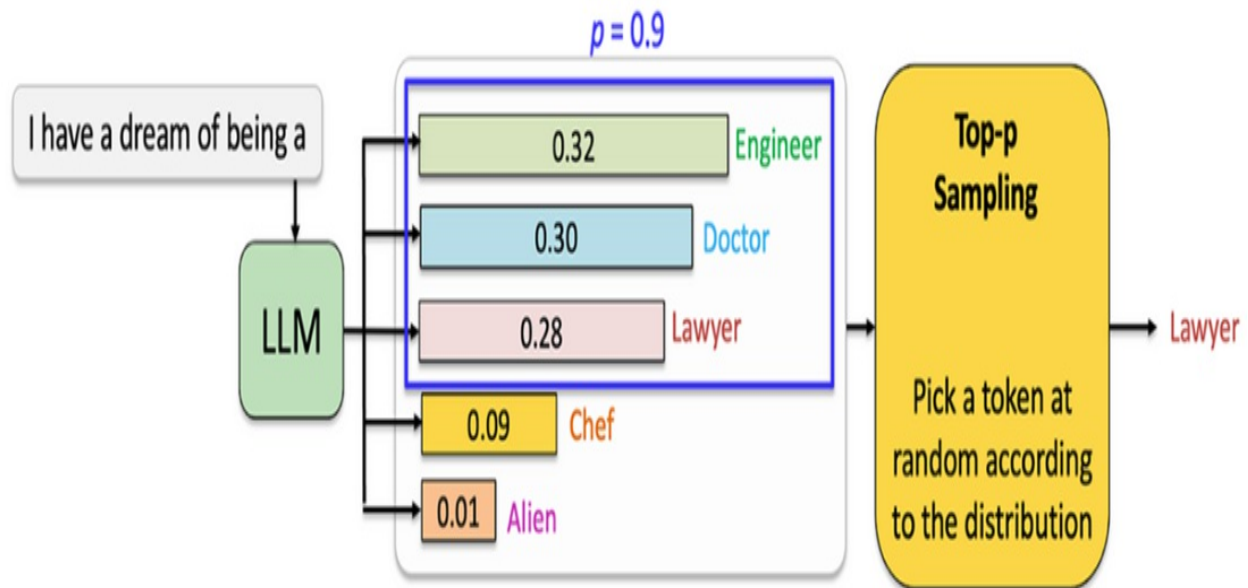
Top-p Sampling (Nucleus Sampling)

Selects the next word from the subset of words whose cumulative probability reaches a certain threshold (p).

Examples:

- $p = 0.8$: Selects words until their cumulative probability reaches 90%.
- Input: "The quick brown fox"
- Top-p Sampling ($p=0.8$) Output: "The quick brown fox jumps over the lazy dog...forest...stream" (Only words that cumulatively cover 80% of the probability mass are considered)

Figure 5.9 Top-p sampling in LLMs, where the model generates a probability distribution for the next word, and only the subset of words with a cumulative probability up to the threshold (p) are considered, with the final word chosen randomly based on their probabilities.



- Advantages: More flexible than top-k sampling. Can control the degree of randomness by adjusting the nucleus probability (p).
- Disadvantages: Can be computationally more expensive than top-k sampling, especially for large vocabularies.

In the next few sections, we will see these techniques in action with code. Sampling methods are crucial for generating diverse and creative text with LLMs. By introducing randomness, they enable the model to explore a wider range of possibilities beyond the most likely word. Temperature sampling, top-k sampling, top-p sampling, and reinforcement learning offer different approaches to controlling randomness and optimizing for specific objectives.

Overall, Decoding algorithms play a pivotal role in harnessing the power of LLMs to generate high-quality, coherent, and diverse text. The choice of the appropriate decoding algorithm is crucial for optimizing the performance of LLMs in various applications. Understanding and selecting the suitable decoding algorithm is essential for achieving the desired balance between creativity and coherence in the generated text.

After exploring **decoding algorithms** in LLMs, which shape how models select and sequence their outputs, the next crucial piece of the puzzle is how do we get started with using LLMs, which will be followed by the important topic of **prompting**, the art and science of framing inputs to guide the

model toward desired responses.

5.3 Getting Started with Large Language Models and Prompting

LLMs are fundamentally **general-purpose predictors**. They generate outputs based on patterns learned during training, but without explicit instructions, their responses may be overly generic, ambiguous, or irrelevant to the task at hand. Prompting serves as the bridge between a model's capabilities and the user's intent, ensuring the output aligns with specific goals. This subsection is credited to Ali Shafique for his invaluable contribution.

5.3.1 What is Prompting?

Prompting involves crafting the input text in a way that effectively communicates the task to the model. It sets the context, defines the problem, and often specifies the format of the expected output.

The Role of Prompt

A prompt is a natural language text that requests generative AI models (LLMs) to perform a specific task. Using a prompt to instruct an AI to do a task is called prompting.

To see LLMs and promoting in action, we'll start with one of the most accessible and powerful platforms: OpenAI's API. The principles we learn here apply broadly to different LLM providers.

Getting started with LLMs requires three key components:

- API access,
- A basic understanding of the interface, and,
- Well-crafted prompts

Here's a basic example using OpenAI's API (you will have to generate an Open AI Key from their website [link:

<https://platform.openai.com/settings/organization/api-keys>]):

Listing 5.1 Using Open AI,Âs Decoder Model [\[Colab Link\]](#)

```
!pip install openai
from openai import OpenAI

client = OpenAI(api_key='your-api-key') #A

prompt = "I am planning to go to Paris, Can you tell me about hot
model = "gpt-4o" #C

temperature = 0.7 #D

max_tokens = 500 #E
```

Initialize the Client and pass a prompt

```
def generate_text(prompt, model="gpt-4o"):
    chat_completion = client.chat.completions.create( #A
        messages=[
            {
                "role": "user",
                "content": prompt,
            }
        ],
        model=model,
        temperature=temperature,
        max_tokens=max_tokens,
    )

    return chat_completion.choices[0].message.content #B
```

Print the Output

```
generate_text(prompt)#A
```

Paris, known as the "City of Light," is home to a wide range of h

- 1. Luxury Hotels:** Paris is renowned for its luxury accommodations
 - ,óã The Ritz Paris: Located in Place Vendôme, it's known for i
 - ,óã Le Meurice: Situated near the Louvre, it combines classic
 - ,óã Hôtel de Crillon: Overlooking the Place de la Concorde, it
- 2. Boutique Hotels:** These smaller, stylish hotels offer a more pe
 - ,óã Hotel des Grands Boulevards: Combines modern design with h
 - ,óã Le Roch Hotel & Spa: Offers a chic, intimate atmosphere wi
- 3. Mid-Range Hotels:** There are plenty of mid-range options that p

- ,óã Hôtel Le 123 Sébastopol: Known for its cinematic theme and
 - ,óã Hotel Fabric: Located in the Oberkampf district, it offers
 - 4. **Budget Hotels and Hostels:** For travelers on a budget, Paris of
 - ,óã Generator Paris: A trendy hostel with both dormitory and p
 - ,óã ibis Budget Paris La Villette: Offers basic accommodations
 - 5. **Unique Stays:** For something different, consider staying in a h
 - ,óã Hôtel Particulier Montmartre: A hidden gem with a garden a
 - ,óã Les Bains: Once a famous nightclub, now a stylish hotel wi
 - 6. **Location Considerations:** When choosing a hotel in Paris, consi
 - ,óã The Marais: Known for its historic charm and vibrant night
 - ,óã Saint-Germain-des-Prés: Offers a mix of art galleries, caf
 - ,óã Montmartre: Famous for its bohemian vibe and the Sacré-C≈i
 - 7. **Booking Tips:**
 - ,óã Book well in advance, especially during peak tourist seaso
 - ,óã Consider staying slightly outside the city center for more
- Each neighborhood in Paris has its own character and charm, so co

While this code will generate a response, the quality, and usefulness of that response heavily depend on how we structure our prompt. Let,Äôs consider a different and more structured approach and then we can compare the two:

```
# More structured prompt (better)
structured_prompt = """
Provide information about hotels in Paris, including:
- Location considerations
- Price ranges
- Popular areas for tourists
- Transportation access
Please format the response in clear parts"""
```

Output:

Location Considerations

When choosing a hotel in Paris, consider the following factors:

Proximity to Attractions: Staying near major attractions like the

Neighborhood Vibe: Each Parisian neighborhood, or arrondissement,

Safety: Paris is generally safe, but like any major city, it's wi

Price Ranges

Hotel prices in Paris can vary widely based on location, season,

Budget Hotels: Prices can start as low as ,Ç"50-,Ç"100 per night.

Mid-Range Hotels: Expect to pay between ,€100- ,€250 per night.

Luxury Hotels: Prices can range from ,€250 to over ,€1,000 per

Popular Areas for Tourists

The Marais (3rd and 4th Arrondissements): Known for its historic

Saint-Germain-des-Prés (6th Arrondissement): Offers a blend of hi

Latin Quarter (5th Arrondissement): Famous for its student life,

Montmartre (18th Arrondissement): Known for its artistic history,

Transportation Access

Paris has an extensive public transportation system, making it ea

Metro: The Paris Metro is efficient and covers most of the city.

RER Trains: Useful for reaching destinations outside the city cen

Buses: Offer a scenic way to travel, though they can be slower du

Walking: Paris is a walkable city, especially in the central arro

Bicycles: The city has a bike-sharing system called Vélib', which

Conclusion

When choosing a hotel in Paris, consider what you value most: pro

The difference in output quality between these two prompts demonstrates why giving a detailed prompt or instruction helps in generate a better answer. Looking at these two responses, we can see:

- The structured prompt organizes information into clear sections, making it easier for readers to find specific details.
- It provides specific guidance on location considerations, including proximity to attractions and neighborhood vibes, which helps readers make informed decisions based on their preferences.
- The structured prompt includes a detailed breakdown of price ranges, offering a clearer understanding of what to expect financially.
- It highlights popular tourist areas with specific characteristics, aiding travelers in selecting neighborhoods that match their interests.
- The structured prompt offers comprehensive information on

transportation access, enhancing the reader's ability to plan their travel logistics.

- It concludes with a summary that reinforces key considerations, helping readers prioritize their needs and preferences.

The Crucial Distinction: LLMs vs Search Engines

Our outputs might feel like we have all the information we want, so why do we need to worry about Googling anything anymore? We could use ChatGPT for all our searches! This is a tricky slope, and we must be very careful. While Large Language Models demonstrate impressive capabilities in generating human-like text and engaging in conversations, it's crucial to understand that they are not search engines. LLMs operate by predicting the next most likely tokens based on their training data (we discuss hallucinations in the upcoming sections), which means they can generate plausible-sounding but potentially inaccurate information. Unlike search engines that retrieve and rank existing documents, LLMs generate responses in their simplest form without direct access to a current, verified knowledge base. This limitation, often referred to as hallucination, means that while an LLM might provide a convincing answer about a hotel in Paris, it might generate details that are outdated, inaccurate, or entirely fictional.

This is precisely why the combination of search capabilities (through encoders) and text generation (through decoders) is so powerful. When we build a search system, we want to ensure that the information provided is accurate, up-to-date, and grounded in real data. By using encoder models to first retrieve relevant, factual information from our database, and then using decoder models to generate natural, personalized responses based on that retrieved information, we can create a system that combines the best of both worlds. This approach, known as Retrieval-Augmented Generation (RAG), which we'll explore in detail in Chapter 6, ensures that our responses are both engaging and accurate, providing users with reliable information presented in a natural, conversational manner.

At its core, a prompt is simply an instruction or input given to a language model. Think of it as having a conversation with a highly knowledgeable assistant who needs clear directions to provide helpful responses. Basic

prompts often yield varied and sometimes unpredictable results. Let's see how we can improve them through *prompt engineering*.

5.3.2 Prompt Engineering

Prompt engineering is the practice of designing prompts that consistently produce desired outputs. The art and science of prompt engineering has emerged as a crucial skill in the era of large language models. While these models possess remarkable capabilities, their outputs are heavily dependent on how we frame our instructions and queries. In this subsection, we explore the evolution of prompting techniques, from basic instructions to sophisticated engineering approaches that unlock the full potential of these AI systems for our travel search tool, Travelle.

At its core, prompt engineering is the practice of crafting inputs that effectively guide language models toward desired outputs. Think of it as learning to communicate with an incredibly knowledgeable but extremely literal assistant.

As our understanding of LLMs has grown, three major prompting paradigms have emerged: zero-shot, few-shot, and chain-of-thought prompting. Each serves different purposes and offers unique advantages in specific situations.

We have three major types:

- *Zero-Shot Prompting*
- *Few-Shot Prompting*
- *Chain-of-Thought (CoT) Prompting*

Zero-shot prompting represents the most straightforward approach, where we ask the model to perform a task without any examples. This technique relies on the model's pre-trained knowledge and ability to understand instructions. For instance, when we ask, "Classify this movie review as positive or negative," we're assuming the model understands the concept of sentiment analysis without additional context. Zero-shot prompting works surprisingly well for many tasks, particularly with more advanced models, but it can sometimes lead to inconsistent results when dealing with complex or nuanced requests.

The best examples of Zero-Shot Prompting were the two prompts we used previously: „*Tell me about Paris hotels*,“ and the *structured prompt*. Although the structured prompt already produced a much better result than the original prompt, we can use other techniques to get even better results.

Few-shot prompting builds upon zero-shot prompting by providing examples within the prompt itself. This technique is especially effective when guiding the model to follow specific patterns or formats. By including a few illustrative examples, we can significantly enhance the model's ability to generate structured and contextually relevant responses. Unlike zero-shot prompting, where the model must infer the expected output from minimal instruction, few-shot prompting explicitly demonstrates the desired behavior. This method is particularly useful when working with tasks that require a structured response, such as summarization, classification, or information retrieval.

Example: Few-Shot Prompting for Hotel Descriptions

Let's apply few-shot prompting to the task of generating descriptions for hotels in Paris. The original zero-shot prompt, "Tell me about Paris hotels," does not specify the expected structure or style. Instead, by including a few examples, we can guide the model toward a more consistent and informative response.

Below is an improved version of the *generate_text* function, incorporating few-shot prompting:

```
def generate_text(prompt)

    few_shot_examples = """ #A

    Example 1:
    User: Tell me about New York hotels
    AI: The Plaza Hotel in New York is an iconic luxury hotel loc

    Example 2:
    User: Tell me about Tokyo hotels
    AI: The Park Hyatt Tokyo is a prestigious hotel known for its

    Now, following the same pattern, provide a response:
```

```

User: {prompt}
AI:
"".strip()

chat_completion = client.chat.completions.create(
    messages=[
        {"role": "user", "content": few_shot_examples.format(
    ],
    model=model,
    temperature=0.7,
    max_tokens=200,
)

return chat_completion.choices[0].message.content

prompt = "Tell me about Paris hotels"
response = generate_text(prompt) #B
print(response)

```

Here is the output. Note how the LLM used the examples in the prompt to create and structure the content of the response:

The Ritz Paris is a legendary luxury hotel situated in the heart

Why Use Few-Shot Prompting?

- Provides explicit guidance: The model sees clear examples of the desired response structure.
- Improves consistency: The output follows a defined pattern, reducing randomness.
- Reduces misinterpretation: The model understands the context more effectively, leading to better results.

By leveraging few-shot prompting, we can significantly enhance the reliability and coherence of AI-generated responses, especially in tasks that require domain-specific knowledge or structured output.

The key differences between zero-shot and few-shot prompting:

- Consistency: Few-shot prompting provides clear patterns that the model

can follow

- **Style Matching:** Examples help establish the desired tone and format
- **Completeness:** The model better understands the expected level of detail
- **Context Awareness:** Examples demonstrate how to handle nuanced situations

To get desired results, always start with zero-shot, then a few-shot or using chain-of-thought. If neither of them works, then fine-tune (more on this in later chapters).

Chain-of-Thought prompting represents a significant advancement in getting LLMs to solve complex problems by breaking them down into logical steps, similar to human reasoning. Let's explore this through comprehensive examples.

It refers to the model's ability to generate a sequence of coherent and logically connected ideas or responses based on a given prompt. LLMs excel at generating text that follows a natural flow of ideas, leveraging their training on vast amounts of text data to predict and generate contextually relevant sequences.

[Zero-shot CoT](#) involves appending „Let's think step by step,“ to the original prompt, utilizing two prompts to derive reasoning and answers.

- **Reasoning Extraction:** In this step, the language model generates a chain of reasoning leading to the answer. The model is given a prompt that includes the question and the trigger sentence, „Let's think step by step,“ The model then produces a sentence explaining the reasoning process.
- **Answer Extraction:** In the second step, we extract the final answer from the model's response. We concatenate the prompt, the generated reasoning sentence, and the trigger sentence, „The answer is,“ This instructs the model to provide the final answer, which it generates in response.

Figure 5.10, demonstrates two distinct approaches to prompting a language model to solve a word problem. In the first approach, the model is directly presented with the word problem and is expected to generate the answer

without any explicit instructions or guidance. This is akin to giving the model a raw question and expecting it to solve it directly. However, in the second example (right), we introduce a subtle but crucial modification. Before the model attempts to solve the problem, it is prompted with a simple instruction: "Let's think step by step." This seemingly minor addition encourages the model to break down the problem into smaller, more manageable steps, mimicking human reasoning.

The results below show that while the Zero-shot approach struggles, producing an incorrect answer 8, the Zero-shot-CoT approach, with the simple "Let's think step by step" prompt, guides the model to correctly solve the problem by reasoning through the steps.

Figure 5.10 In this example we explore two approaches to prompting a language model to solve a word problem: reasoning extraction, where the model explains its steps, and answer extraction, where the model directly provides the numerical answer. [\[Source\]](#)

(a) Few-shot

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The answer is 8. **X**

(b) Few-shot-CoT

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The juggler can juggle 16 balls. Half of the balls are golf balls. So there are $16 / 2 = 8$ golf balls. Half of the golf balls are blue. So there are $8 / 2 = 4$ blue golf balls. The answer is 4. **✓**

(c) Zero-shot

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: The answer (arabic numerals) is

(Output) 8 **X**

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.**

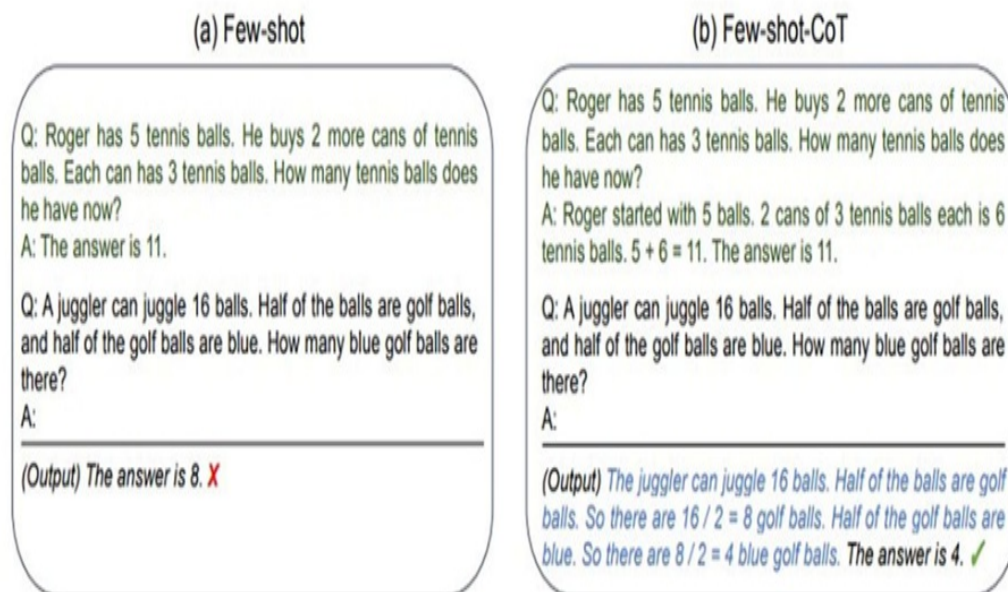
(Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. **✓**

The method demonstrates two distinct approaches to prompting a language model to solve a word problem. In the first approach, labeled "Reasoning Extraction," the prompt asks the model to break down the problem and explain its reasoning step-by-step. The model then outputs a series of statements outlining its thought process, leading to the final answer. In contrast, the second approach, "Answer Extraction," presents the problem with an incomplete answer and prompts the model to fill in the blank. This method directly elicits the numerical solution from the model, bypassing the

need for an explicit reasoning explanation. Both methods leverage the language model's capabilities to solve mathematical problems, but they differ in their focus: one emphasizes understanding and explanation, while the other prioritizes direct result extraction.

Another technique, [Few-shot CoT](#) is better at improving how well LLMs reason than the few-shot baseline because it gives them examples of similar problems to learn from. It can be harder to set up because you need to create example prompts. However, the benefits of few-shot CoT are worth the complexity. In short Few-Shot CoT builds on Few-Shot prompting, enables the same process of „Thinking step by step,“.

Figure 5.11 In this example we explore adding a simple "Let's think step by step" instruction to Few-shot prompts can significantly improve a language model's ability to solve word problems by encouraging it to reason through the steps. (image source: <https://arxiv.org/pdf/2201.11903>)



Applying CoT to Our Hotel Search Application

Now, we showcase how CoT can be utilized in our application, *Travelle*, which we introduced in Chapter 4. From our previous semantic search, we had a huge list of hotels from our query: „Hotel near the Louvre with great food nearby,“ While the search results were relevant, they were overwhelming and difficult to process without additional analysis.

The huge output looked as follows:

```
Faiss search time: 0.0016 seconds Query: Hotel near the Louvre wi
----- results truncated due to a huge output.
```

Here's how we can transform those raw search results into actionable insights using Chain-of-Thought prompting:

```
def analyze_hotel_recommendations(query, results, model="gpt-4o")

    prompt = f""" #A

    Analyze the following hotel search results and answer these que

    1. Which hotel best matches the user's query: '{query}'? Consi
    2. What are the pros and cons of the top 3 hotels? Support you
    3. Are there any recurring themes or patterns in the positive/n
    4. Summarize the top 3 hotels in bullet points highlighting the
    5. Based on the provided data, provide an overall recommendatio

    Hotel Search Results:
    {results} #C
    """

    # Use OpenAI API to get the analysis
    chat_completion = client.chat.completions.create(
        messages=[
            {"role": "user", "content": prompt}
        ],
        model=model,
        temperature=0.7,
        max_tokens=1500,
    )

    analysis = chat_completion.choices[0].message.content
    return analysis
```

```
analysis = analyze_hotel_recommendations(hotel_query, result) #D
display(Markdown(analysis))
```

Here,Âs the output from the function above:

1. Best Match for 'Hotel near the Louvre with great food nearby'

Winner: Grand Hotel du Palais Royal

- ,Ä Location: Directly next to the Louvre, offering unparalleled
- ,Ä Food Options: The area around the Louvre is filled with res
- ,Ä Overall Experience: Clean, excellent service, and accommoda

Close Contenders:

- ,Ä Hotel Malte - Astotel (Also near the Louvre, great food opt
- ,Ä Hotel Moliere (Excellent location and breakfast, though roo

2. Pros and Cons of the Top 3 Hotels

,Ä 1. Grand Hotel du Palais Royal

Pros:

- ,ò ,ÄGreat hotel located in the best part of town just next to
- ,ò ,ÄVery clean and the service was wonderful.,Ä (High-qualit
- ,ò ,ÄAlmost all of the staff spoke English.,Ä (Good for inter

Cons:

- ,ò ,ÄThe spa is small though which can make things awkward if
- ,Ä 2. Hotel Malte - Astotel

Pros:

- ,ò ,ÄIdeal location for The Louvre, city centre shops, and ple
- ,ò ,ÄThere are numerous restaurants and cafes close by.,Ä (Pl
- ,ò ,ÄThe room was very comfortable... the bed was as good as y
- ,ò ,ÄMini bar fridge with soft drinks and water in the room at

Cons:

- ,ò ,ÄThere is no restaurant for evening meals and no bar in th
- ,Ä 3. Hotel Moliere

Pros:

- ,ò ,ÄExcellent location near Louvre and other key sites.,Ä (P
- ,ò ,ÄWe really enjoyed the helpfulness of the staff.,Ä (Frien
- ,ò ,ÄDelicious breakfast and refreshments offered.,Ä (Good fo

Cons:

- ,ò ,ÄRooms have good amenities but are small and varied.,Ä (S
- ,ò ,ÄWe didn't love the bathroom layout with a glass wall that

3. Recurring Themes in Positive & Negative Reviews

,ò Positive Themes:

- ,Ä Location: Most hotels are praised for being near the Louvre
- ,Ä Staff Service: Many reviews highlight friendly, helpful, an
- ,Ä Cleanliness: Guests frequently mention spotless rooms and c
- ,Ä Extras: Complimentary minibar, free afternoon snacks, and g

,ò Negative Themes:

- ,Ä¢ Small Rooms: A common Paris issue,Ä¢many reviews mention co
- ,Ä¢ Bathroom Design: Glass walls and lack of privacy are noted
- ,Ä¢ Limited In-Hotel Dining: Some hotels don,Ä¢t offer evening

4. Summary of Top 3 Hotels

,Ä¢